🎨 Introduction to Seaborn

Seaborn is a **Python data visualization library** built on top of Matplotlib. It provides a **high-level interface** for creating **attractive and informative statistical graphics** with less code.

🔹 Why Use Seaborn?

- Built-in themes for visually appealing plots.
- Simplifies complex visualizations like heatmaps, violin plots, and pair plots.
- Works seamlessly with Pandas DataFrames.
- Provides statistical insights directly in plots.
- Supports automatic estimation and visualization of regression models.

🔹 Common Use Cases

✔ Exploratory Data Analysis (EDA)
✔ Statistical Data Visualization
✔ Relationship & Distribution Analysis
✔ Heatmaps & Correlation Analysis

📌 *Let's explore Seaborn to create insightful visualizations!* 🚀

Seaborn Roadmap

Types of Functions

- Figure Level
- Axis Level

Main Classification

- Relational Plot
- Distribution Plot
- Categorical Plot
- Regression Plot
- Matrix Plot
- Multiplots

https://seaborn.pydata.org/api.html

1. Relational Plot

- to see the statistical relation between 2 or more variables.
- Bivariate Analysis

Plots under this section

- scatterplot
- lineplot

In [1]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
tips = sns.load_dataset('tips')
tips
```

Out[2]:

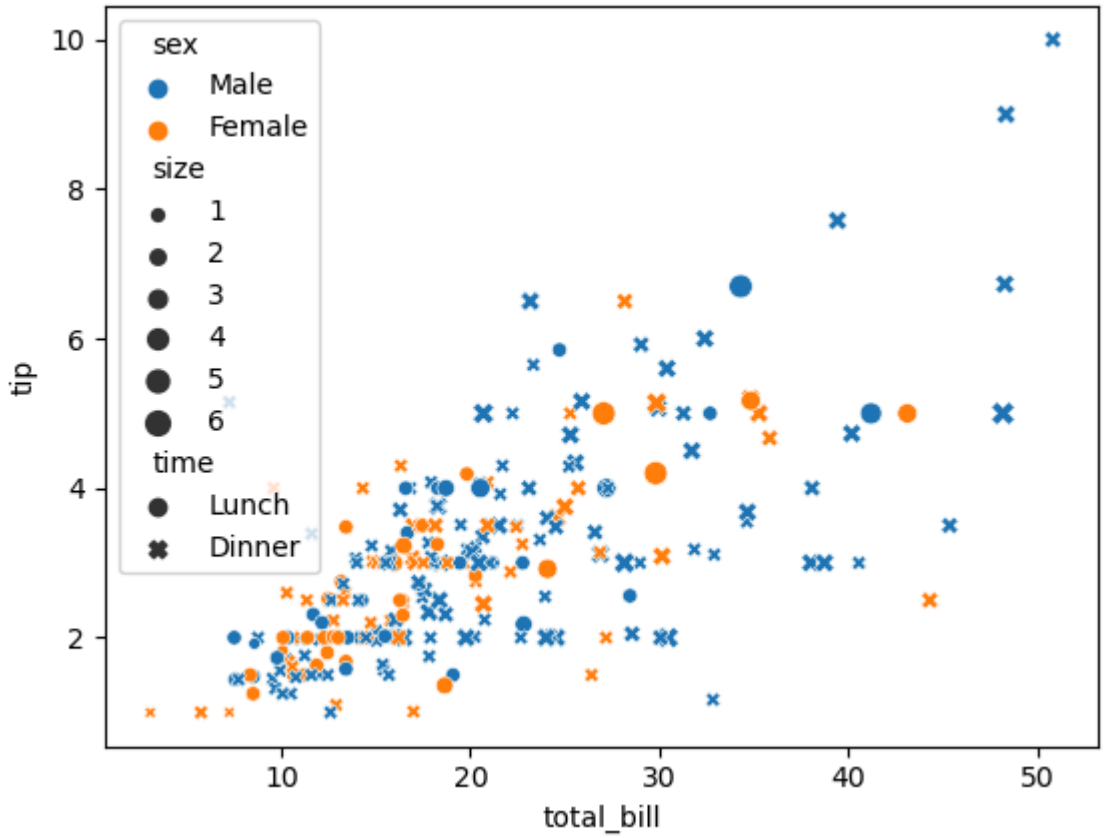
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

In [3]:
```
# scatter plot -> axes level function

sns.scatterplot(data = tips, x = 'total_bill', y = 'tip',hue = 'sex',
                style = 'time',size = 'size')
```

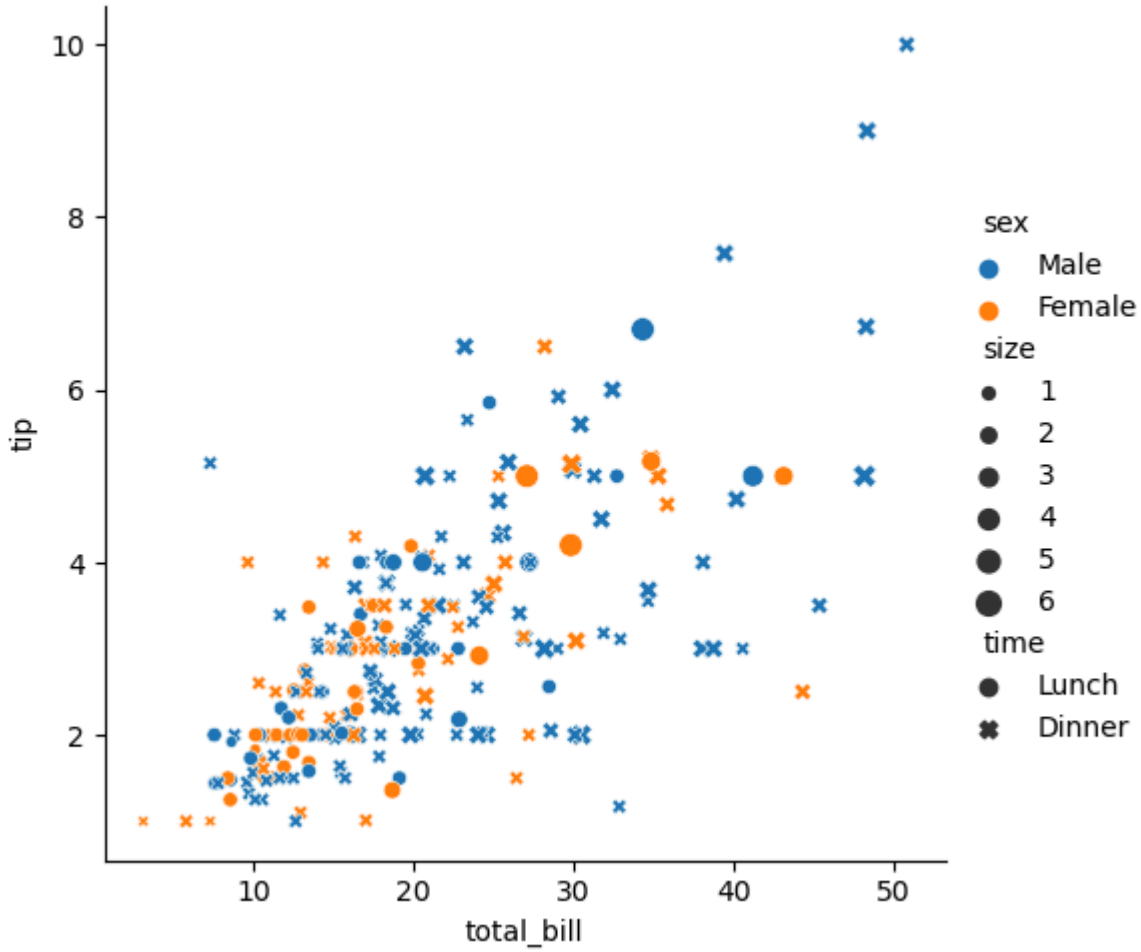
Out[3]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



In [4]:
```
# relplot -> figure level -> square shape

sns.relplot(data=tips, x='total_bill', y='tip', kind='scatter', hue='sex',
            style='time', size='size')
```

Out[4]: <seaborn.axisgrid.FacetGrid at 0x26e91ec7f50>



In [5]:
```
# line plot

gap = px.data.gapminder()
temp_df = gap[gap['country'] == 'India']
temp_df
```

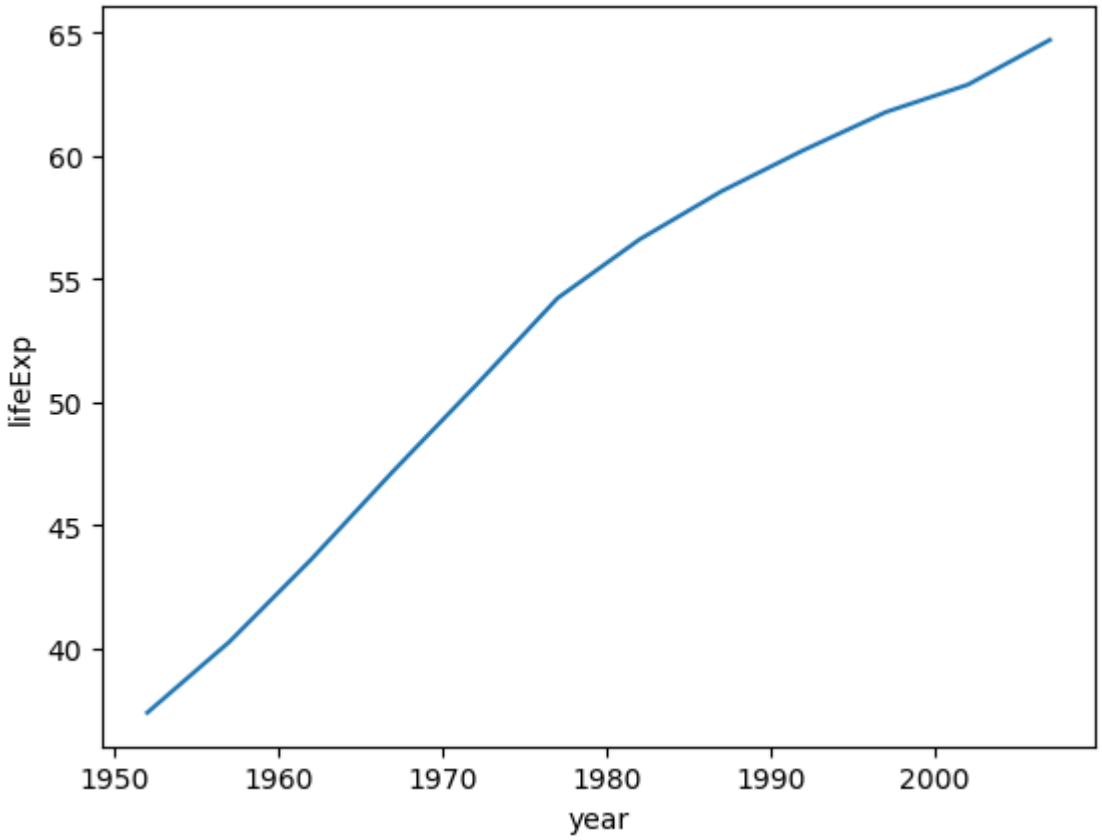
Out[5]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
696	India	Asia	1952	37.373	372000000	546.565749	IND	356
697	India	Asia	1957	40.249	409000000	590.061996	IND	356
698	India	Asia	1962	43.605	454000000	658.347151	IND	356
699	India	Asia	1967	47.193	506000000	700.770611	IND	356
700	India	Asia	1972	50.651	567000000	724.032527	IND	356
701	India	Asia	1977	54.208	634000000	813.337323	IND	356
702	India	Asia	1982	56.596	708000000	855.723538	IND	356
703	India	Asia	1987	58.553	788000000	976.512676	IND	356
704	India	Asia	1992	60.223	872000000	1164.406809	IND	356
705	India	Asia	1997	61.765	959000000	1458.817442	IND	356
706	India	Asia	2002	62.879	1034172547	1746.769454	IND	356
707	India	Asia	2007	64.698	1110396331	2452.210407	IND	356

In [6]:
```
# axes level function

sns.lineplot(data = temp_df, x = 'year', y = 'lifeExp')
```

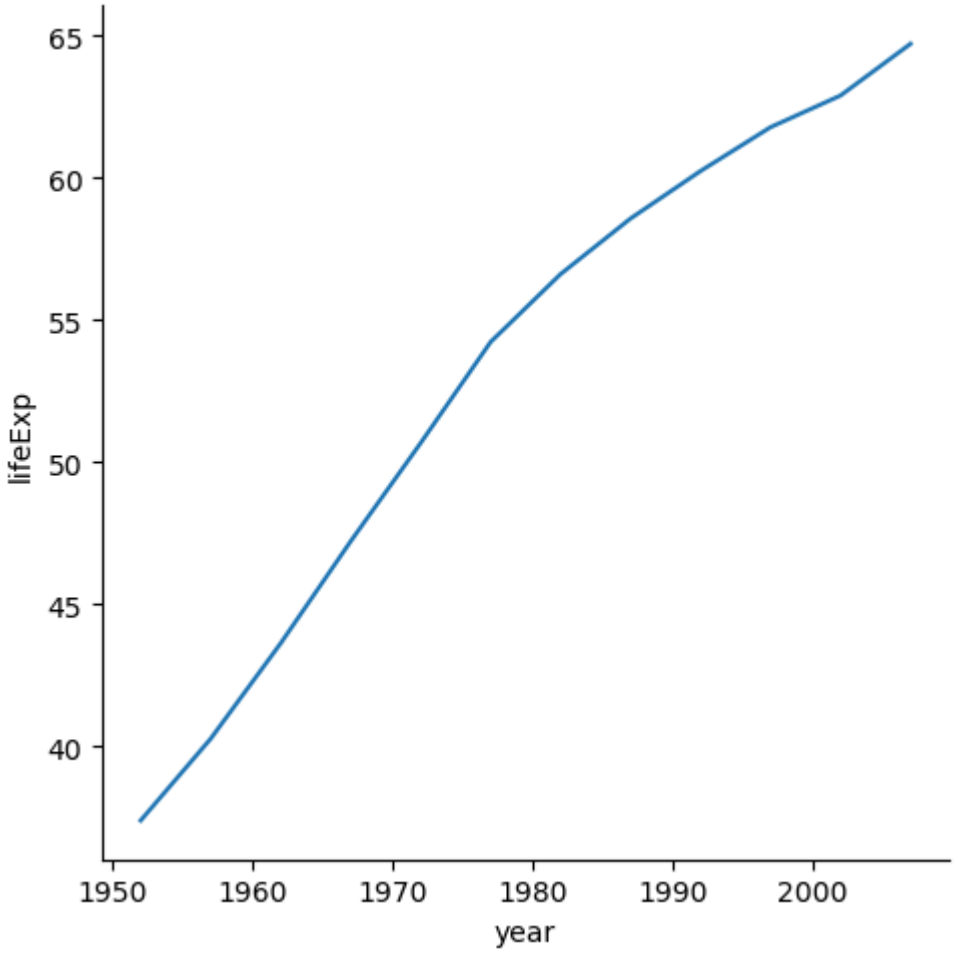
Out[6]: <Axes: xlabel='year', ylabel='lifeExp'>



In [7]:
```
# using Relplot

sns.relplot(data = temp_df, x = 'year', y = 'lifeExp',kind = 'line')
```

Out[7]: <seaborn.axisgrid.FacetGrid at 0x26e920153d0>



In [8]:
```
# hue -->> style

temp_df = gap[gap['country'].isin(['India','Pakistan','China'])]
temp_df
```

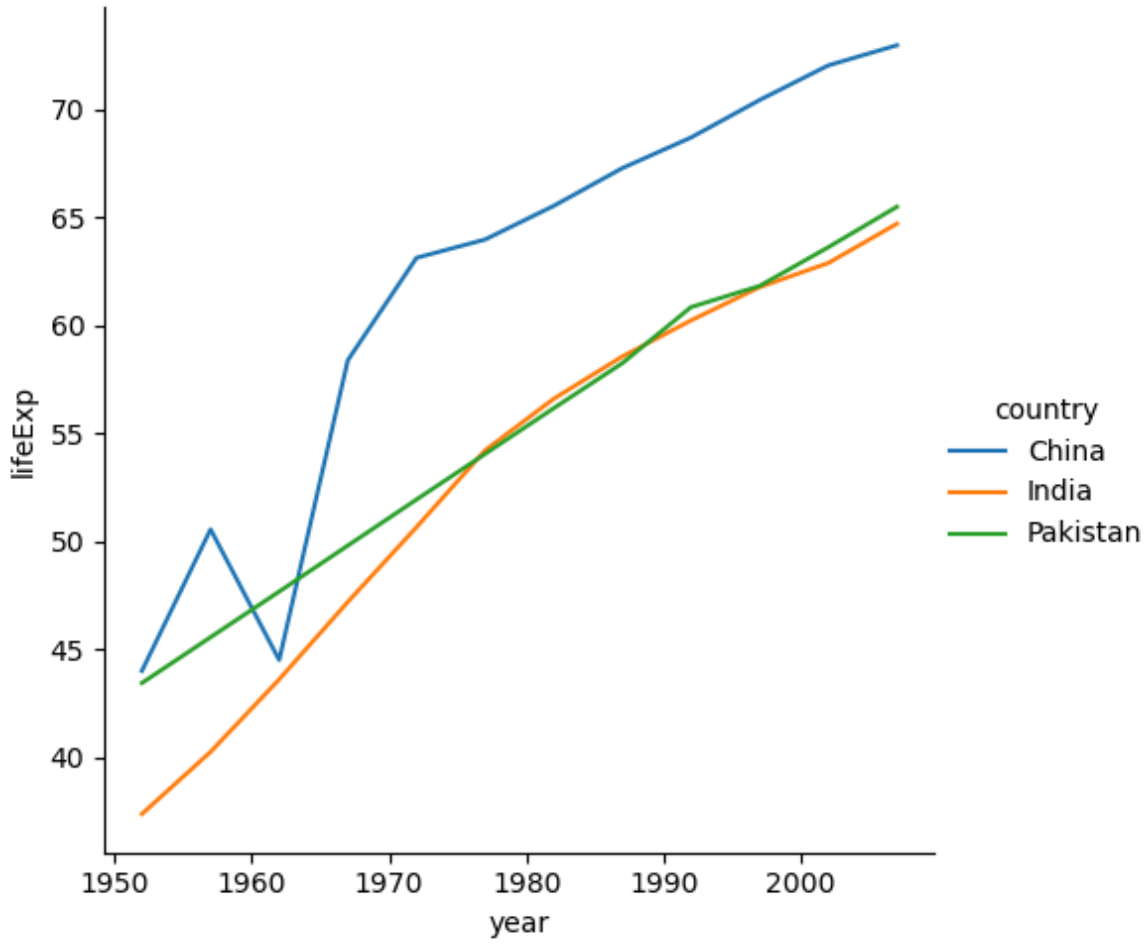
Out[8]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
288	China	Asia	1952	44.00000	556263527	400.448611	CHN	156
289	China	Asia	1957	50.54896	637408000	575.987001	CHN	156
290	China	Asia	1962	44.50136	665770000	487.674018	CHN	156
291	China	Asia	1967	58.38112	754550000	612.705693	CHN	156
292	China	Asia	1972	63.11888	862030000	676.900092	CHN	156
293	China	Asia	1977	63.96736	943455000	741.237470	CHN	156
294	China	Asia	1982	65.52500	1000281000	962.421381	CHN	156
295	China	Asia	1987	67.27400	1084035000	1378.904018	CHN	156
296	China	Asia	1992	68.69000	1164970000	1655.784158	CHN	156
297	China	Asia	1997	70.42600	1230075000	2289.234136	CHN	156
298	China	Asia	2002	72.02800	1280400000	3119.280896	CHN	156
299	China	Asia	2007	72.96100	1318683096	4959.114854	CHN	156
696	India	Asia	1952	37.37300	372000000	546.565749	IND	356
697	India	Asia	1957	40.24900	409000000	590.061996	IND	356
698	India	Asia	1962	43.60500	454000000	658.347151	IND	356
699	India	Asia	1967	47.19300	506000000	700.770611	IND	356
700	India	Asia	1972	50.65100	567000000	724.032527	IND	356
701	India	Asia	1977	54.20800	634000000	813.337323	IND	356
702	India	Asia	1982	56.59600	708000000	855.723538	IND	356
703	India	Asia	1987	58.55300	788000000	976.512676	IND	356
704	India	Asia	1992	60.22300	872000000	1164.406809	IND	356
705	India	Asia	1997	61.76500	959000000	1458.817442	IND	356
706	India	Asia	2002	62.87900	1034172547	1746.769454	IND	356
707	India	Asia	2007	64.69800	1110396331	2452.210407	IND	356
1164	Pakistan	Asia	1952	43.43600	41346560	684.597144	PAK	586
1165	Pakistan	Asia	1957	45.55700	46679944	747.083529	PAK	586
1166	Pakistan	Asia	1962	47.67000	53100671	803.342742	PAK	586
1167	Pakistan	Asia	1967	49.80000	60641899	942.408259	PAK	586
1168	Pakistan	Asia	1972	51.92900	69325921	1049.938981	PAK	586
1169	Pakistan	Asia	1977	54.04300	78152686	1175.921193	PAK	586
1170	Pakistan	Asia	1982	56.15800	91462088	1443.429832	PAK	586
1171	Pakistan	Asia	1987	58.24500	105186881	1704.686583	PAK	586
1172	Pakistan	Asia	1992	60.83800	120065004	1971.829464	PAK	586

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
1173	Pakistan	Asia	1997	61.81800	135564834	2049.350521	PAK	586
1174	Pakistan	Asia	2002	63.61000	153403524	2092.712441	PAK	586
1175	Pakistan	Asia	2007	65.48300	169270617	2605.947580	PAK	586

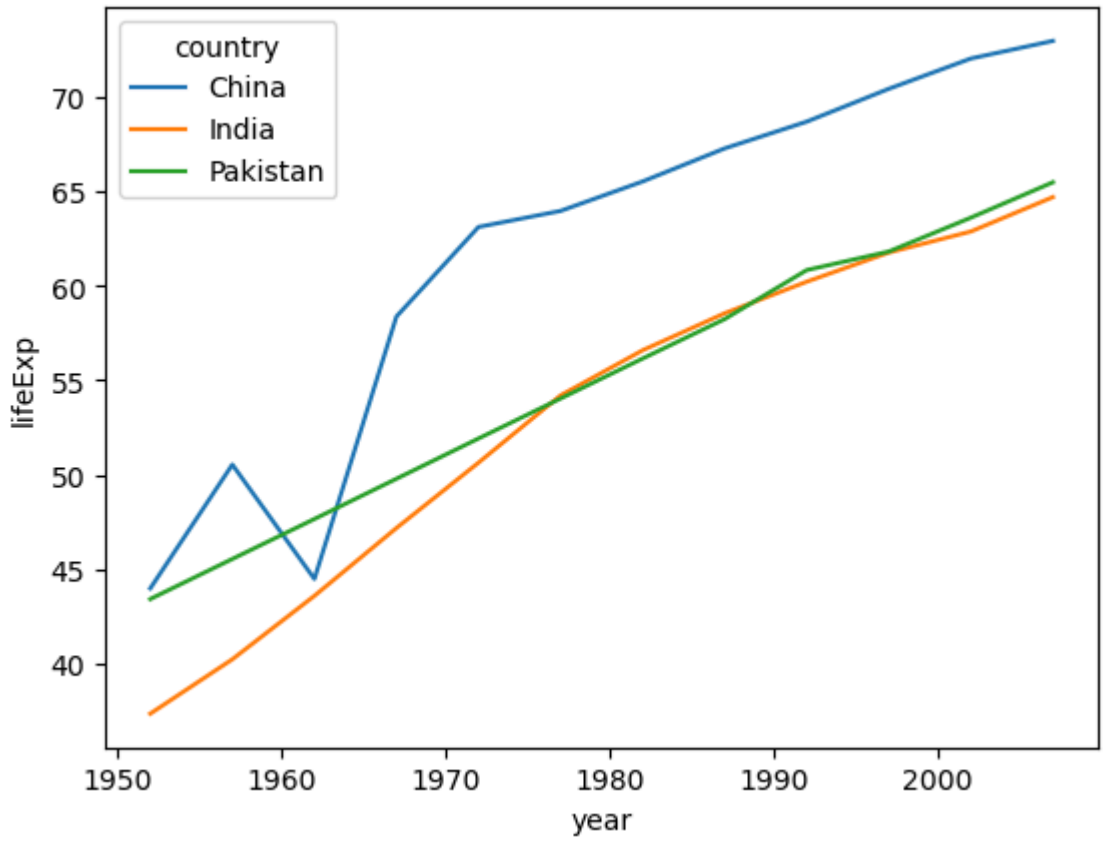
In [9]:
```
sns.relplot(kind = 'line', data = temp_df, x = 'year',y = 'lifeExp',
            hue = 'country')
```

Out[9]: <seaborn.axisgrid.FacetGrid at 0x26e9209d010>



In [10]:
```
sns.lineplot(data = temp_df,x = 'year', y = 'lifeExp',hue = 'country')
```

Out[10]: <Axes: xlabel='year', ylabel='lifeExp'>



In [11]:
```
temp_df = gap[gap['country'].isin(['India','Brazil','Germany'])]
temp_df
```

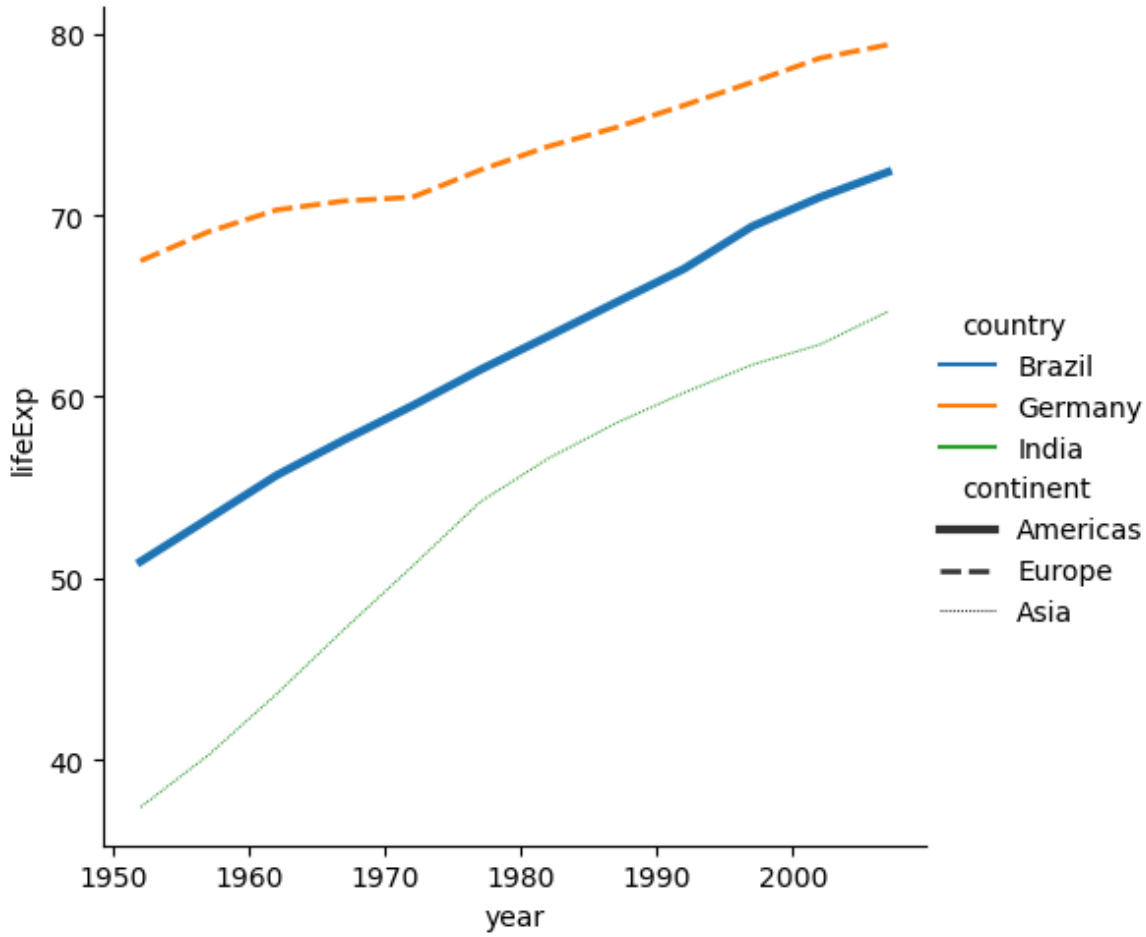
Out[11]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
168	Brazil	Americas	1952	50.917	56602560	2108.944355	BRA	76
169	Brazil	Americas	1957	53.285	65551171	2487.365989	BRA	76
170	Brazil	Americas	1962	55.665	76039390	3336.585802	BRA	76
171	Brazil	Americas	1967	57.632	88049823	3429.864357	BRA	76
172	Brazil	Americas	1972	59.504	100840058	4985.711467	BRA	76
173	Brazil	Americas	1977	61.489	114313951	6660.118654	BRA	76
174	Brazil	Americas	1982	63.336	128962939	7030.835878	BRA	76
175	Brazil	Americas	1987	65.205	142938076	7807.095818	BRA	76
176	Brazil	Americas	1992	67.057	155975974	6950.283021	BRA	76
177	Brazil	Americas	1997	69.388	168546719	7957.980824	BRA	76
178	Brazil	Americas	2002	71.006	179914212	8131.212843	BRA	76
179	Brazil	Americas	2007	72.390	190010647	9065.800825	BRA	76
564	Germany	Europe	1952	67.500	69145952	7144.114393	DEU	276
565	Germany	Europe	1957	69.100	71019069	10187.826650	DEU	276
566	Germany	Europe	1962	70.300	73739117	12902.462910	DEU	276
567	Germany	Europe	1967	70.800	76368453	14745.625610	DEU	276
568	Germany	Europe	1972	71.000	78717088	18016.180270	DEU	276
569	Germany	Europe	1977	72.500	78160773	20512.921230	DEU	276
570	Germany	Europe	1982	73.800	78335266	22031.532740	DEU	276
571	Germany	Europe	1987	74.847	77718298	24639.185660	DEU	276
572	Germany	Europe	1992	76.070	80597764	26505.303170	DEU	276
573	Germany	Europe	1997	77.340	82011073	27788.884160	DEU	276
574	Germany	Europe	2002	78.670	82350671	30035.801980	DEU	276
575	Germany	Europe	2007	79.406	82400996	32170.374420	DEU	276
696	India	Asia	1952	37.373	372000000	546.565749	IND	356
697	India	Asia	1957	40.249	409000000	590.061996	IND	356
698	India	Asia	1962	43.605	454000000	658.347151	IND	356
699	India	Asia	1967	47.193	506000000	700.770611	IND	356
700	India	Asia	1972	50.651	567000000	724.032527	IND	356
701	India	Asia	1977	54.208	634000000	813.337323	IND	356
702	India	Asia	1982	56.596	708000000	855.723538	IND	356
703	India	Asia	1987	58.553	788000000	976.512676	IND	356
704	India	Asia	1992	60.223	872000000	1164.406809	IND	356

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
705	India	Asia	1997	61.765	959000000	1458.817442	IND	356
706	India	Asia	2002	62.879	1034172547	1746.769454	IND	356
707	India	Asia	2007	64.698	1110396331	2452.210407	IND	356

In [12]:
```
sns.relplot(kind='line', data=temp_df, x='year', y='lifeExp', hue='country',
            style='continent', size='continent')
```

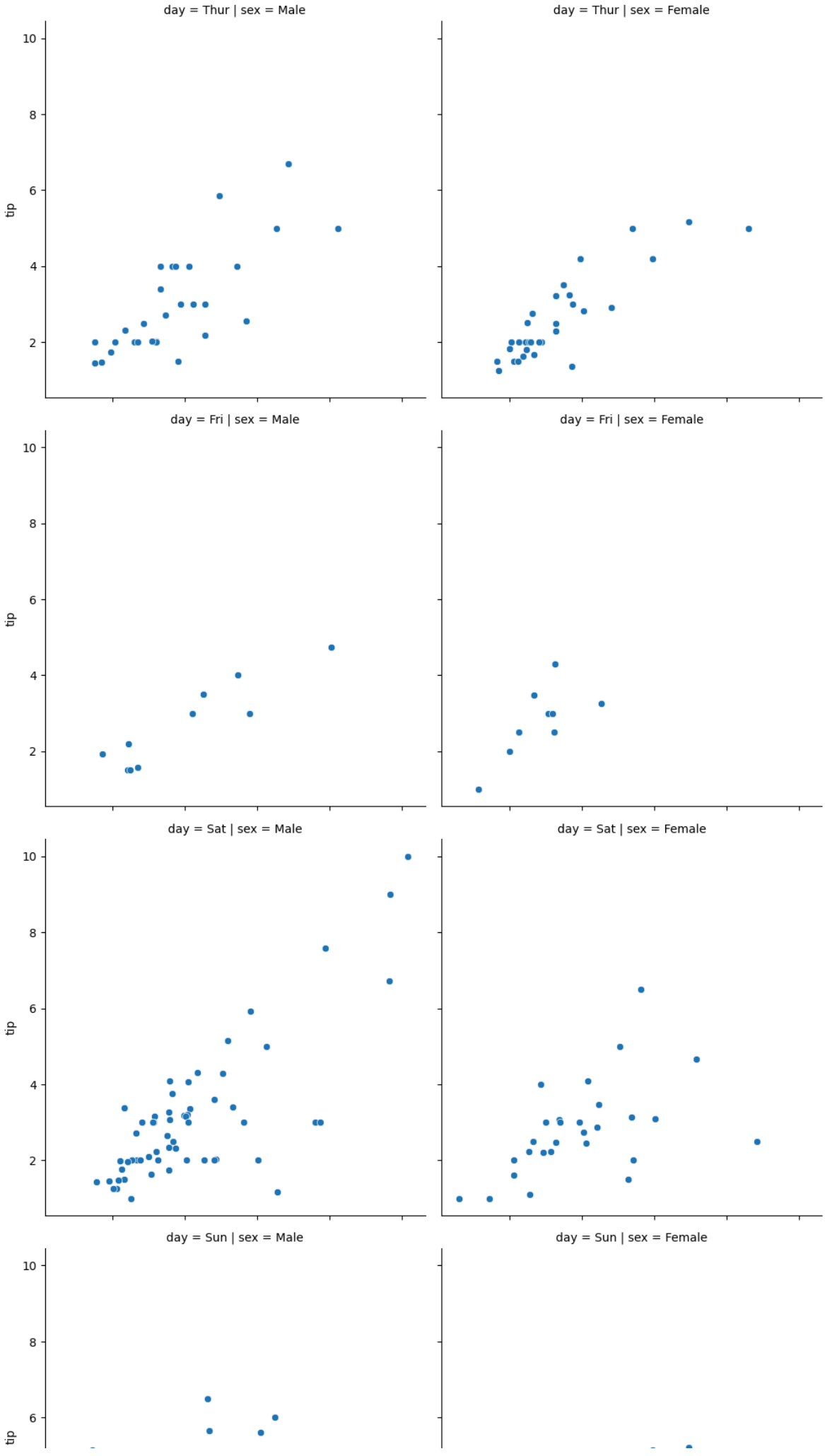
Out[12]: <seaborn.axisgrid.FacetGrid at 0x26e92009690>

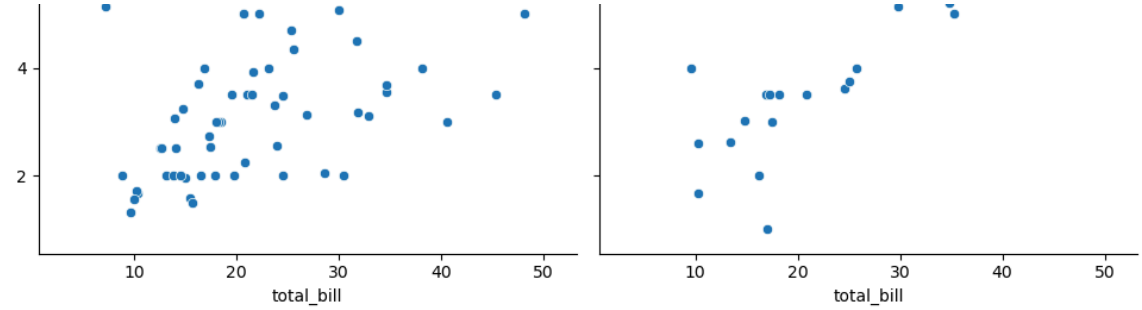


In [13]:
```
# facet plot -> figure level function -> work with relplot
# it will not work with scatterplot and lineplot

sns.relplot(data=tips, x='total_bill', y='tip', kind='scatter', col='sex',
            row='day')
```

Out[13]: <seaborn.axisgrid.FacetGrid at 0x26e932c0d50>

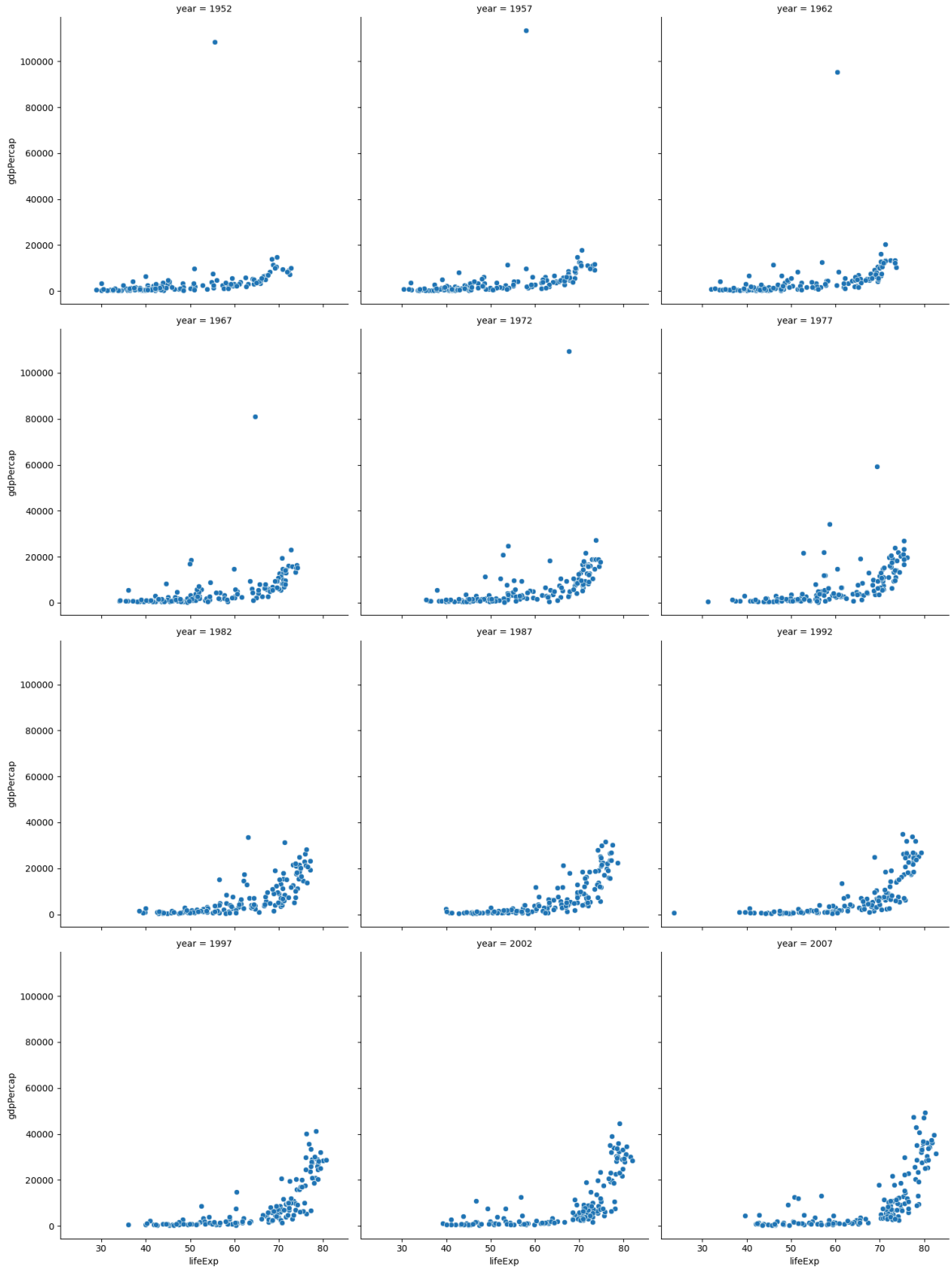

day = Thur | sex = Male
day = Thur | sex = Female
day = Fri | sex = Male
day = Fri | sex = Female
day = Sat | sex = Male
day = Sat | sex = Female
day = Sun | sex = Male
day = Sun | sex = Female
tip
10
8
6
4
2



In [14]:
```
# col wrap

sns.relplot(data = gap, x = 'lifeExp', y = 'gdpPercap',kind = 'scatter',
            col = 'year',col_wrap = 3)
```

Out[14]: <seaborn.axisgrid.FacetGrid at 0x26e93410a50>



2. Distribution Plots

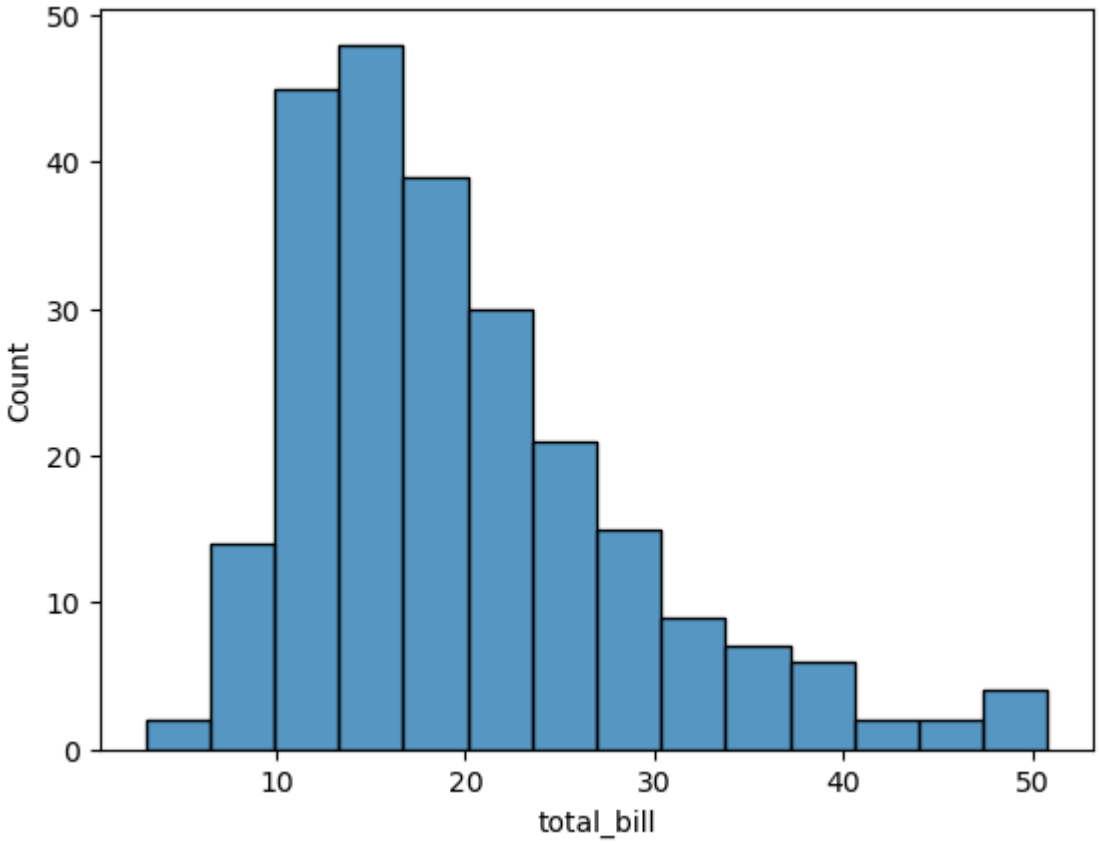
- used for univariate analysis
- used to find out the distribution
- Range of the observation
- Central Tendency
- is the data bimodal?
- Are there outliers?

Plots under distribution plot

- histplot
- kdeplot
- rugplot

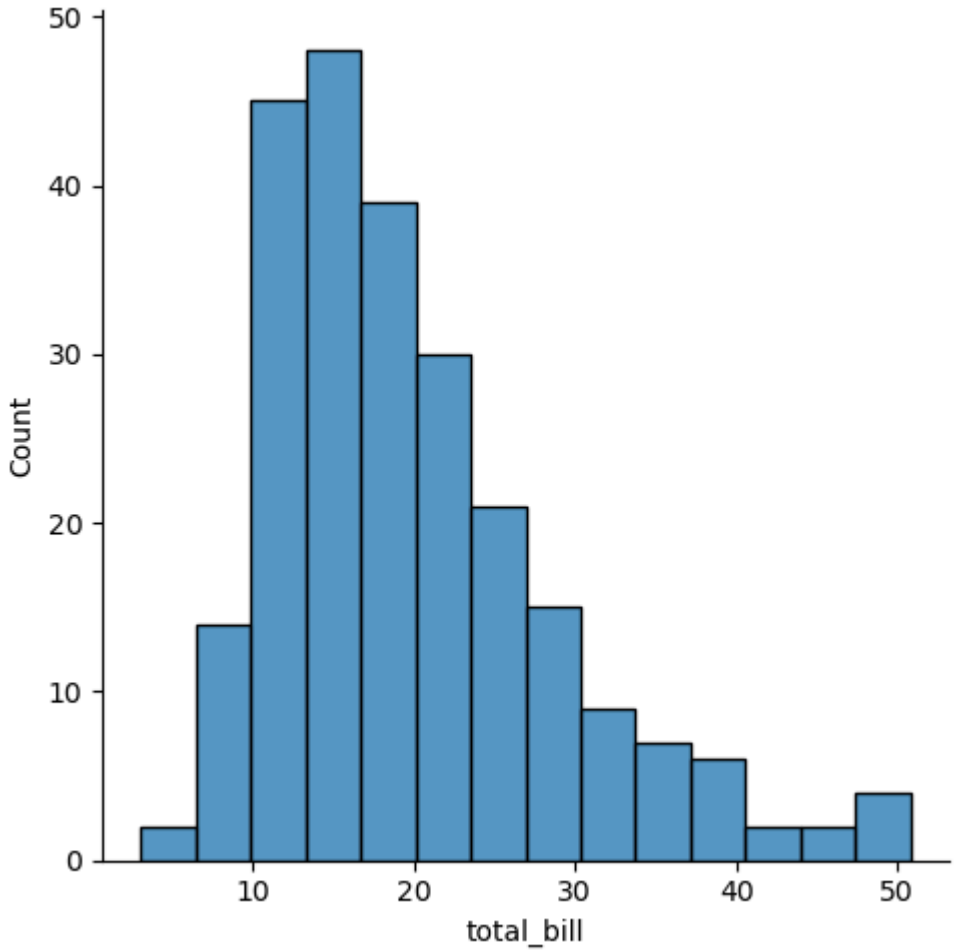
In [15]:
```
# plotting univariate histogram
sns.histplot(data=tips, x='total_bill')
```

Out[15]:
```
<Axes: xlabel='total_bill', ylabel='Count'>
```



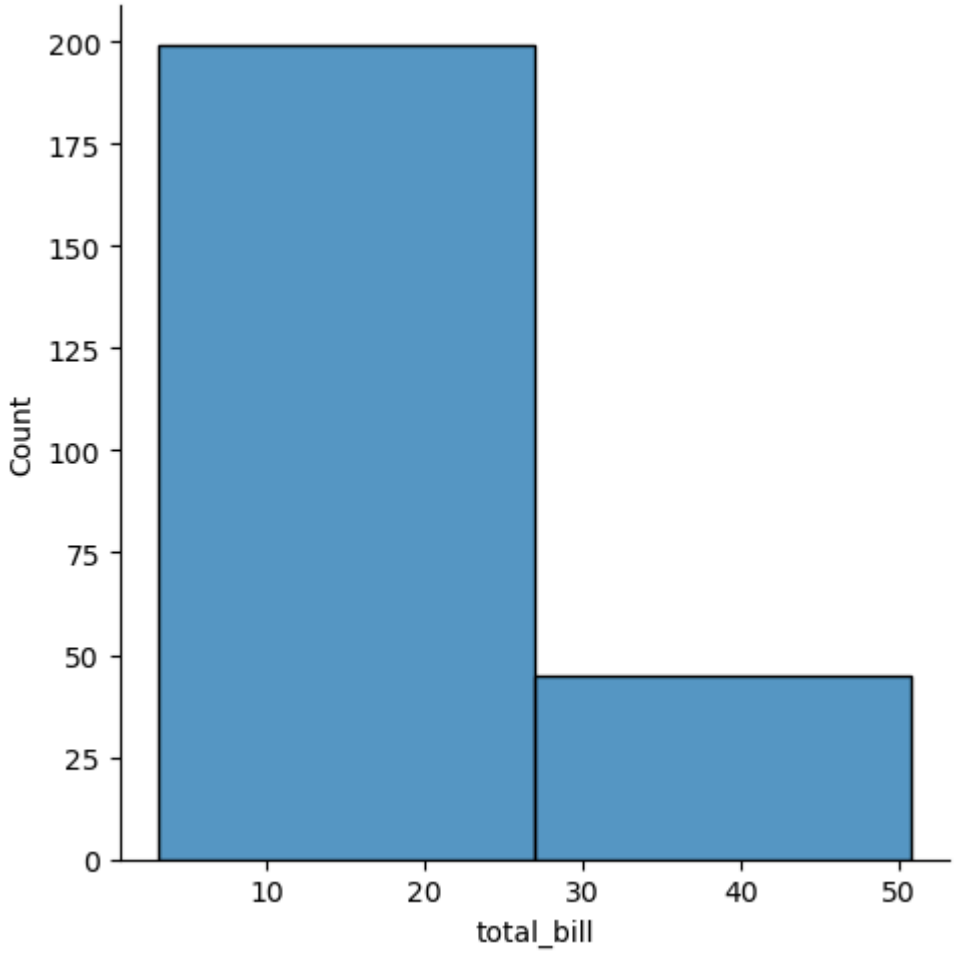
In [16]:
```
sns.displot(data=tips, x='total_bill', kind='hist')
```

Out[16]:
```
<seaborn.axisgrid.FacetGrid at 0x26e958a6a10>
```



In [17]:
```
# bins parameter
sns.displot(data=tips, x='total_bill', kind='hist',bins=2)
```

Out[17]:
```
<seaborn.axisgrid.FacetGrid at 0x26e959519d0>
```

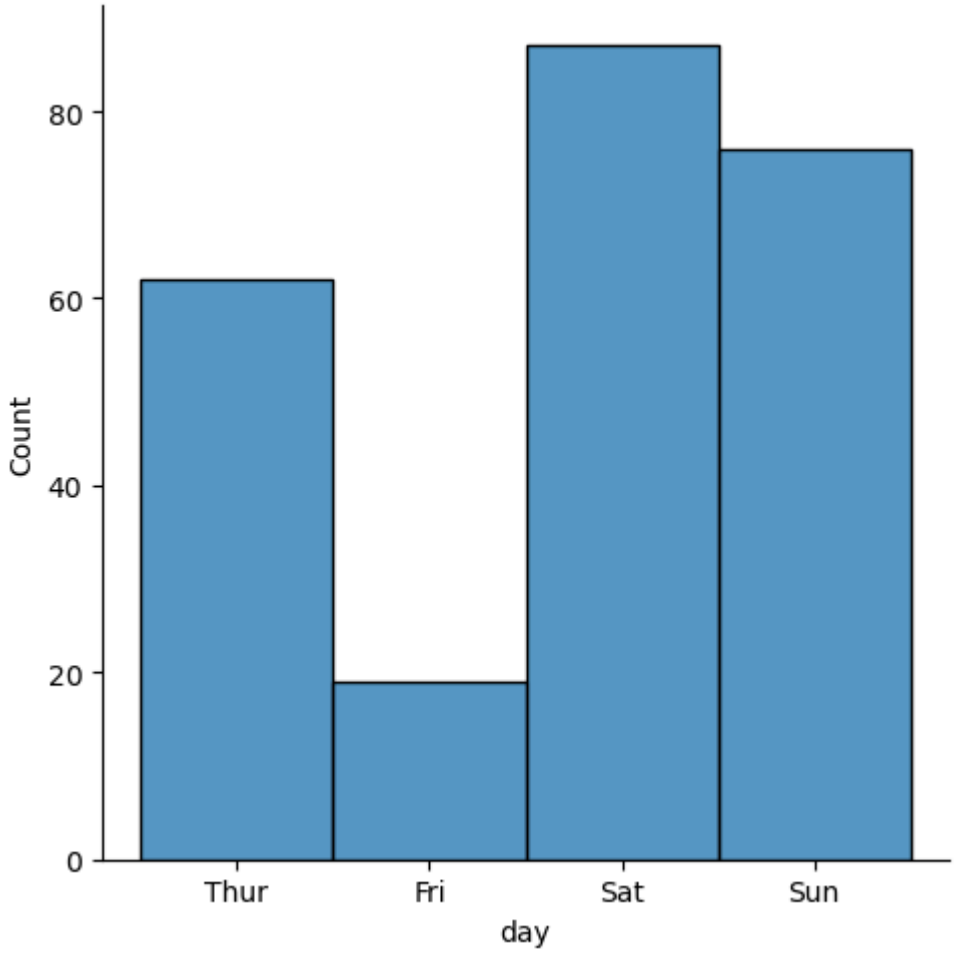


In [18]:
```
# It's also possible to visualize the distribution of a categorical variable
#                                                 using the logic of a histogram

# Discrete bins are automatically set for categorical variables

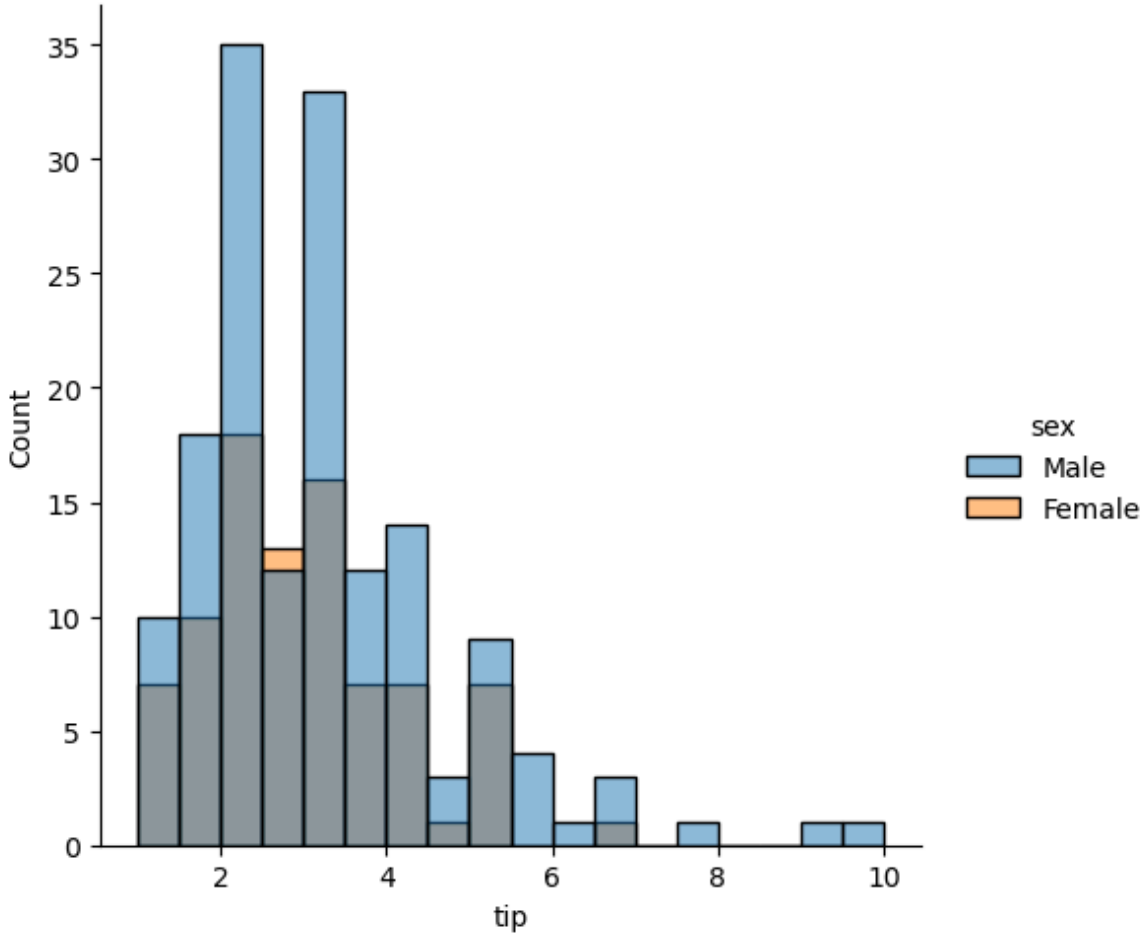
# countplot
sns.displot(data=tips, x='day', kind='hist')
```

Out[18]: <seaborn.axisgrid.FacetGrid at 0x26e966f3d90>



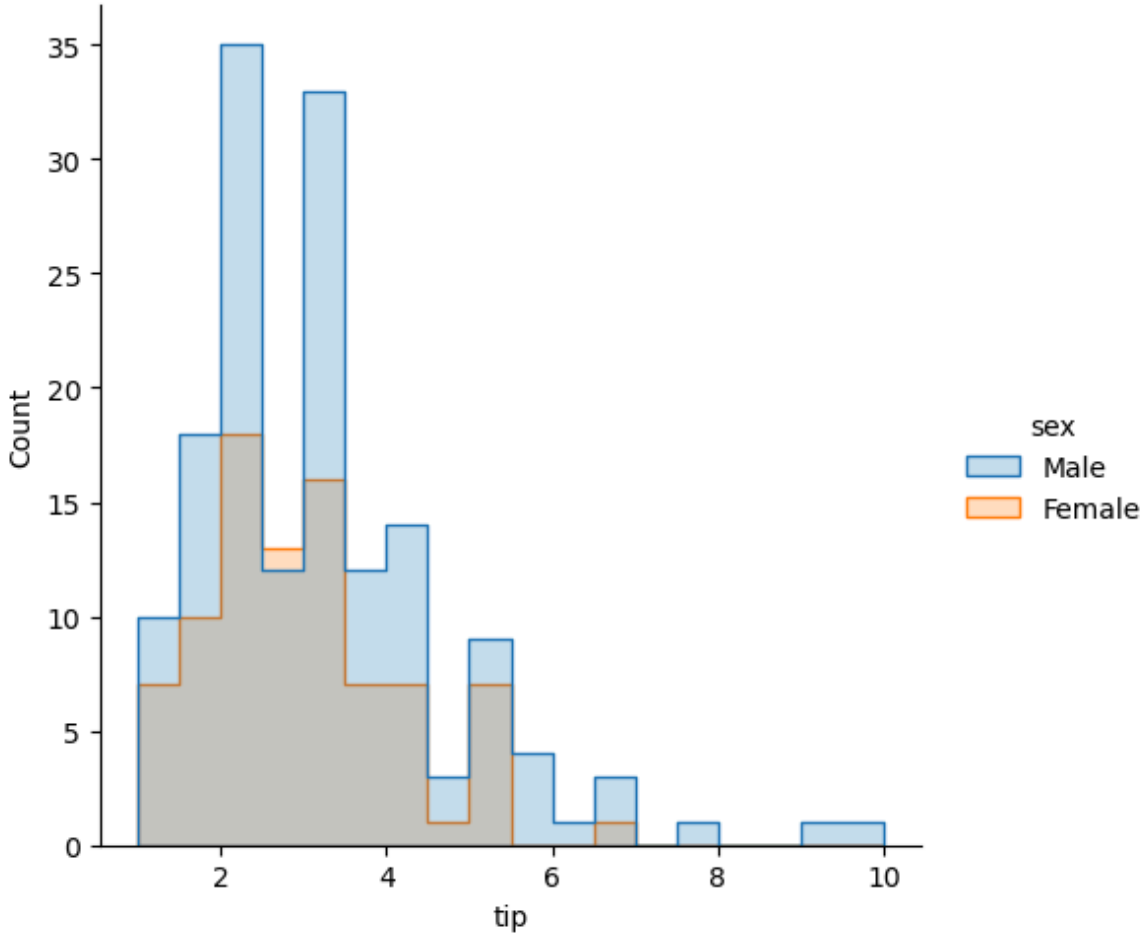
In [19]:
```
# hue parameter
sns.displot(data=tips, x='tip', kind='hist',hue='sex')
```

Out[19]:
```
<seaborn.axisgrid.FacetGrid at 0x26e93883e50>
```



In [20]:
```
# element -> step
sns.displot(data=tips, x='tip', kind='hist',hue='sex',element='step')
```

Out[20]:
```
<seaborn.axisgrid.FacetGrid at 0x26e96636650>
```



In [21]:
```
titanic = sns.load_dataset('titanic')
titanic
```

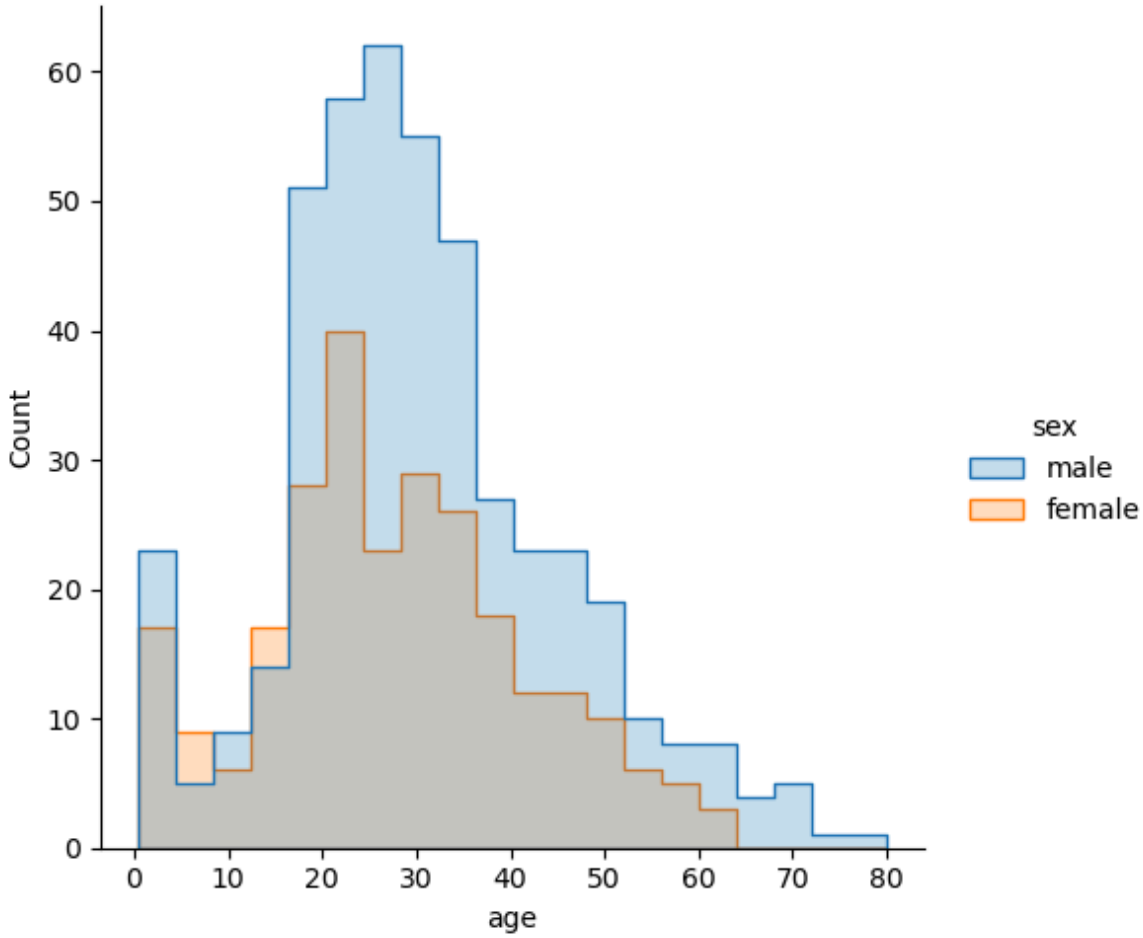
Out[21]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
0	0	3	male	22.0	1	0	7.2500	S	Third	man
1	1	1	female	38.0	1	0	71.2833	C	First	woman
2	1	3	female	26.0	0	0	7.9250	S	Third	woman
3	1	1	female	35.0	1	0	53.1000	S	First	woman
4	0	3	male	35.0	0	0	8.0500	S	Third	man
...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man
887	1	1	female	19.0	0	0	30.0000	S	First	woman
888	0	3	female	NaN	1	2	23.4500	S	Third	woman
889	1	1	male	26.0	0	0	30.0000	C	First	man
890	0	3	male	32.0	0	0	7.7500	Q	Third	man

891 rows × 15 columns

In [22]:
```
sns.displot(data=titanic, x='age', kind='hist',element='step',hue='sex')
```

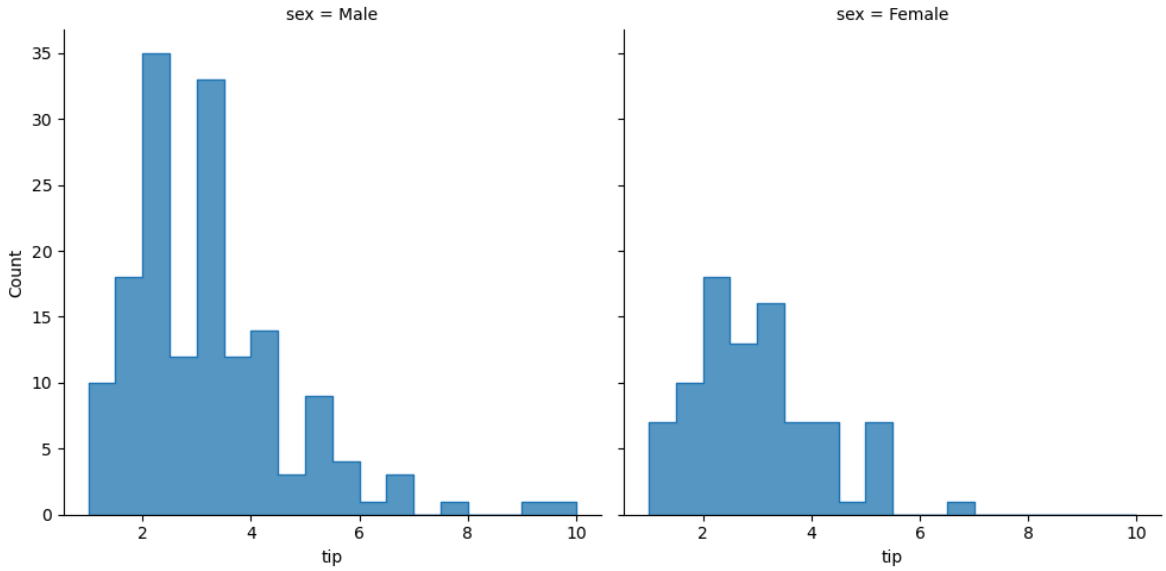
Out[22]: <seaborn.axisgrid.FacetGrid at 0x26e965ad250>



In [23]:
```
# faceting using col and row -> not work on histplot function

sns.displot(data=tips, x='tip', kind='hist',col='sex',element='step')
```

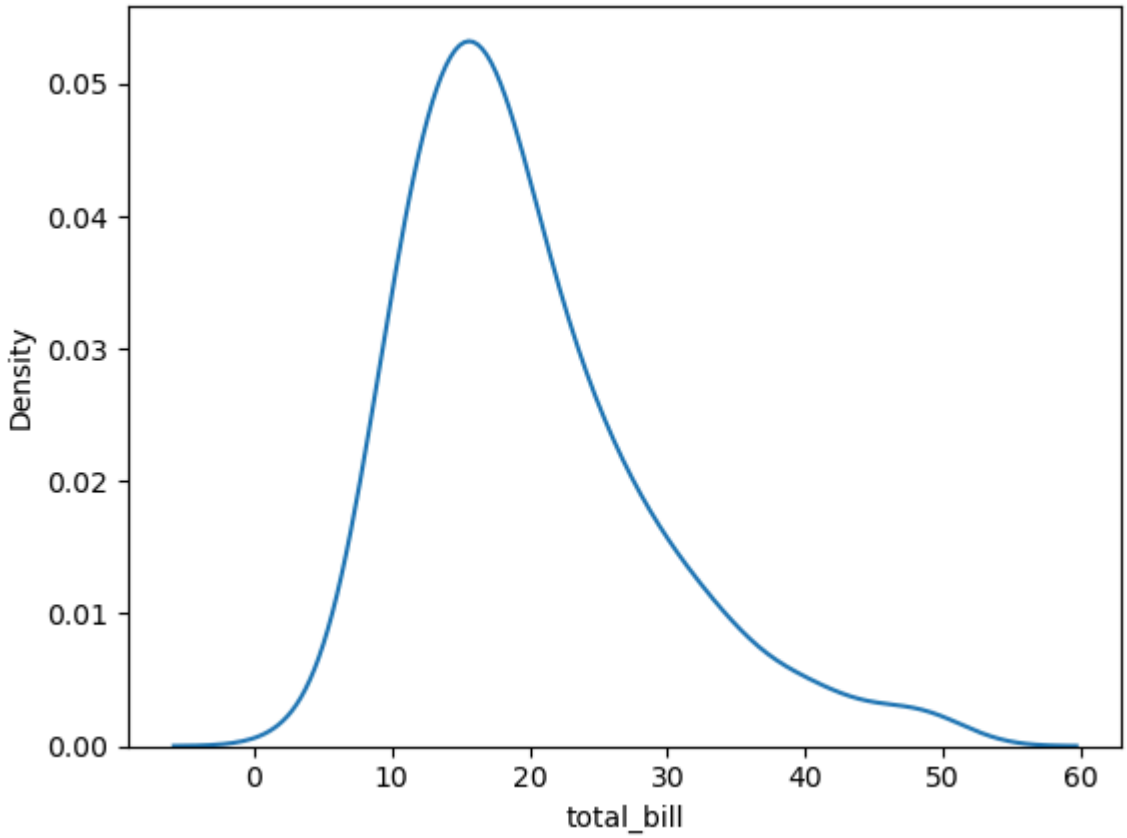
Out[23]: <seaborn.axisgrid.FacetGrid at 0x26e977f8c90>



In [24]:
```
# kdeplot
# Rather than using discrete bins, a KDE plot smooths the observations with a
#                       Gaussian kernel, producing a continuous density estimate

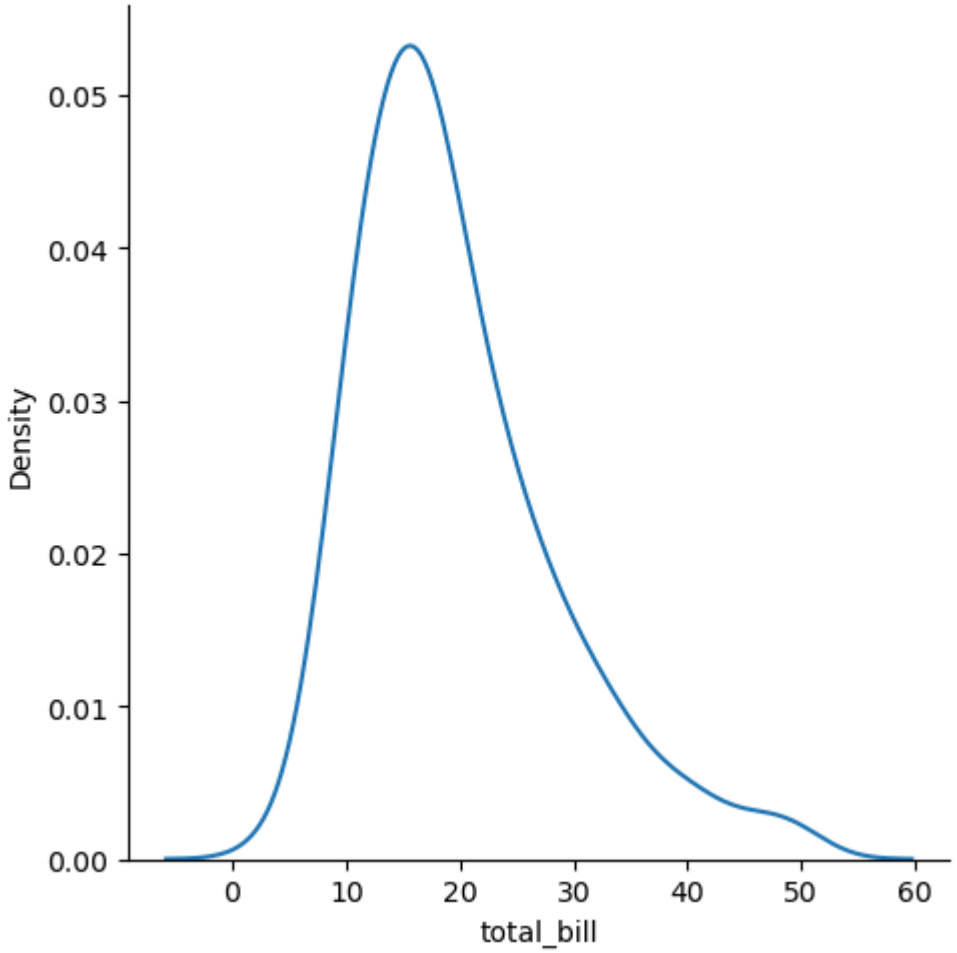
sns.kdeplot(data=tips,x='total_bill')
```

Out[24]: <Axes: xlabel='total_bill', ylabel='Density'>



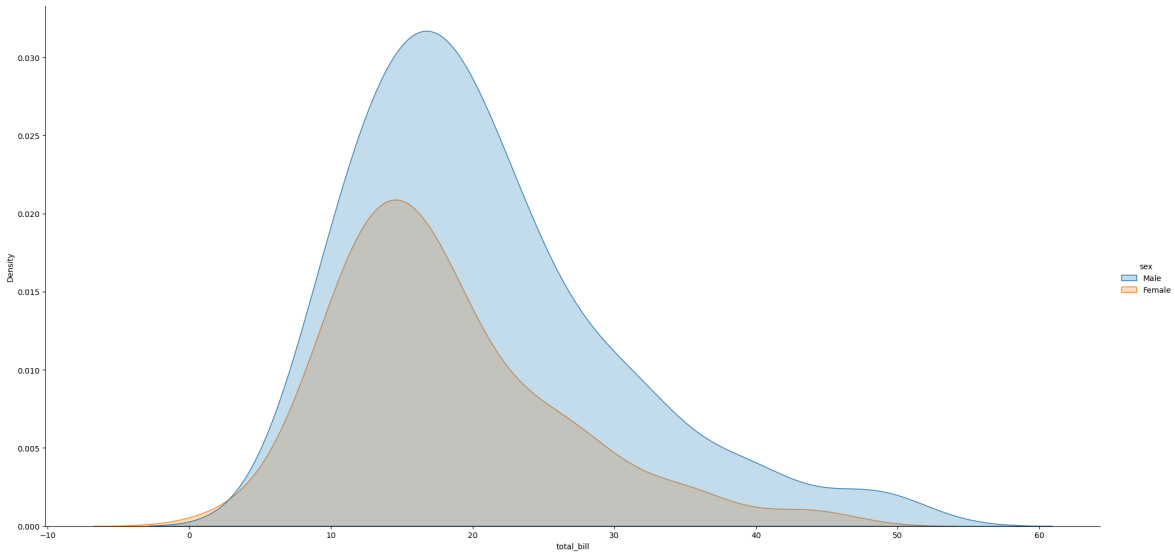
In [25]:
```
sns.displot(data=tips,x='total_bill',kind='kde')
```

Out[25]: <seaborn.axisgrid.FacetGrid at 0x26e977d99d0>



In [26]:
```
# hue -> fill
sns.displot(data=tips,x='total_bill',kind='kde',hue='sex',fill=True,
            height=10,aspect=2)
```

Out[26]: <seaborn.axisgrid.FacetGrid at 0x26e97c7e410>



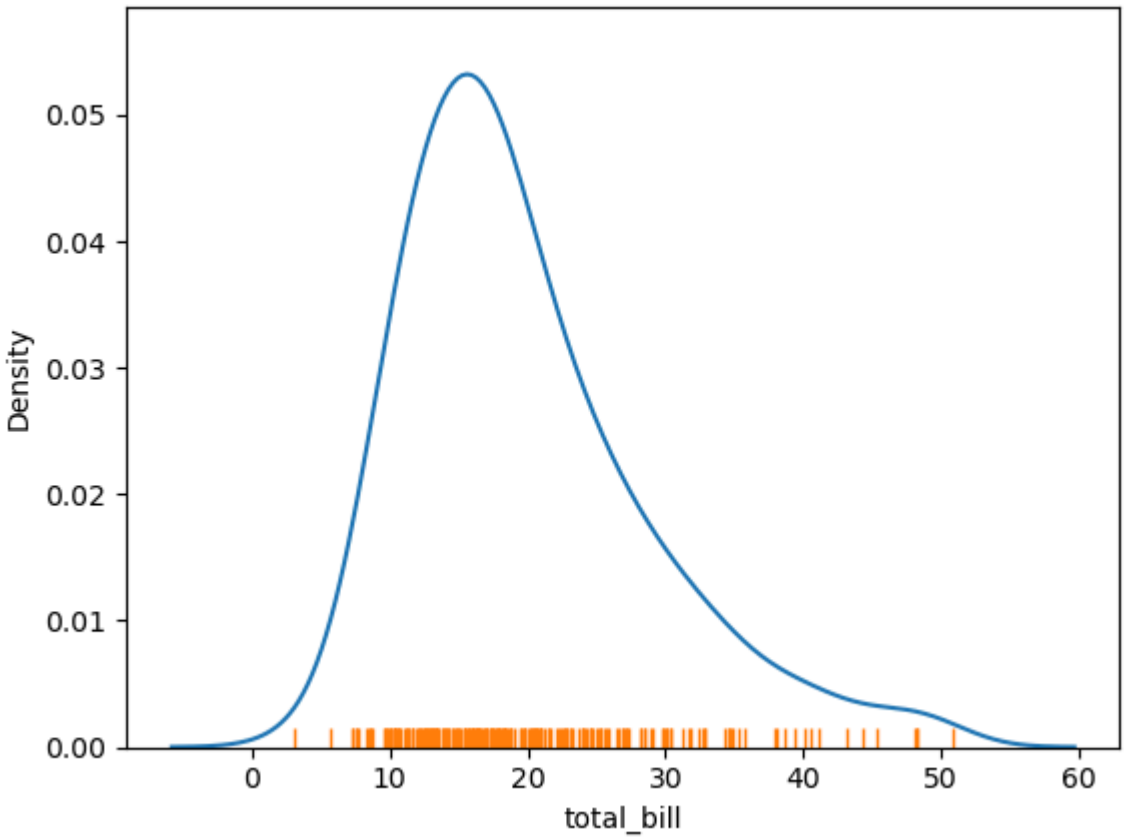
In [27]:
```
# Rugplot

# Plot marginal distributions by drawing ticks along the x and y axes.

# This function is intended to complement other plots by showing the
#                      location of individual observations in an unobtrusive way.
```

```
sns.kdeplot(data=tips,x='total_bill')
sns.rugplot(data=tips,x='total_bill')
```

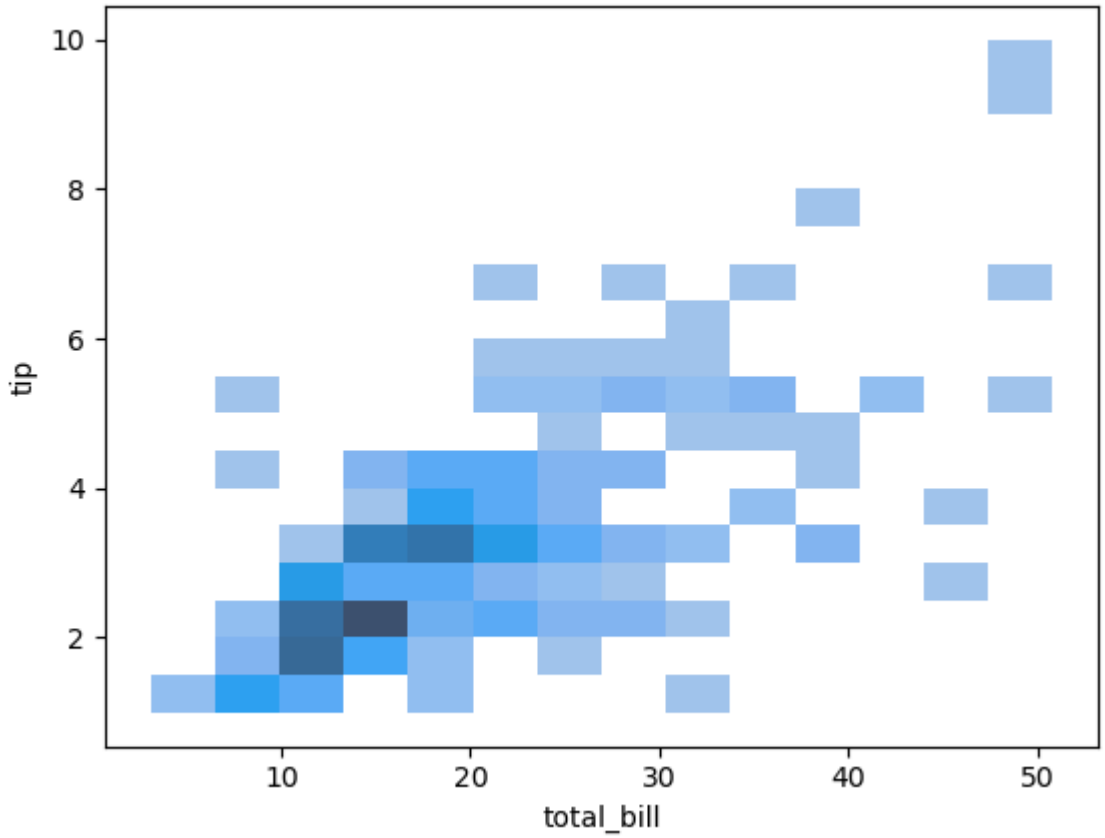
Out[27]: <Axes: xlabel='total_bill', ylabel='Density'>



In [28]:
```
# Bivariate histogram
# A bivariate histogram bins the data within rectangles that tile the plot
# and then shows the count of observations within each rectangle with the fill
#                                                                      color

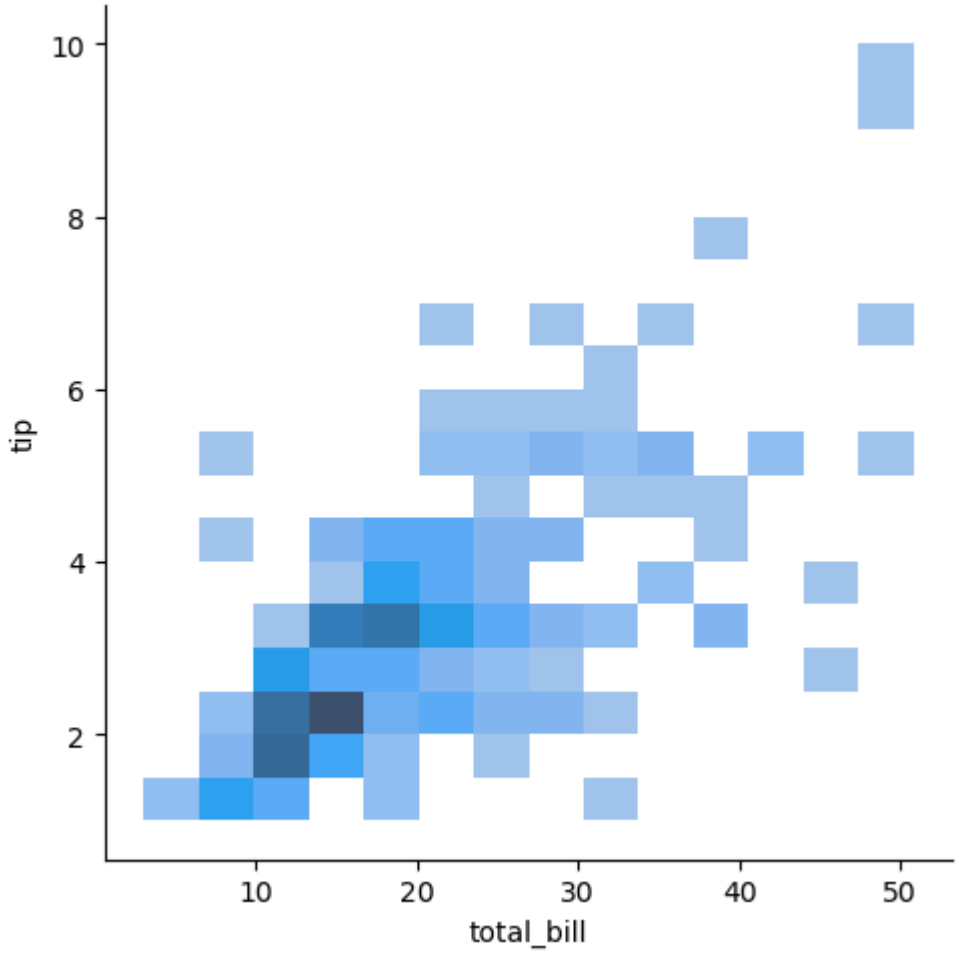
sns.histplot(data=tips, x='total_bill', y='tip')
```

Out[28]: <Axes: xlabel='total_bill', ylabel='tip'>



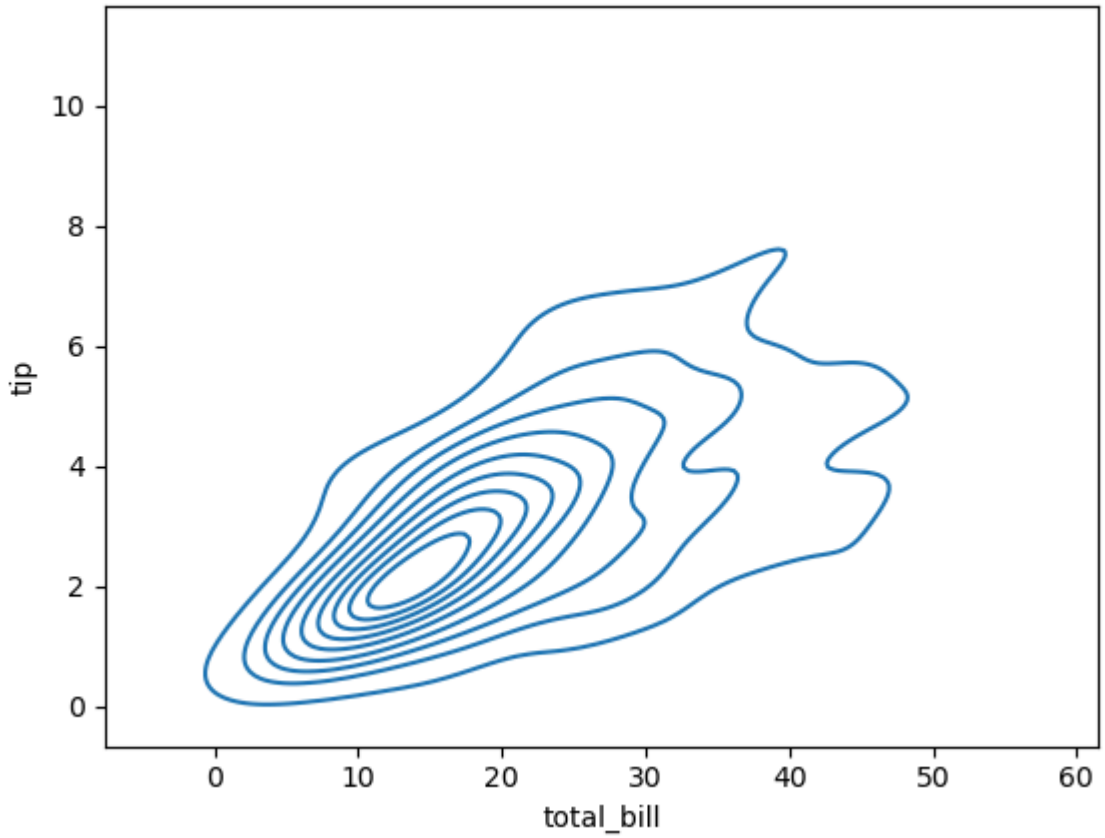
In [29]:
```
sns.displot(data=tips, x='total_bill', y='tip',kind='hist')
```

Out[29]:
```
<seaborn.axisgrid.FacetGrid at 0x26e9a275b50>
```



In [30]:
```
# Bivariate Kdeplot
# a bivariate KDE plot smoothes the (x, y) observations with a 2D Gaussian
sns.kdeplot(data=tips, x='total_bill', y='tip')
```

Out[30]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



2. Matrix Plot

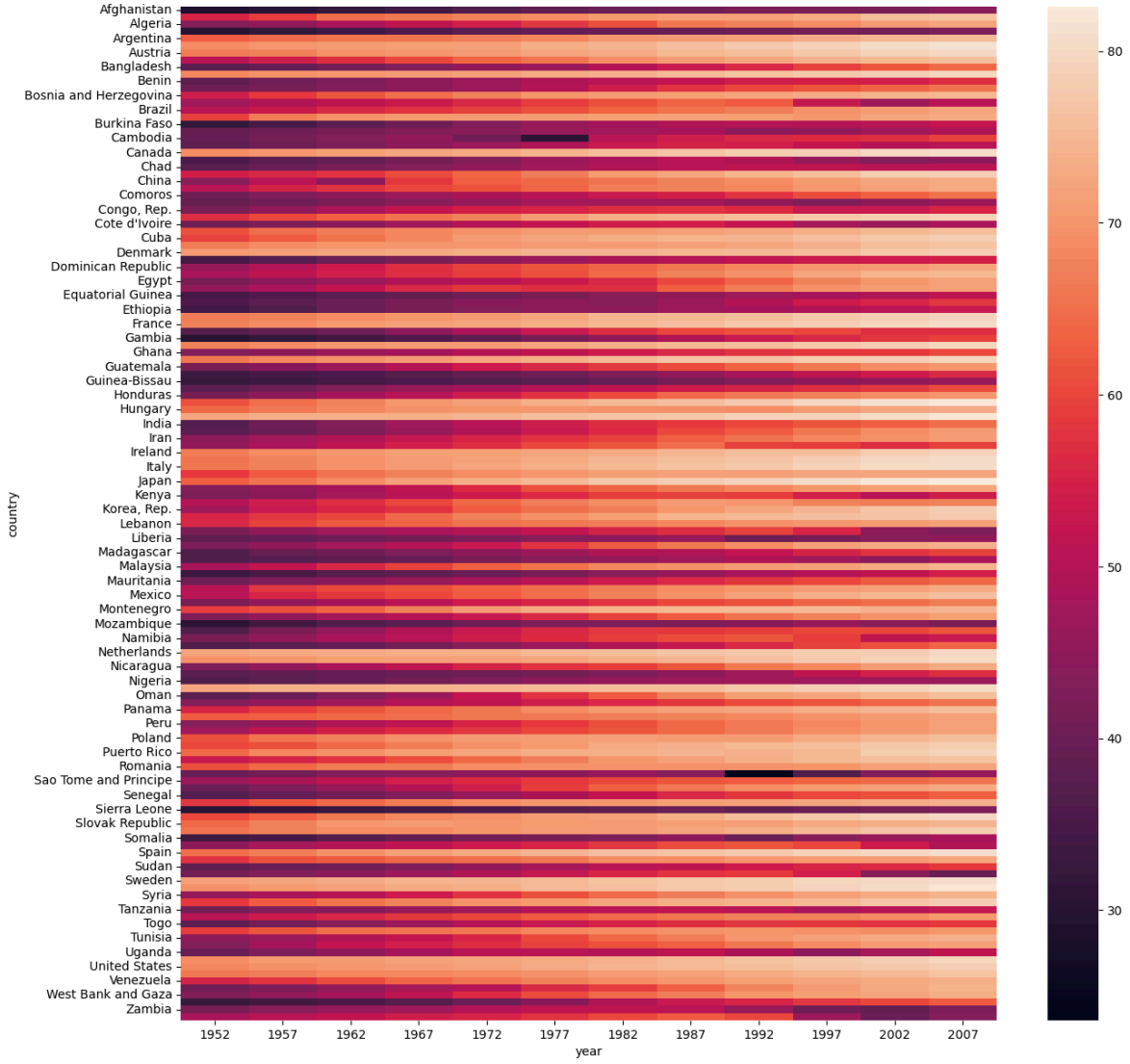
- Heatmap
- Clustermap

In [31]:
```
# Heatmap

# Plot rectangular data as a color-encoded matrix
temp_df = gap.pivot(index='country',columns='year',values='lifeExp')

# axes level function
plt.figure(figsize=(15,15))
sns.heatmap(temp_df)
```

Out[31]:
```
<Axes: xlabel='year', ylabel='country'>
```



In [32]:
```
# annot
temp_df = gap[gap['continent'] == 'Europe'].pivot(index='country',columns='year'

plt.figure(figsize=(15,15))
sns.heatmap(temp_df,annot=True,linewidth=0.5, cmap='summer')
```

Out[32]: <Axes: xlabel='year', ylabel='country'>



In [33]:
```
# Clustermap

# Plot a matrix dataset as a hierarchically-clustered heatmap.

# This function requires scipy to be available.

iris = px.data.iris()
iris
```

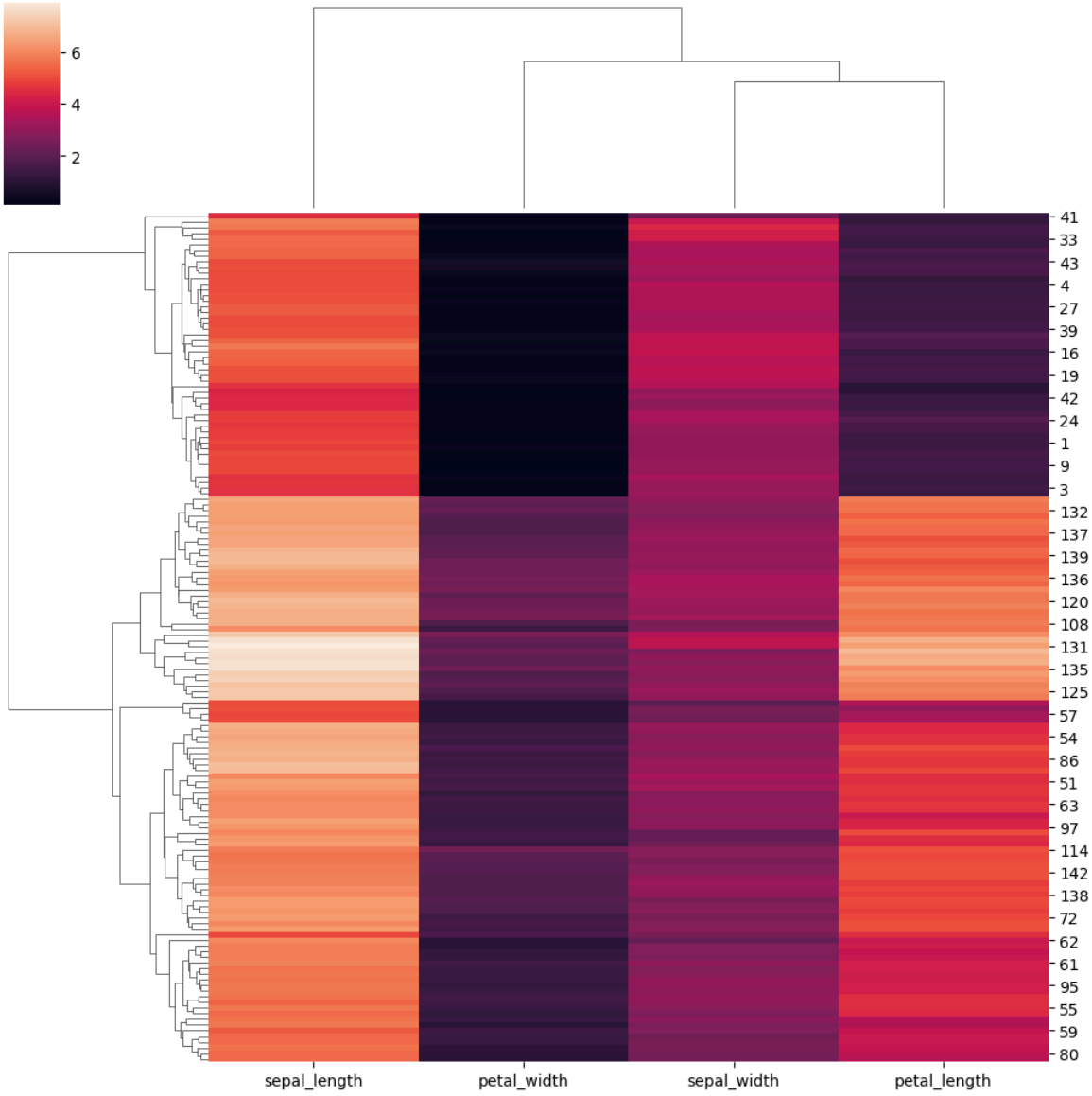
Out[33]:

	sepal_length	sepal_width	petal_length	petal_width	species	species_id
0	5.1	3.5	1.4	0.2	setosa	1
1	4.9	3.0	1.4	0.2	setosa	1
2	4.7	3.2	1.3	0.2	setosa	1
3	4.6	3.1	1.5	0.2	setosa	1
4	5.0	3.6	1.4	0.2	setosa	1
...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica	3
146	6.3	2.5	5.0	1.9	virginica	3
147	6.5	3.0	5.2	2.0	virginica	3
148	6.2	3.4	5.4	2.3	virginica	3
149	5.9	3.0	5.1	1.8	virginica	3

150 rows × 6 columns

In [34]:
```
sns.clustermap(iris.iloc[:,[0,1,2,3]])
```

Out[34]:
```
<seaborn.matrix.ClusterGrid at 0x26e9a505850>
```



TASK

In [35]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("ggplot")
```

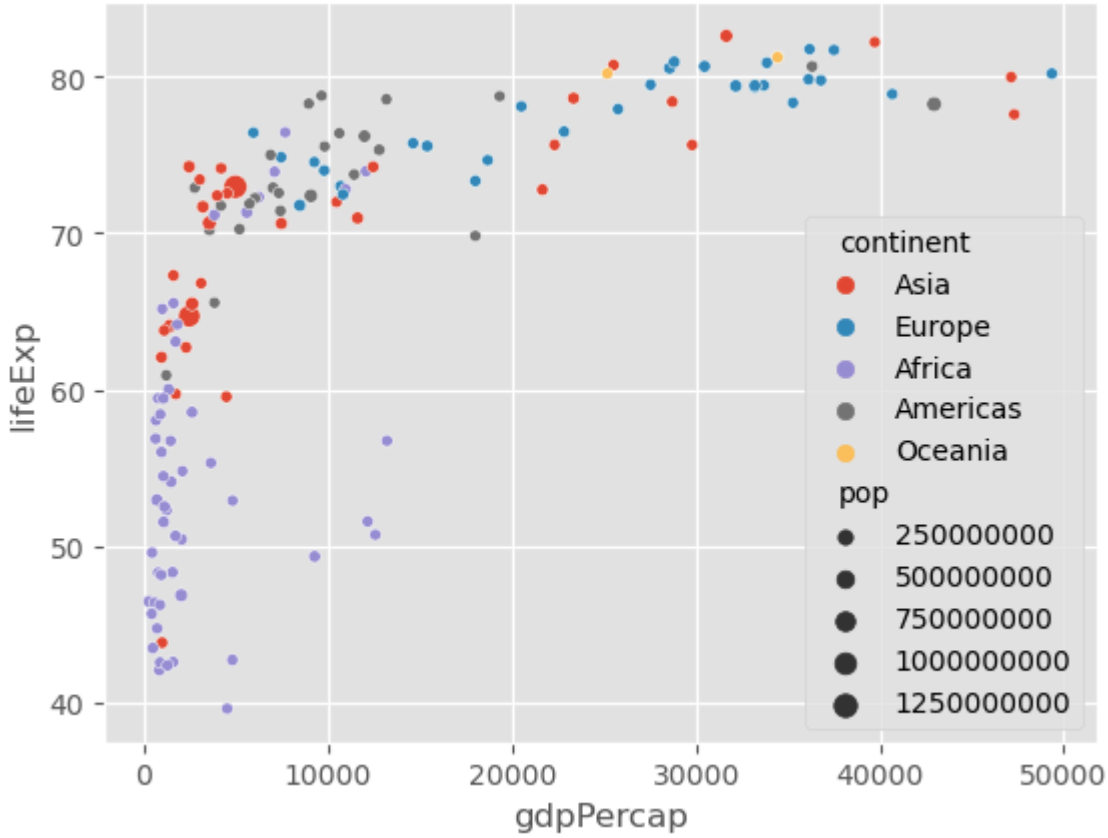
In [36]:
```
# code here
import plotly.express as px
df = px.data.gapminder()
df.head()
```

Out[36]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	4
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	4
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	4
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	4
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	4

In [37]:
```
sns.scatterplot(data=df.query('year == 2007'), x='gdpPercap',y='lifeExp'
                ,hue='continent',size='pop')
```

Out[37]:
```
<Axes: xlabel='gdpPercap', ylabel='lifeExp'>
```



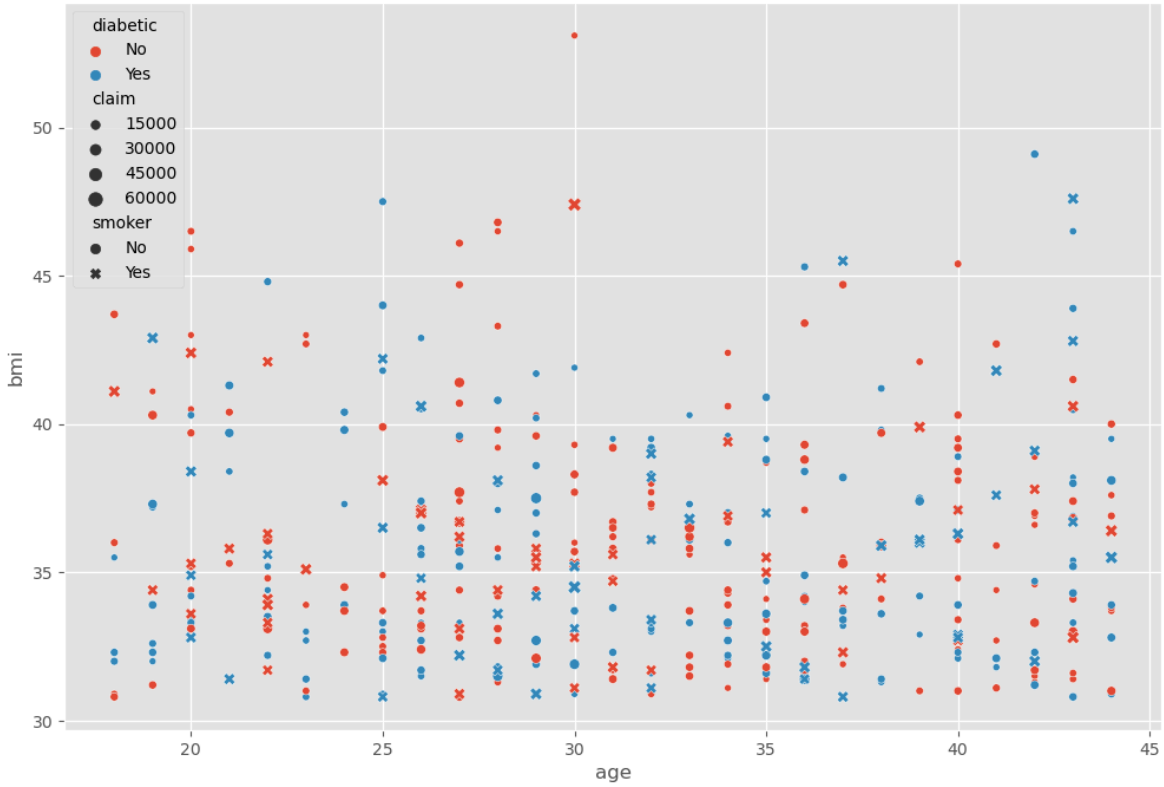
In [38]:
```
# code here
df = pd.read_csv('https://docs.google.com/spreadsheets/d/e/2PACX-1vQVpcVtdYdZU4z
df.head()
```

Out[38]:

	index	PatientID	age	gender	bmi	bloodpressure	diabetic	children	smoker	
0	0	1	39.0	male	23.2	91	Yes	0	No	sou
1	1	2	24.0	male	30.1	87	No	0	No	sou
2	2	3	NaN	male	33.3	82	Yes	0	No	sou
3	3	4	NaN	male	33.7	80	No	0	No	nor
4	4	5	NaN	male	34.1	100	No	0	No	nor

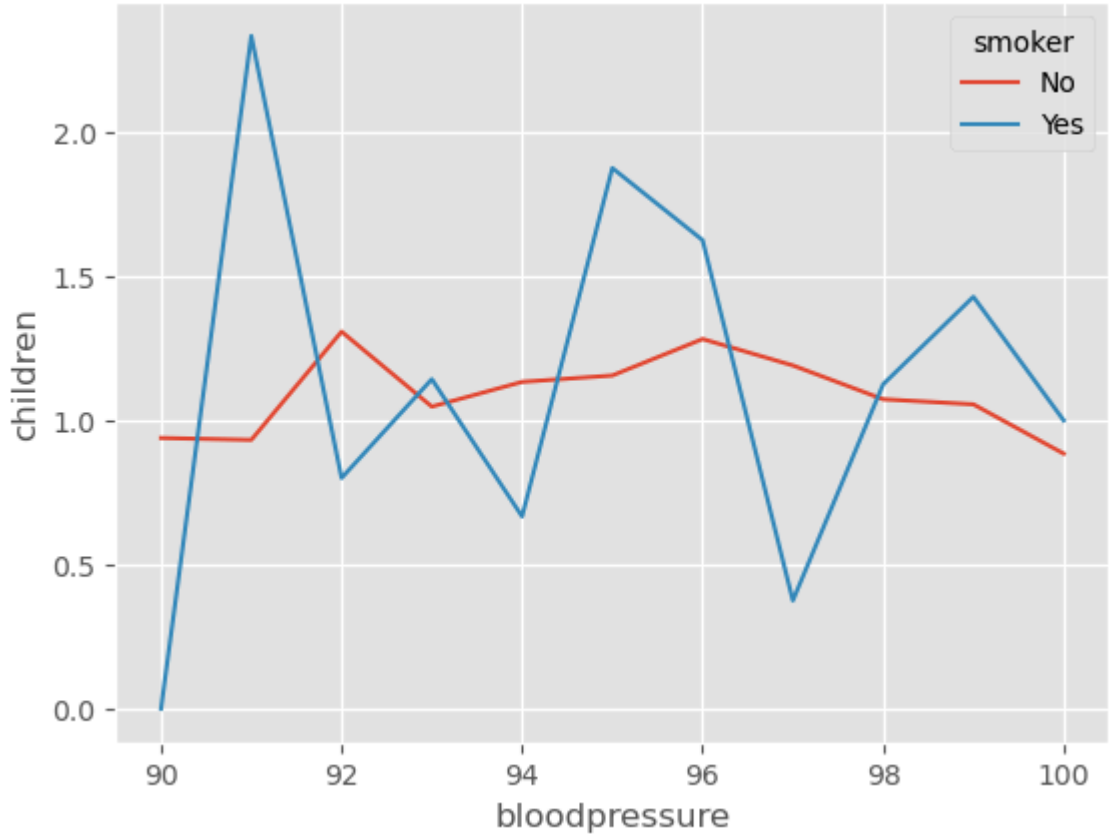
In [39]:
```
temp_df = df[df['age'] < df['age'].quantile(0.70)]
temp_df = temp_df[temp_df['bmi'] > temp_df['bmi'].mean()]
```

In [40]:
```
plt.subplots(figsize=(12,8))
sns.scatterplot(data=temp_df, x='age', y='bmi', hue='diabetic',size='claim',
                style='smoker')
plt.show()
```



In [41]:
```
# Code here
sns.lineplot(data = df.query("bloodpressure >= 90 and bloodpressure <= 100"),
             x='bloodpressure', y='children',hue='smoker',err_style=None)
```

Out[41]:
```
<Axes: xlabel='bloodpressure', ylabel='children'>
```



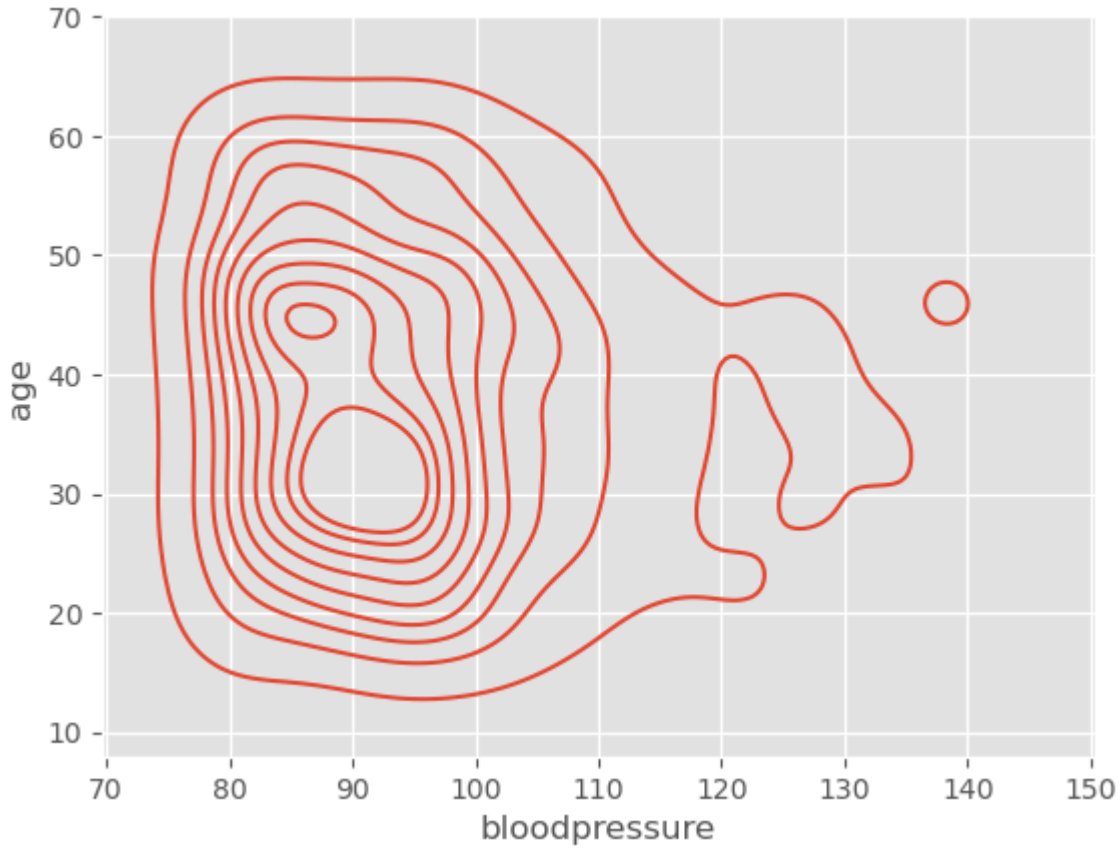
In [42]:
```
# code here
sns.displot(data=df,x='age',hue='smoker',kind='hist', col='gender')
```

Out[42]: <seaborn.axisgrid.FacetGrid at 0x26e9a9f8d10>



In [43]:
```
# code here
sns.kdeplot(data=df,x='bloodpressure',y='age')
```

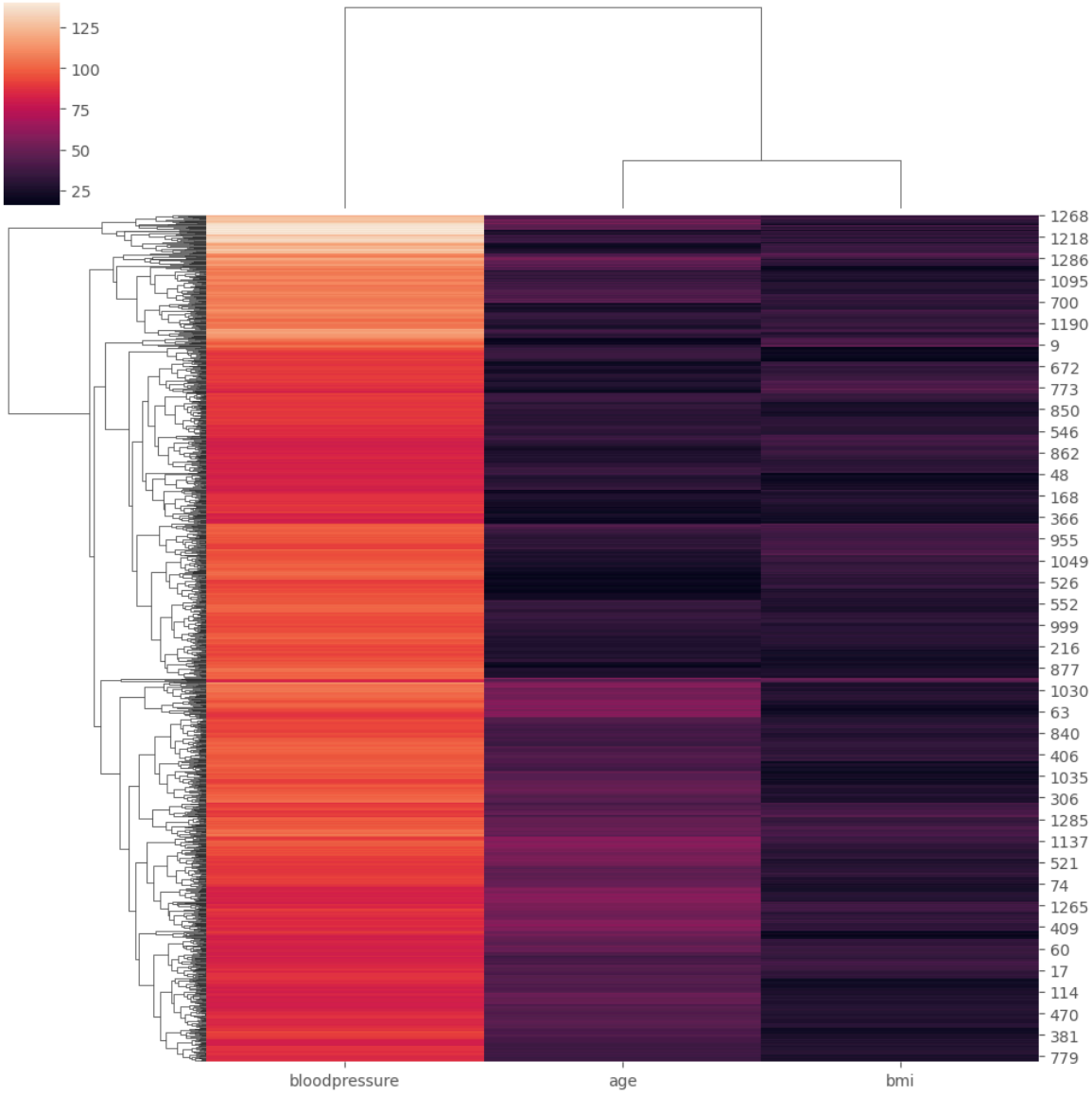
Out[43]: <Axes: xlabel='bloodpressure', ylabel='age'>



In [44]:
```
# code here

sns.clustermap(df[['age','bmi','bloodpressure']].dropna())
```

Out[44]: <seaborn.matrix.ClusterGrid at 0x26e9a999690>



In [45]:
```
df.isnull().sum()
```

Out[45]:
```
index            0
PatientID        0
age              5
gender           0
bmi              0
bloodpressure    0
diabetic         0
children         0
smoker           0
region           3
claim            0
dtype: int64
```

Categorical Plots

Categorical Scatter Plot

- Stripplot
- Swarmplot

Categorical Distribution Plots

- Boxplot
- Violinplot

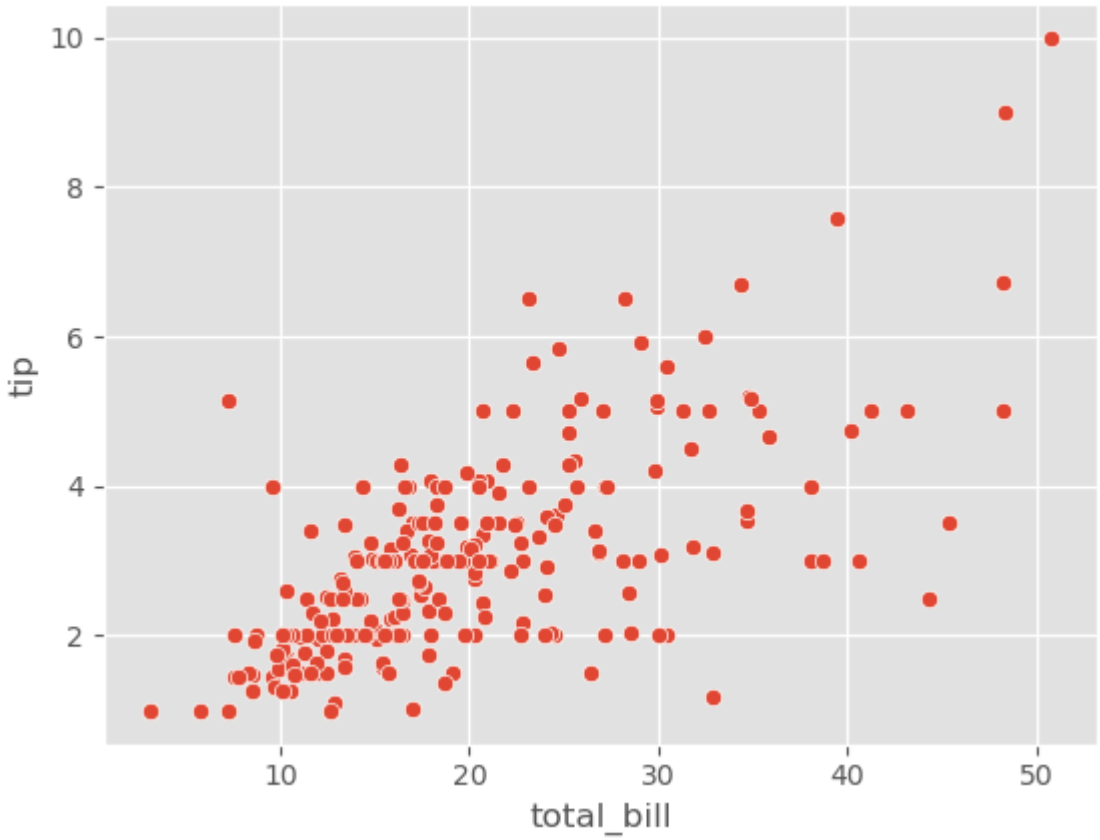
Categorical Estimate Plot -> for central tendency

- Barplot
- Pointplot
- Countplot

Figure level function -> `catplot`

In [46]:
```
sns.scatterplot(data = tips, x = 'total_bill', y = 'tip')
```

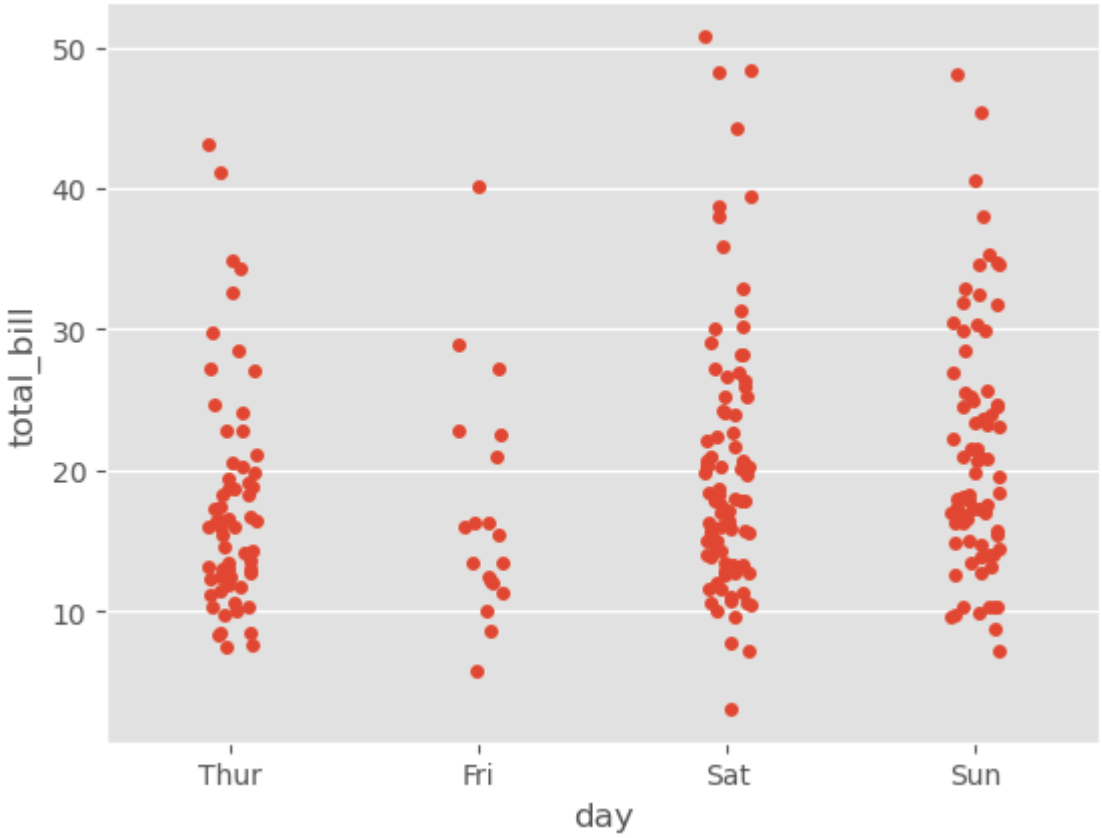
Out[46]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



In [47]:
```
# strip plot
# axes level function

sns.stripplot(data = tips, x= 'day', y = 'total_bill')
```

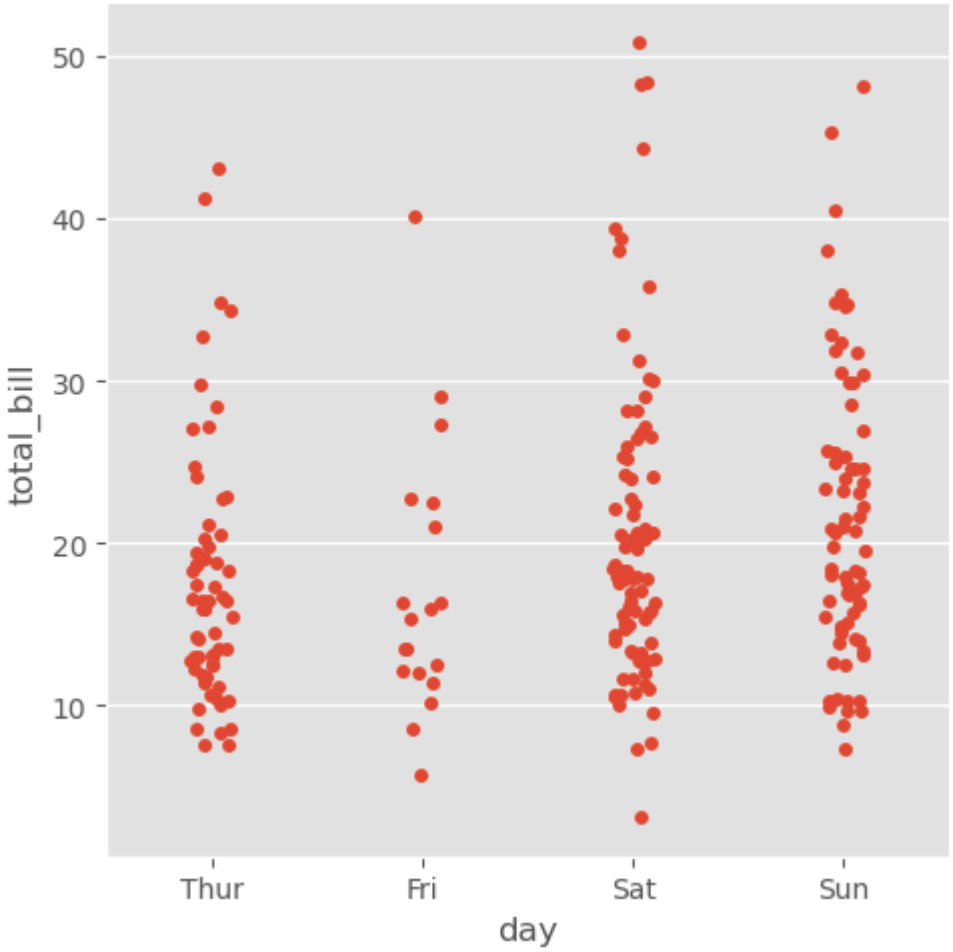
Out[47]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



In [48]:
```
# using catplot
# figure level function

sns.catplot(data = tips, x = 'day', y = 'total_bill', kind = 'strip')
```

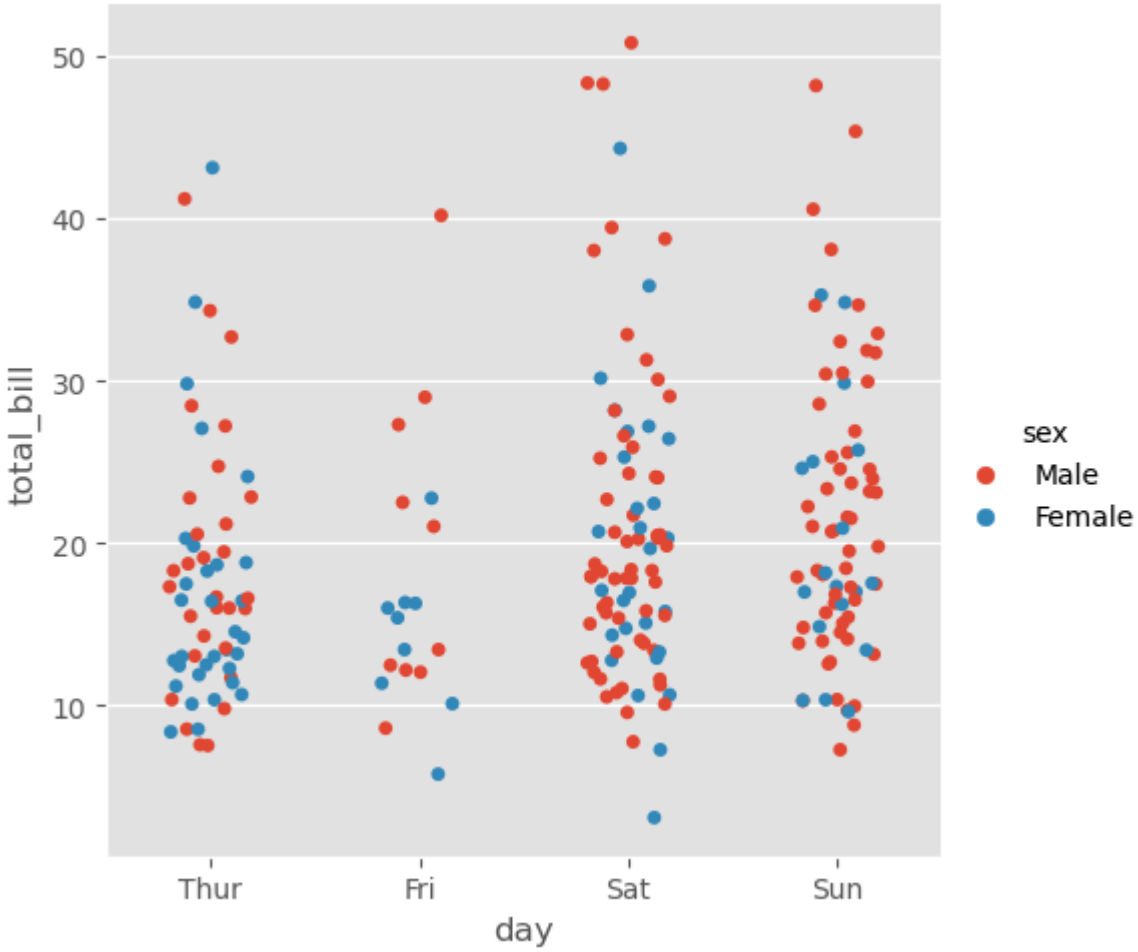
Out[48]:
```
<seaborn.axisgrid.FacetGrid at 0x26e9dc9ff90>
```



In [49]:
```
# jitter

sns.catplot(data = tips, x = 'day', y = 'total_bill', kind = 'strip',
            jitter = 0.2,hue = 'sex')
```

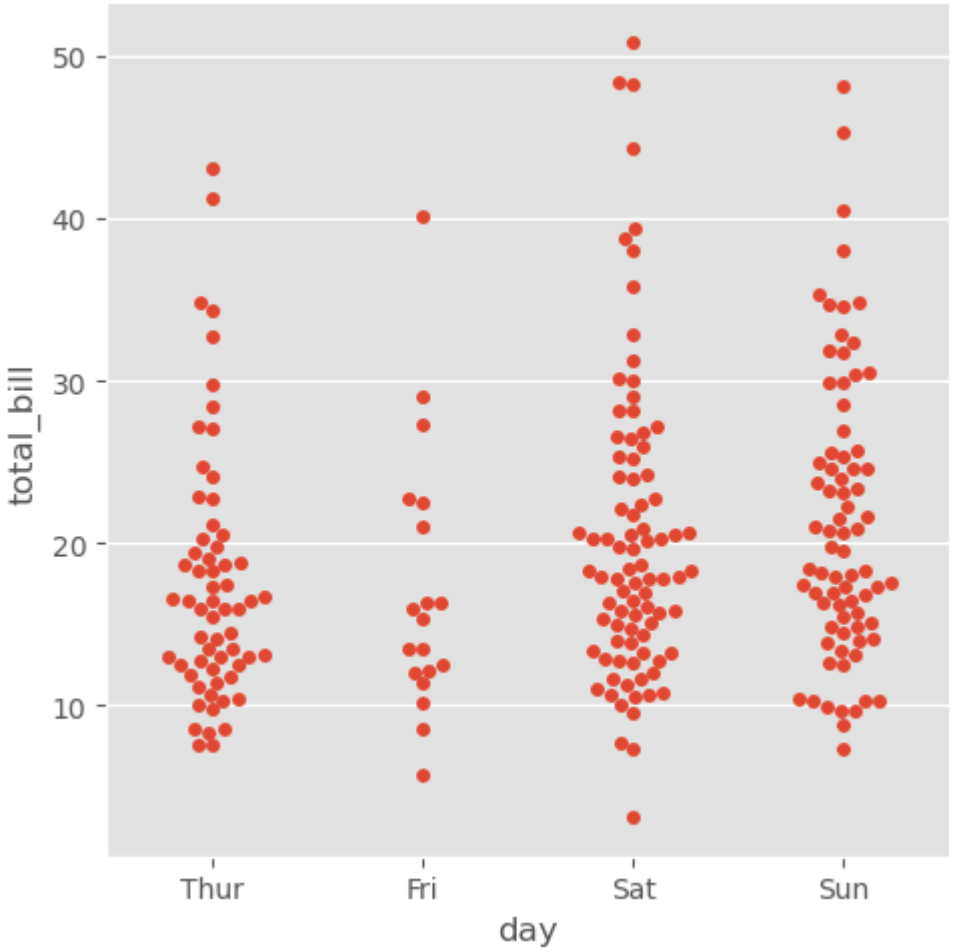
Out[49]:
```
<seaborn.axisgrid.FacetGrid at 0x26e9a933dd0>
```



In [50]:
```
# swarm plot

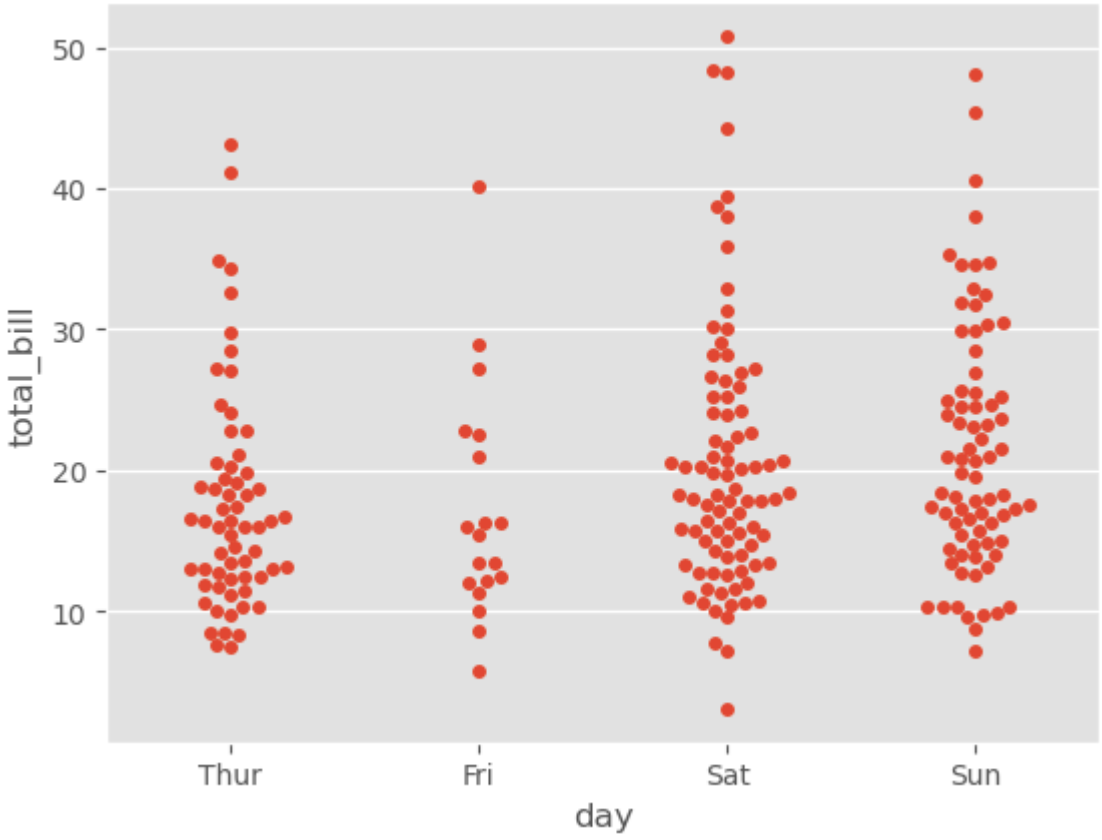
sns.catplot(data = tips, x = 'day', y = 'total_bill', kind = 'swarm')
```

Out[50]: <seaborn.axisgrid.FacetGrid at 0x26e9e809f10>



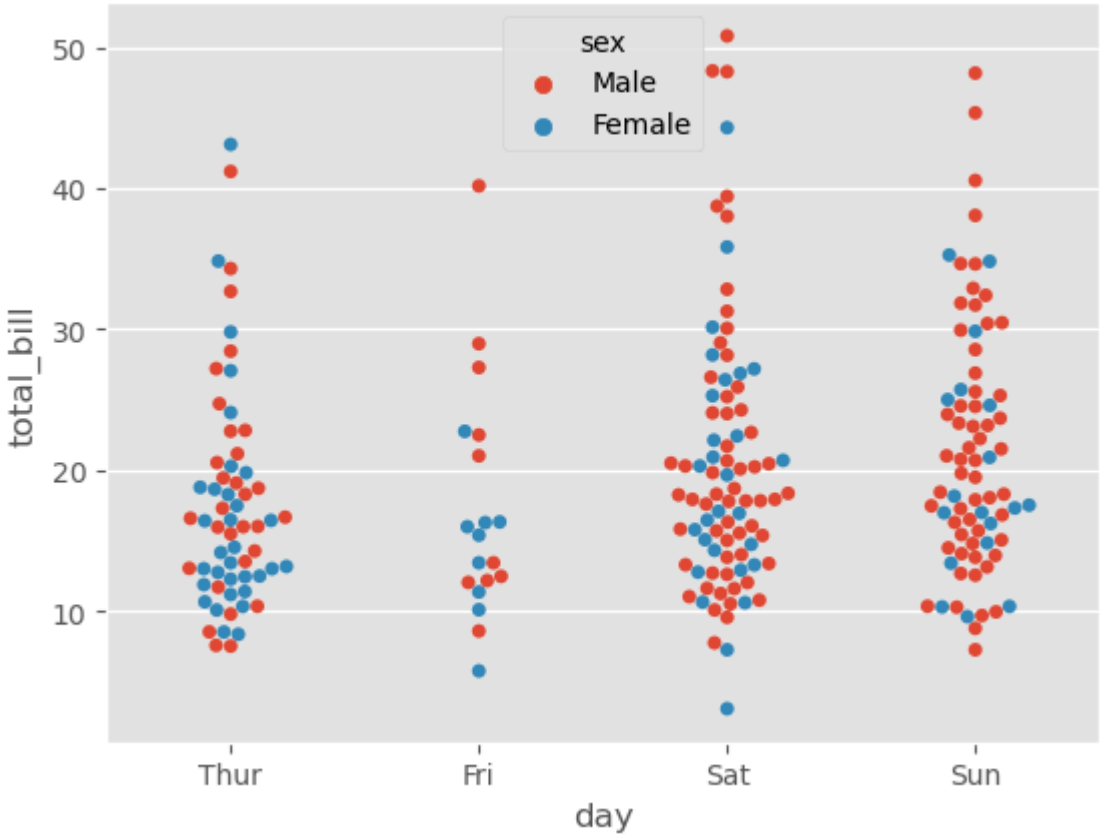
In [51]:
```
sns.swarmplot(data = tips, x = 'day', y = 'total_bill')
```

Out[51]: <Axes: xlabel='day', ylabel='total_bill'>



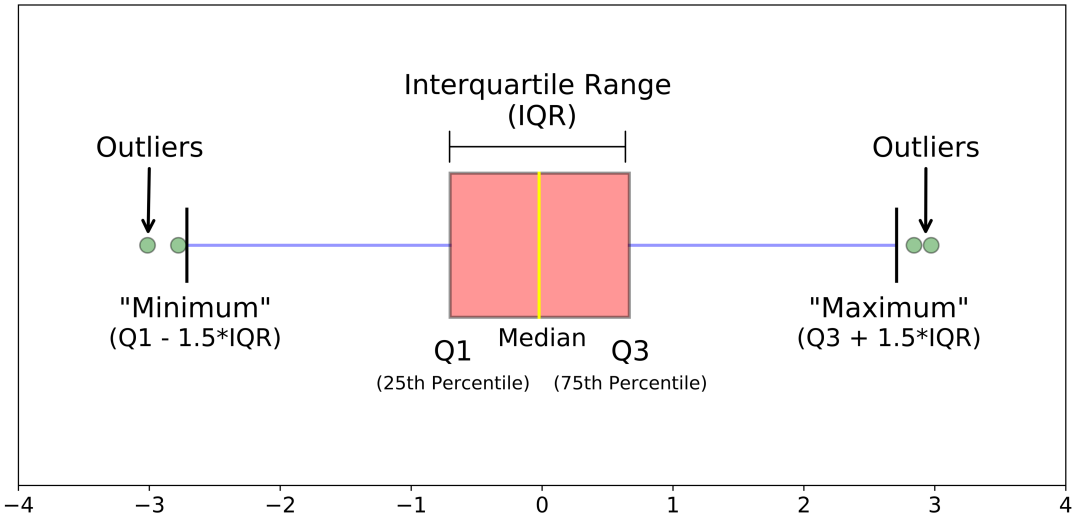
In [52]:
```
# hue
sns.swarmplot(data = tips, x = 'day', y = 'total_bill',hue = 'sex')
```

Out[52]: <Axes: xlabel='day', ylabel='total_bill'>



Boxplot

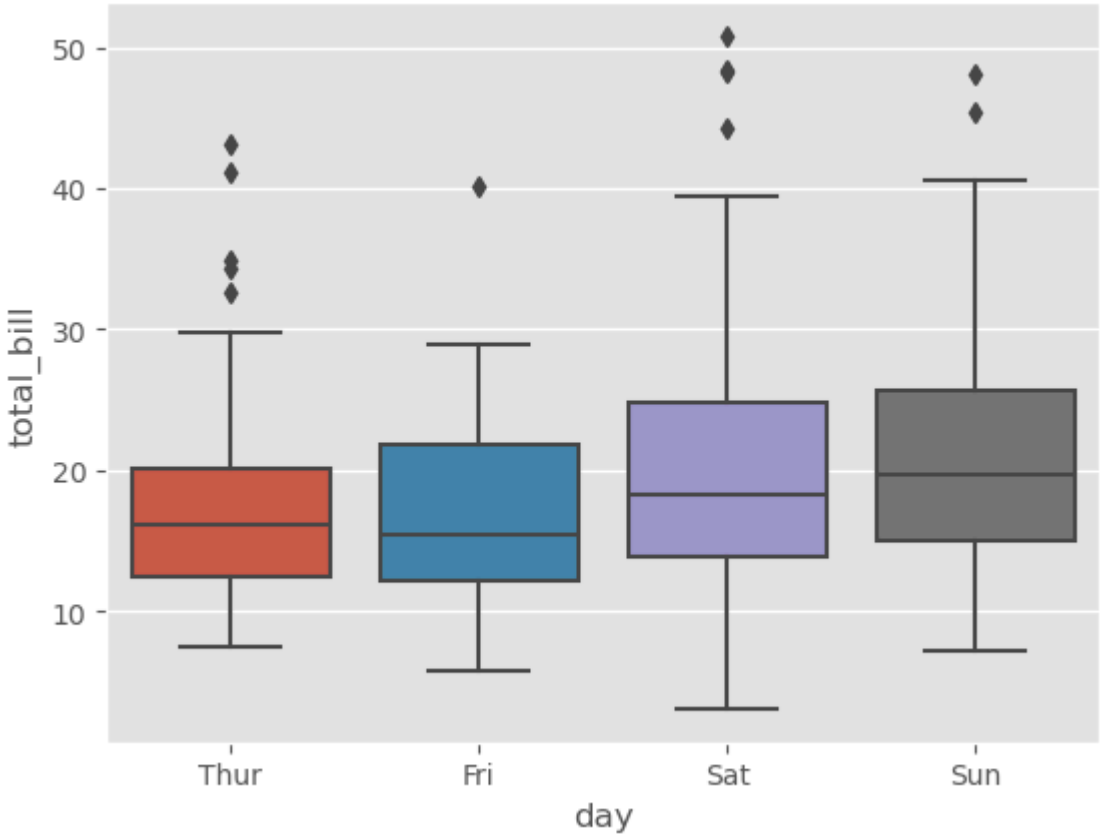
A boxplot is a standardized way of displaying the distribution of data based on a five number summary (“minimum”, first quartile [Q1], median, third quartile [Q3] and “maximum”). It can tell you about your outliers and what their values are. Boxplots can also tell you if your data is symmetrical, how tightly your data is grouped and if and how your data is skewed.



In [53]:
```
# Box plot

sns.boxplot(data = tips, x = 'day', y = 'total_bill')
```

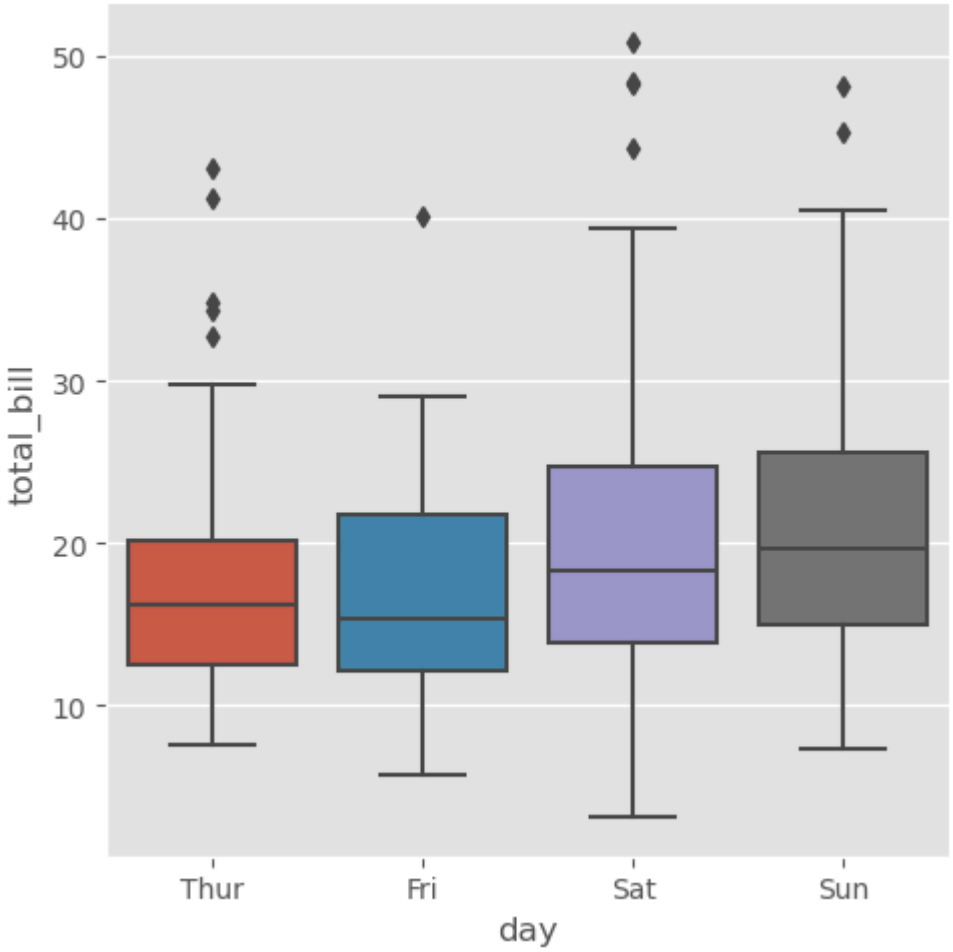
Out[53]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



In [54]:
```
# Using catplot

sns.catplot(data = tips, x = 'day', y = 'total_bill', kind = 'box')
```

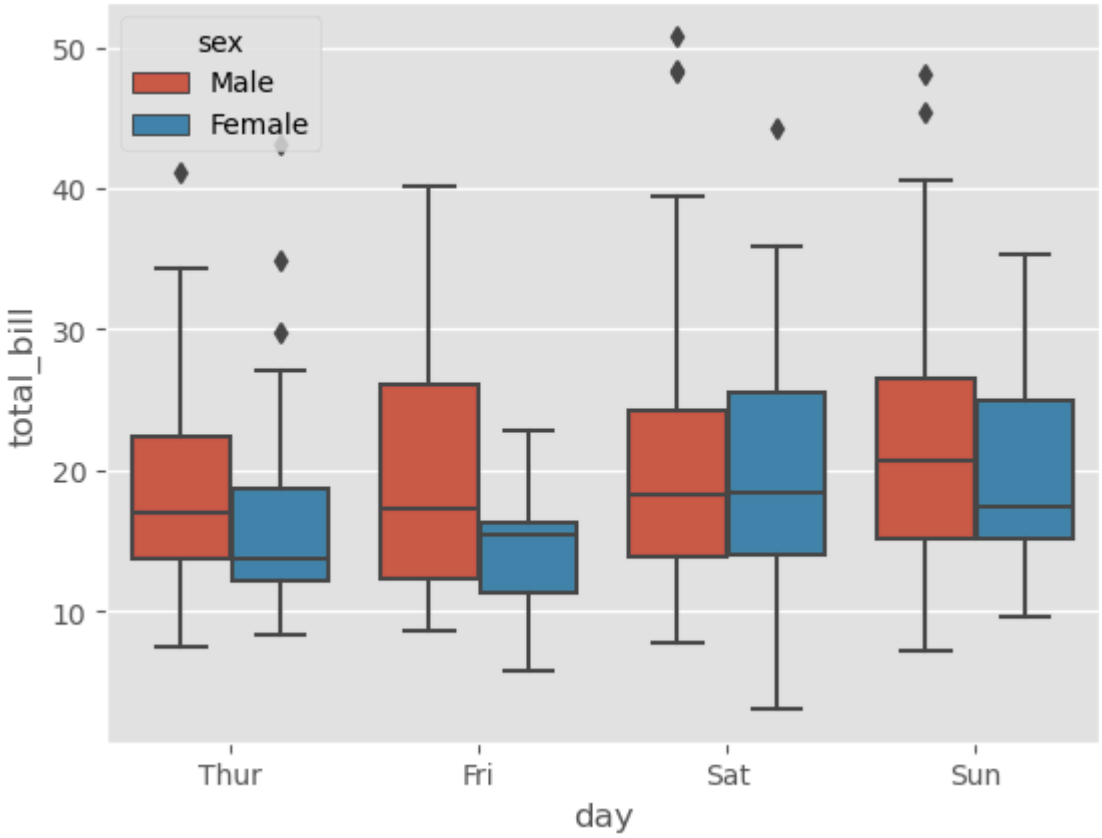
Out[54]:
```
<seaborn.axisgrid.FacetGrid at 0x26e9ea86190>
```



In [55]:
```
# Hue

sns.boxplot(data = tips, x = 'day', y = 'total_bill', hue = 'sex')
```

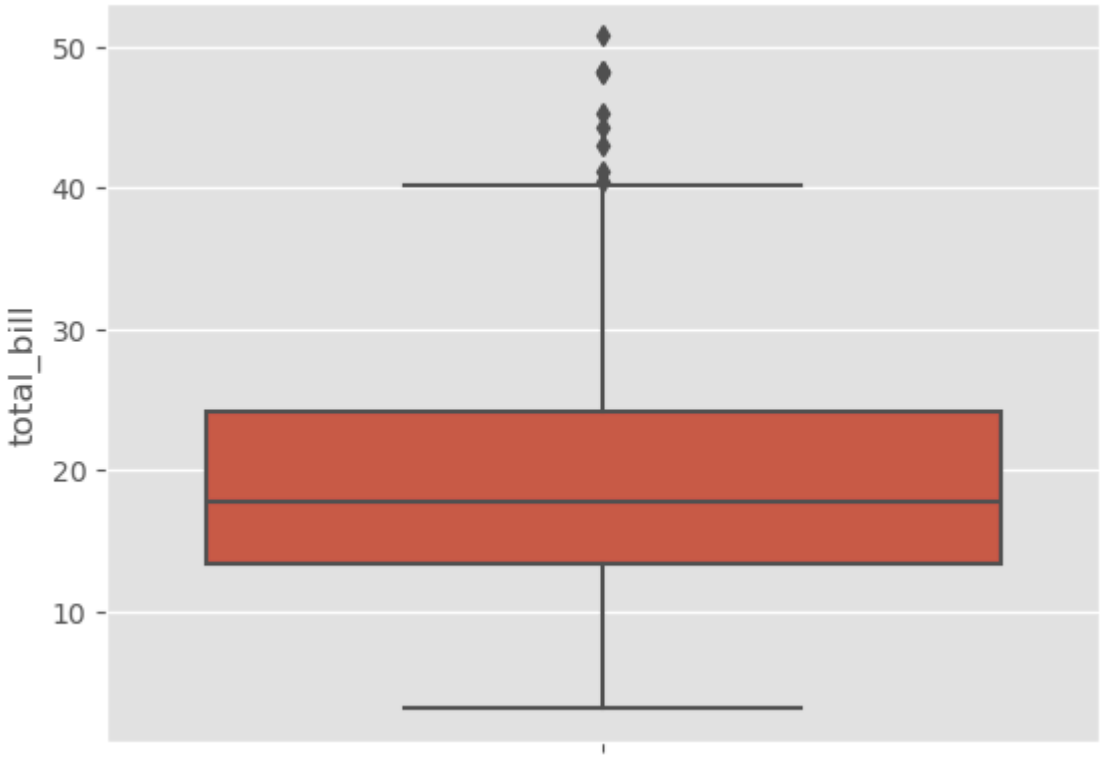
Out[55]: <Axes: xlabel='day', ylabel='total_bill'>



In [56]:
```
# single boxplot -->> numerical col

sns.boxplot(data = tips, y = 'total_bill')
```

Out[56]: <Axes: ylabel='total_bill'>

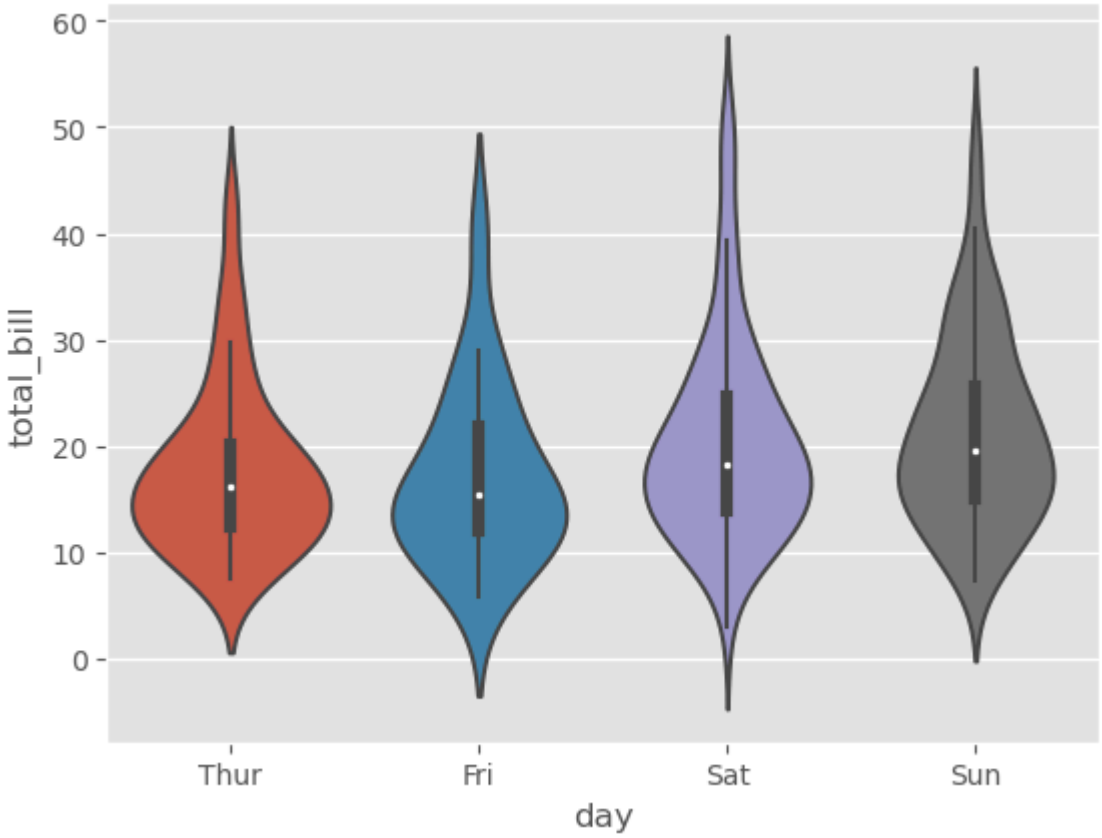


Violinplot = (Boxplot + KDEplot)

In [57]:
```
# Violin plot

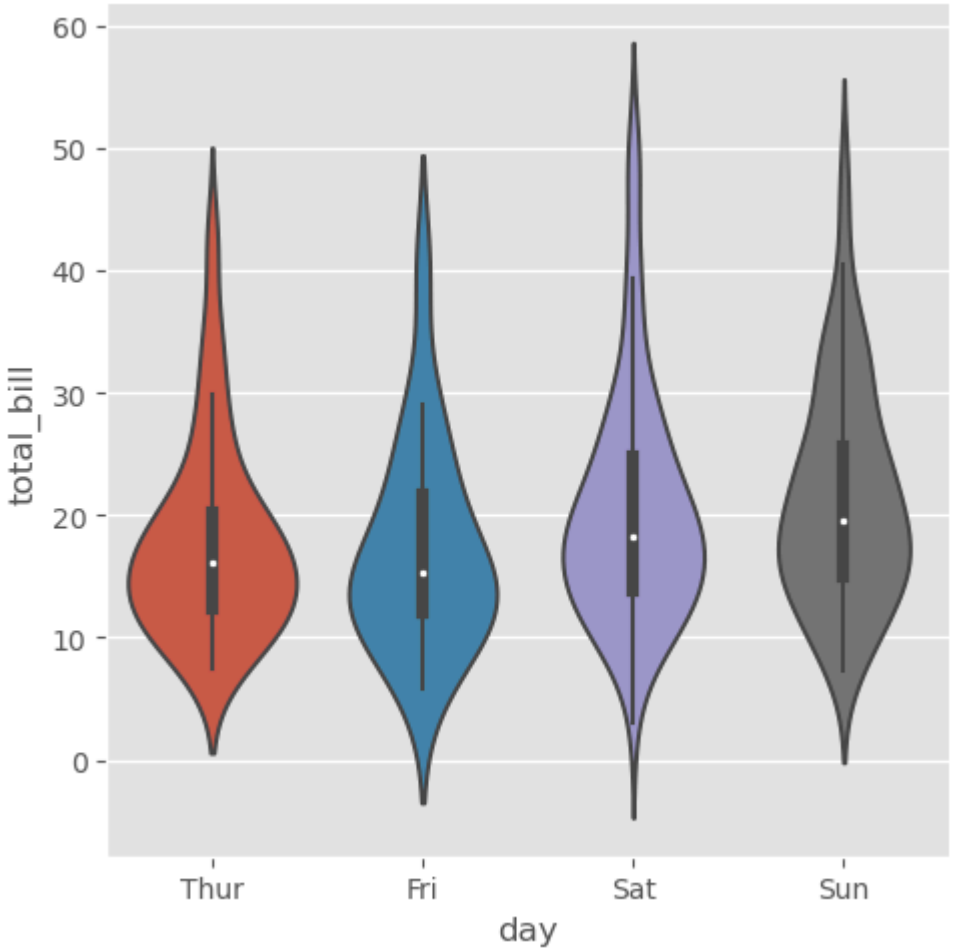
sns.violinplot(data = tips, x = 'day', y = 'total_bill')
```

Out[57]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



In [58]:
```
sns.catplot(data = tips, x = 'day',y = 'total_bill',kind = 'violin')
```

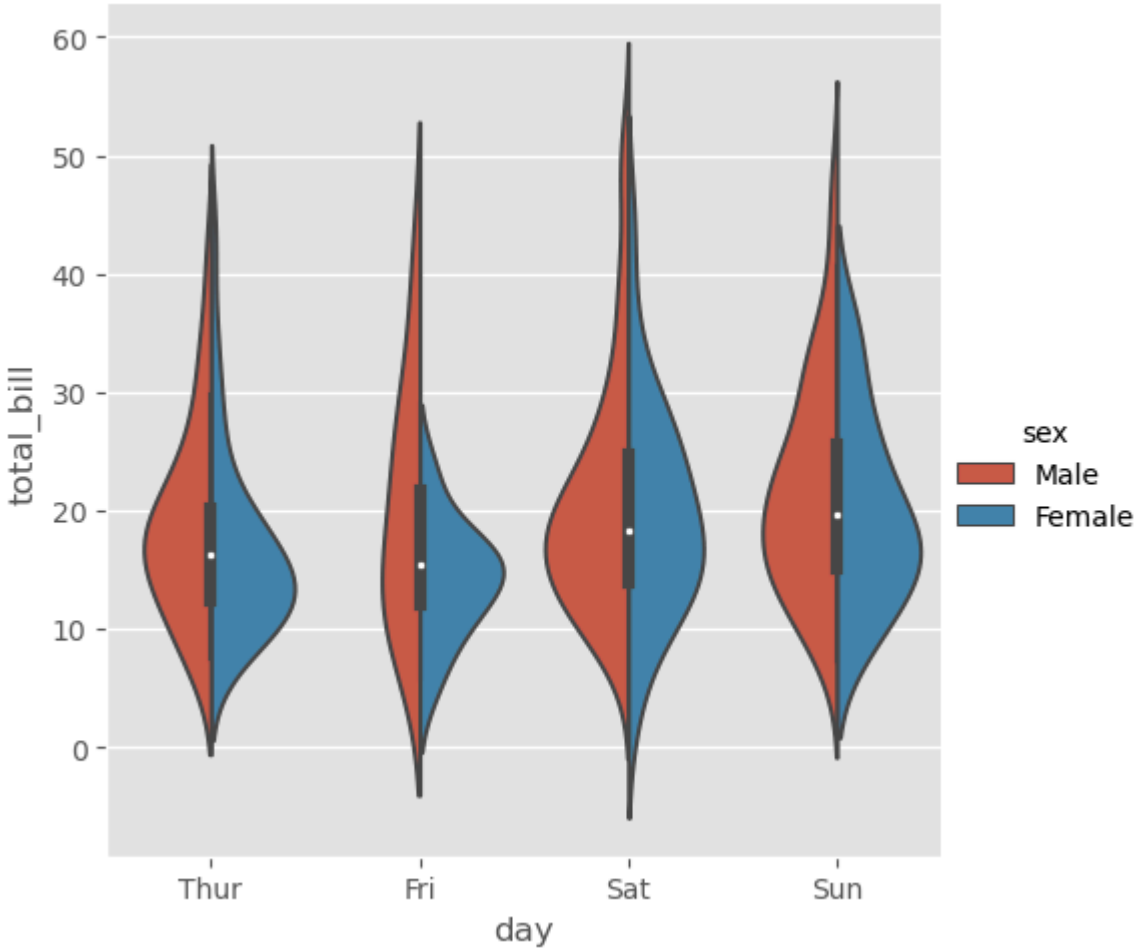
Out[58]:
```
<seaborn.axisgrid.FacetGrid at 0x26e9ea19550>
```



In [59]:
```
# hue

sns.catplot(data = tips, x = 'day',y = 'total_bill',kind = 'violin',
            hue = 'sex', split = True)
```

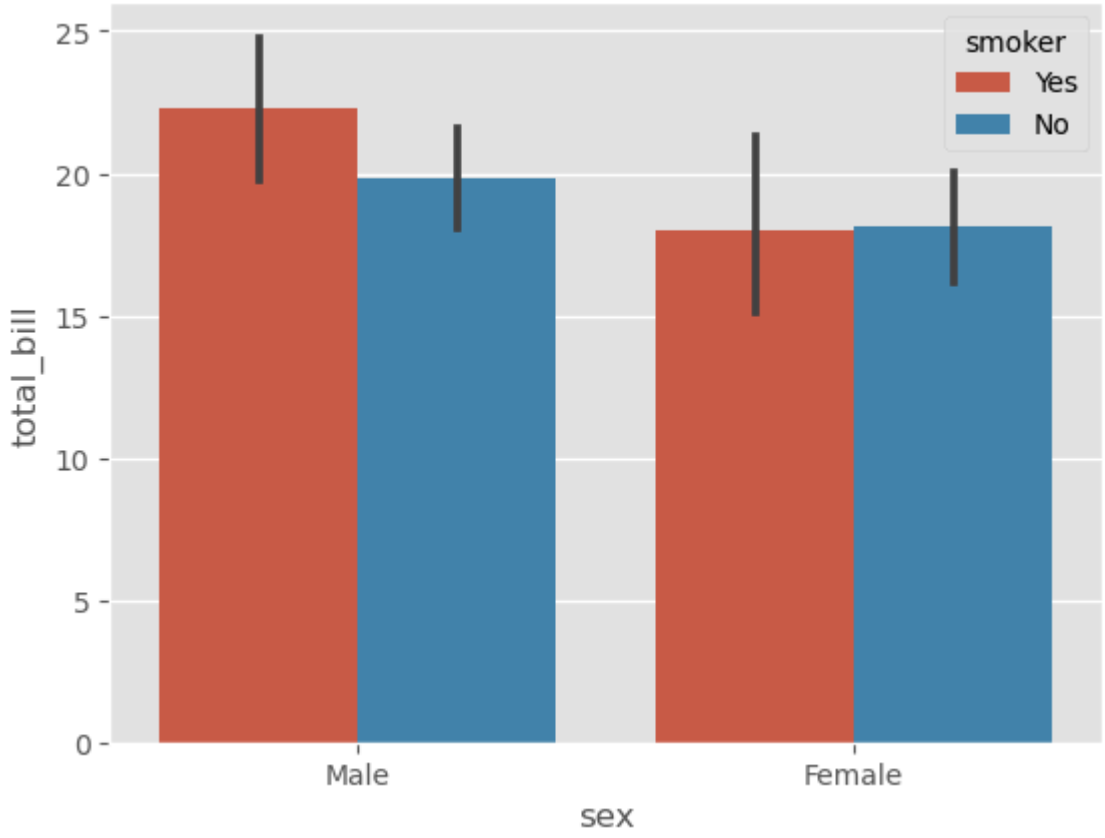
Out[59]: <seaborn.axisgrid.FacetGrid at 0x26e9ff37bd0>



In [60]:
```
# barplot
# some issue with errorbar

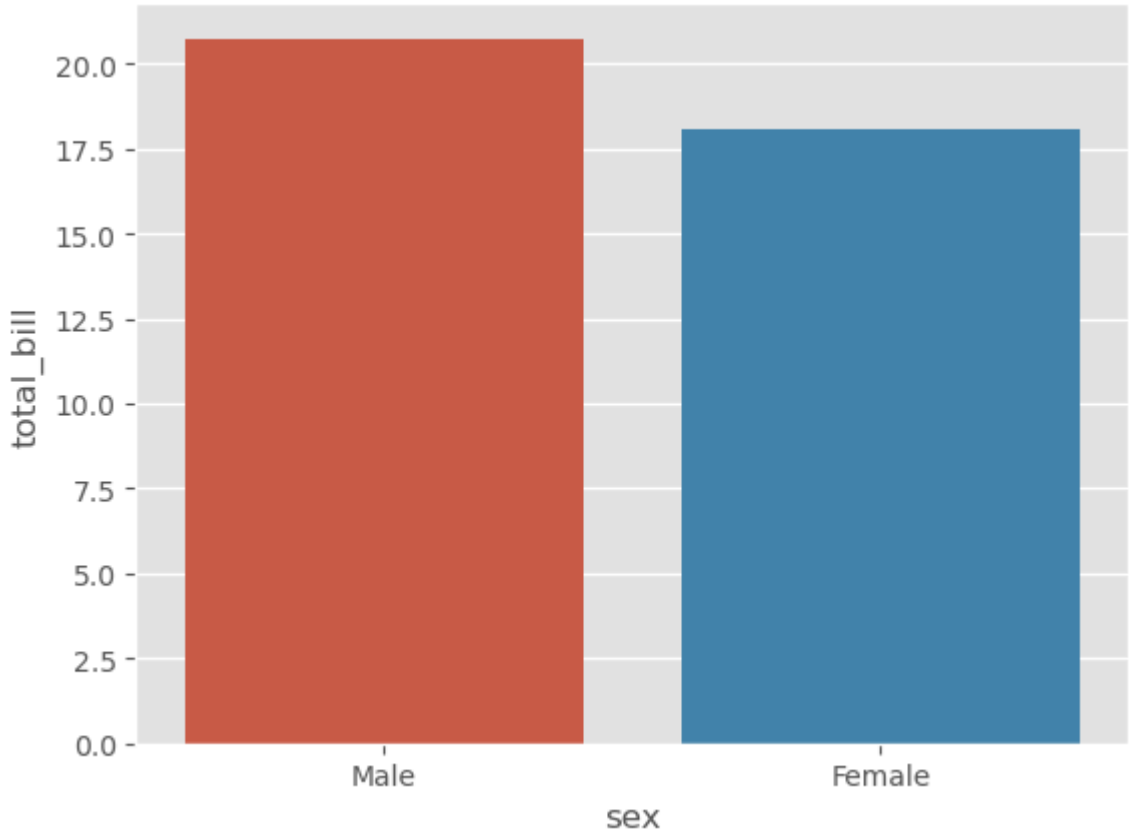
import numpy as np
sns.barplot(data = tips, x = 'sex', y = 'total_bill',hue = 'smoker',
            estimator = np.mean)
```

Out[60]: <Axes: xlabel='sex', ylabel='total_bill'>



In [61]:
```
sns.barplot(data = tips, x = 'sex', y = 'total_bill', ci = None)
```

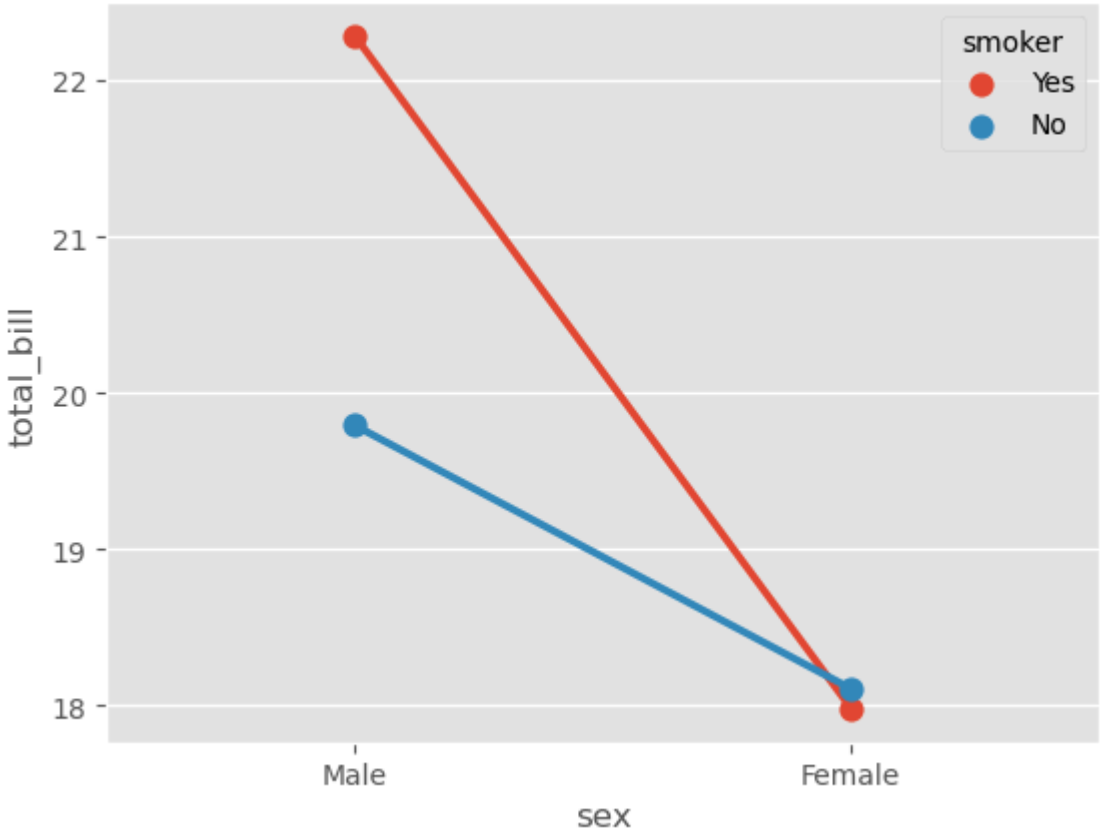
Out[61]: <Axes: xlabel='sex', ylabel='total_bill'>



In [62]:
```
# point plot

sns.pointplot(data = tips, x = 'sex', y= 'total_bill',hue = 'smoker',ci = None)
```

Out[62]: <Axes: xlabel='sex', ylabel='total_bill'>

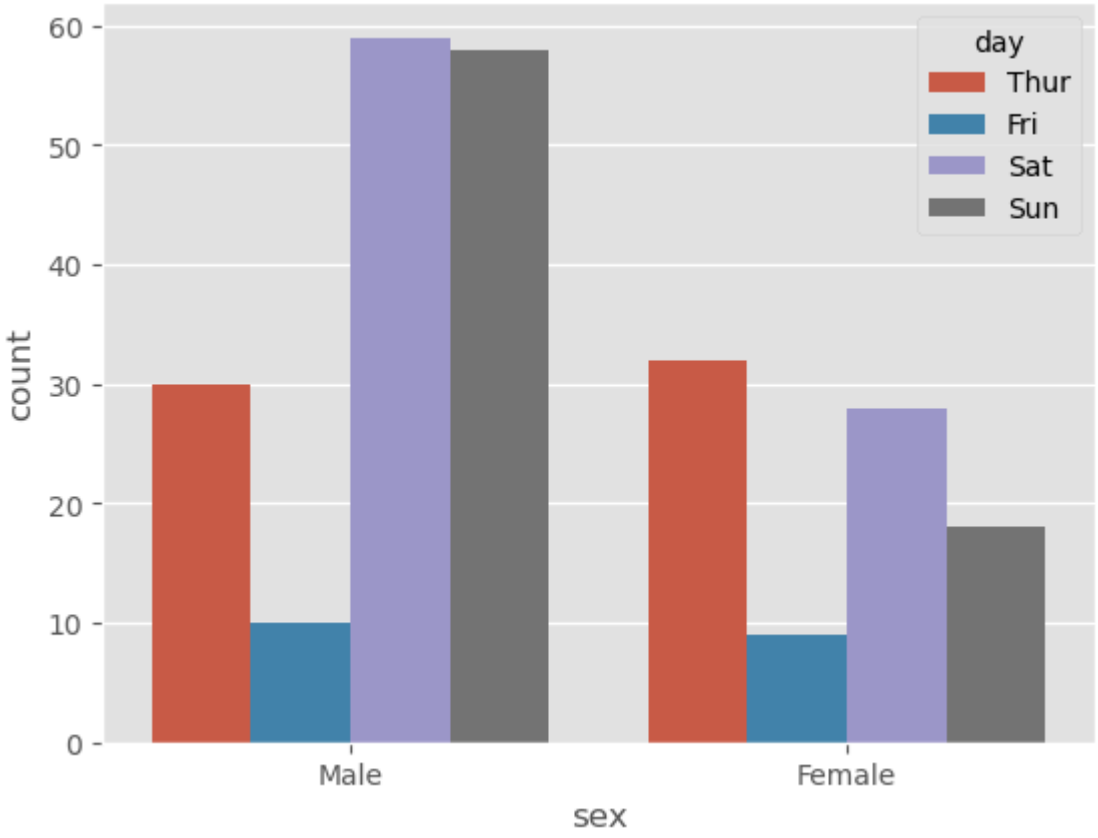


When there are multiple observations in each category, it also uses bootstrapping to compute a confidence interval around the estimate, which is plotted using error bars

In [63]:
```
# countplot

sns.countplot(data = tips, x = 'sex', hue = 'day')
```

Out[63]: <Axes: xlabel='sex', ylabel='count'>

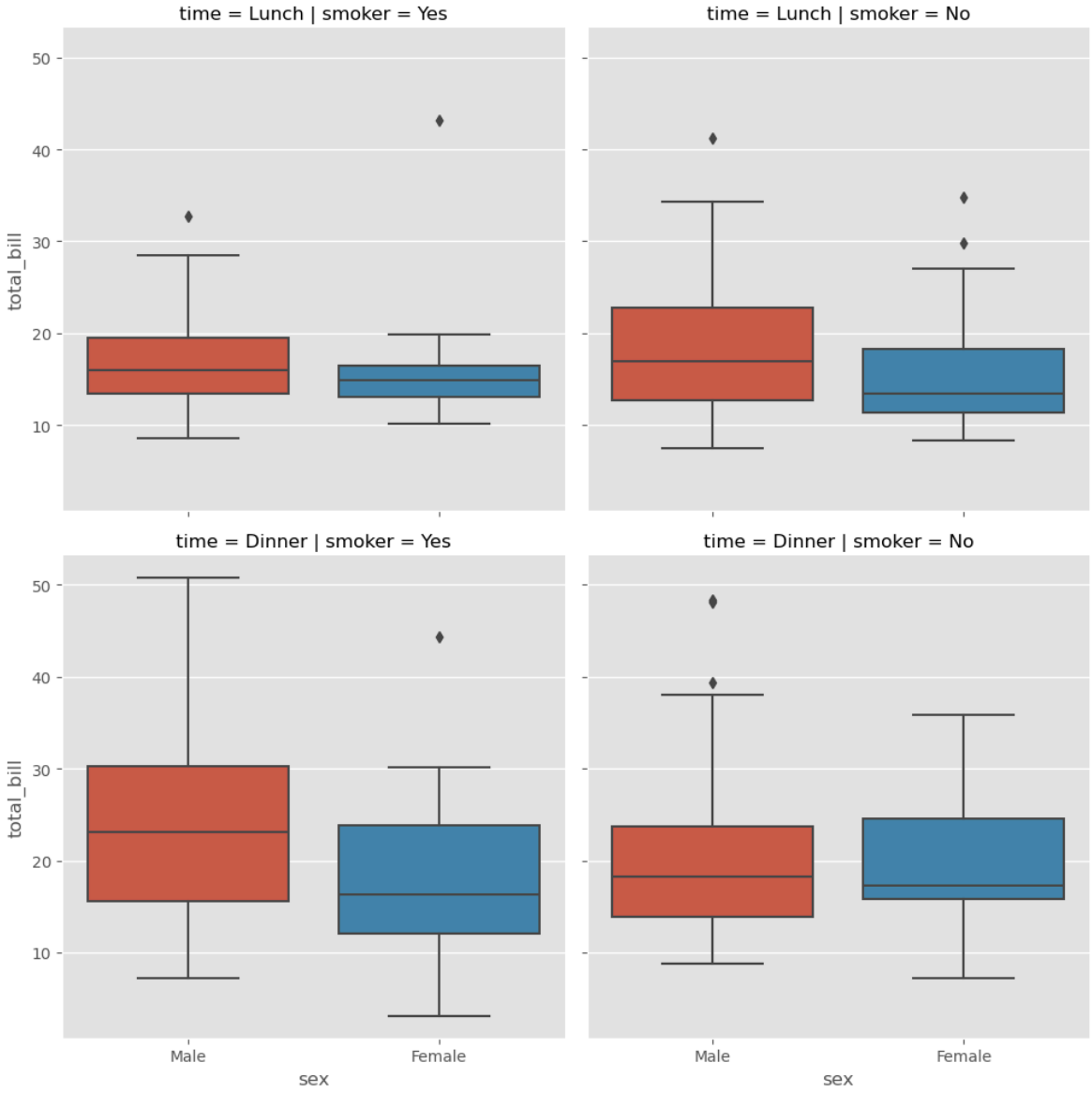


A special case for the bar plot is when you want to show the number of observations in each category rather than computing a statistic for a second variable. This is similar to a histogram over a categorical, rather than quantitative, variable

In [64]:
```
# Faceting using catplot

sns.catplot(data = tips, x = 'sex', y= 'total_bill',col = 'smoker',
            kind = 'box', row = 'time')
```

Out[64]:
```
<seaborn.axisgrid.FacetGrid at 0x26ea0066190>
```



Regression Plots

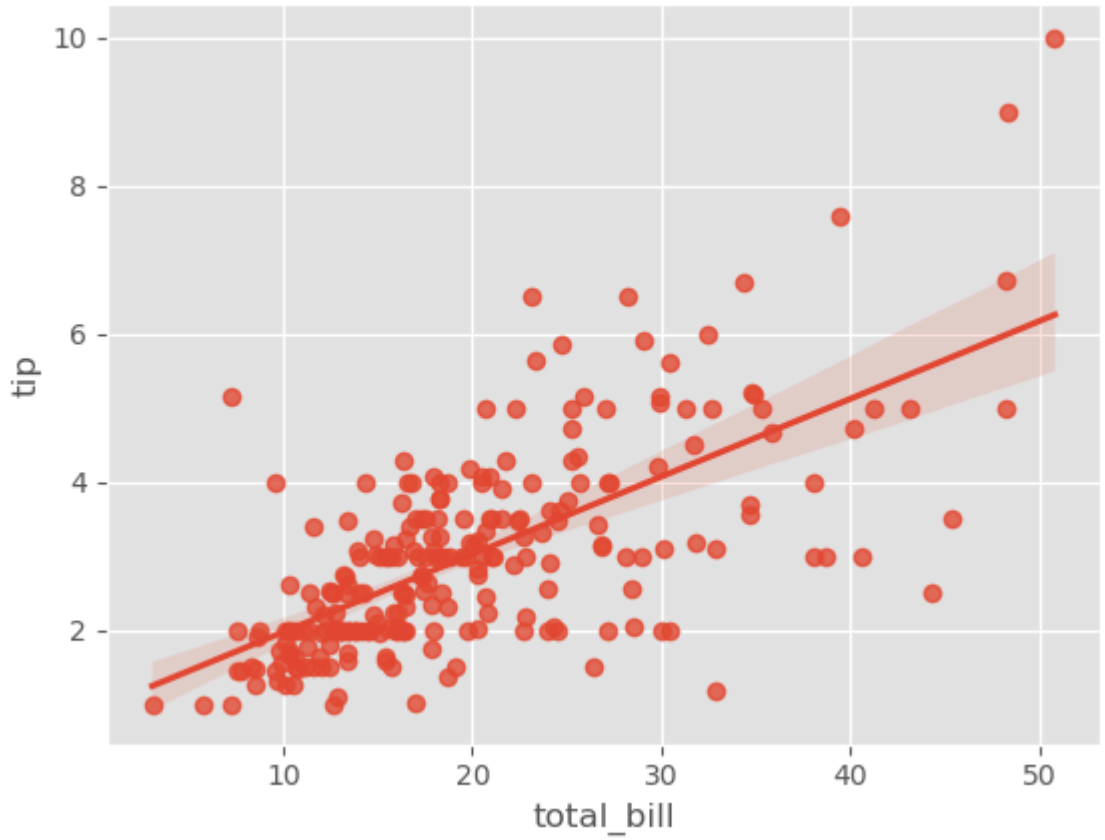
- regplot
- lmplot

In the simplest invocation, both functions draw a scatterplot of two variables, x and y, and then fit the regression model y ~ x and plot the resulting regression line and a 95% confidence interval for that regression.

In [66]:
```
# axis level
# hue parameter is not available

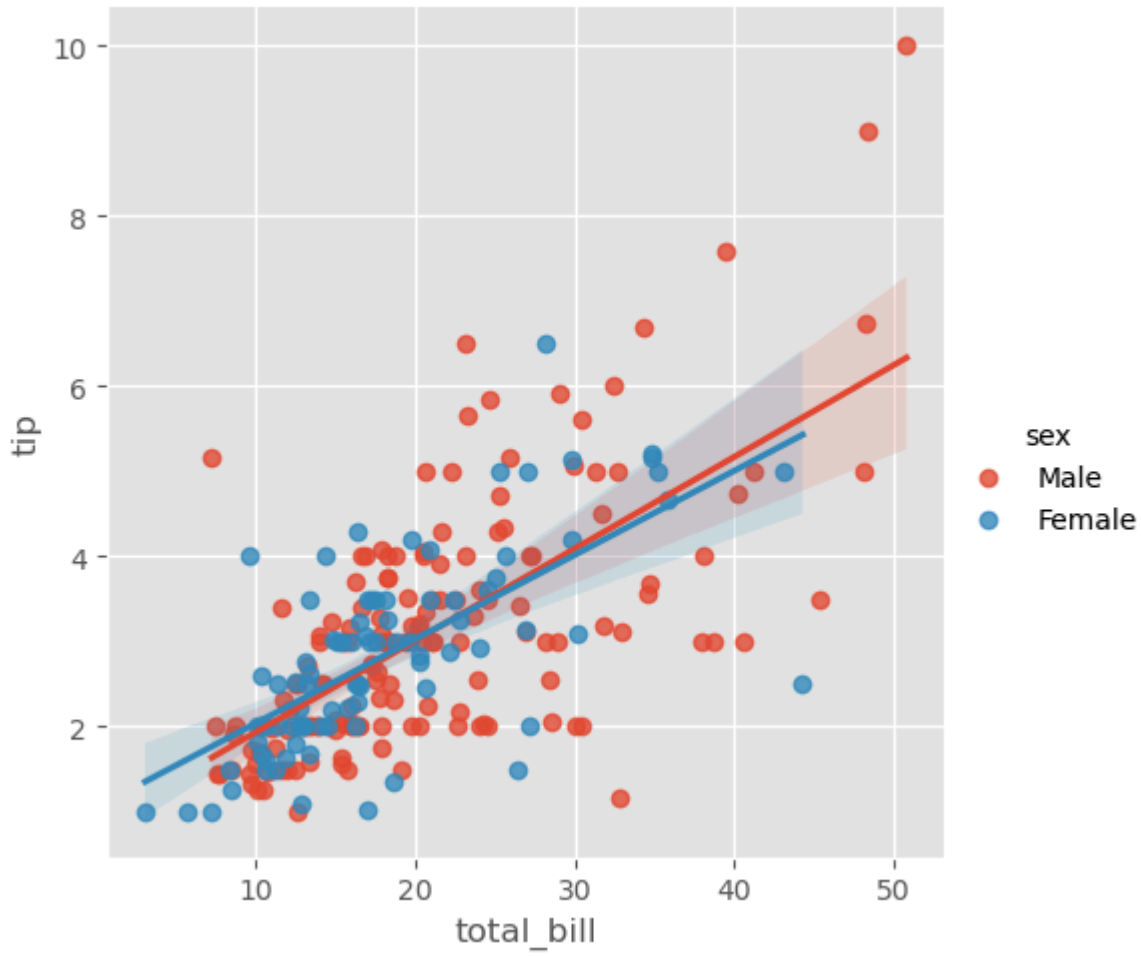
sns.regplot(data = tips, x = 'total_bill', y = 'tip')
```

Out[66]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



In [67]:
```
sns.lmplot(data = tips, x = 'total_bill', y = 'tip', hue = 'sex')
```

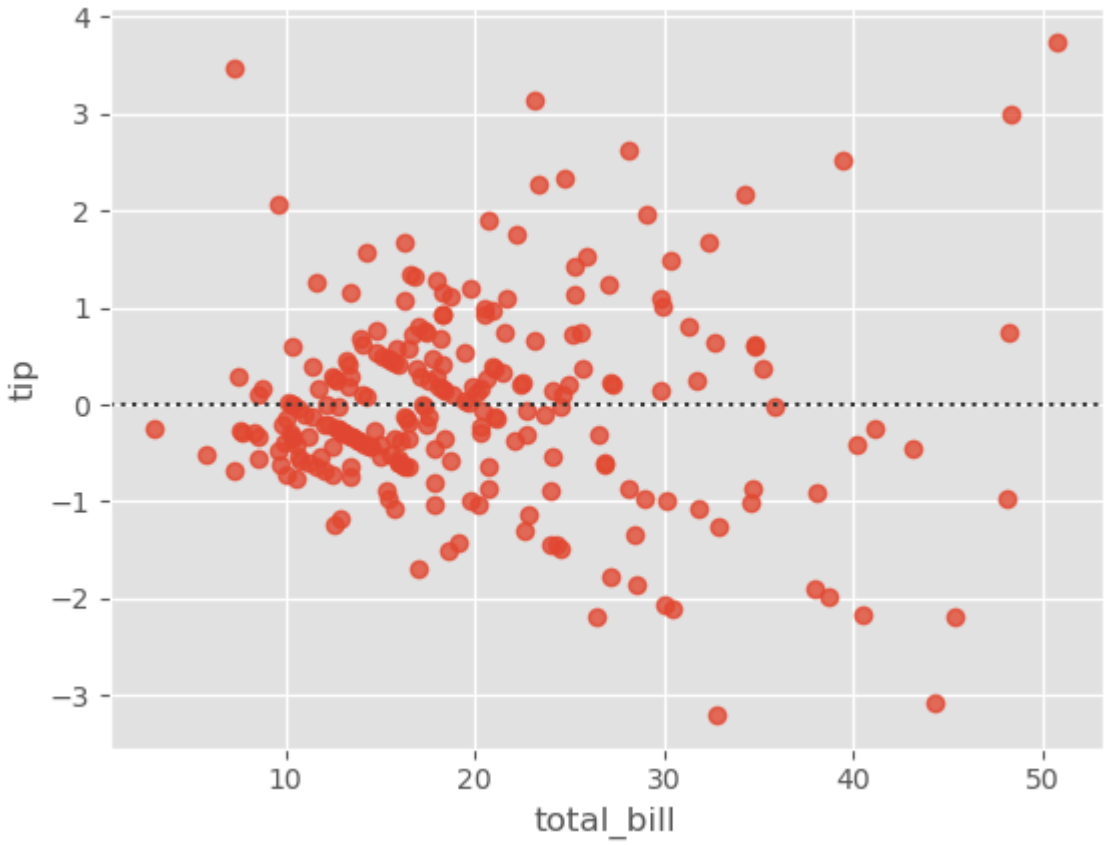
Out[67]: <seaborn.axisgrid.FacetGrid at 0x26e9e9469d0>



In [68]:
```
# residplot

sns.residplot(data = tips, x = 'total_bill', y = 'tip')
```

Out[68]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```

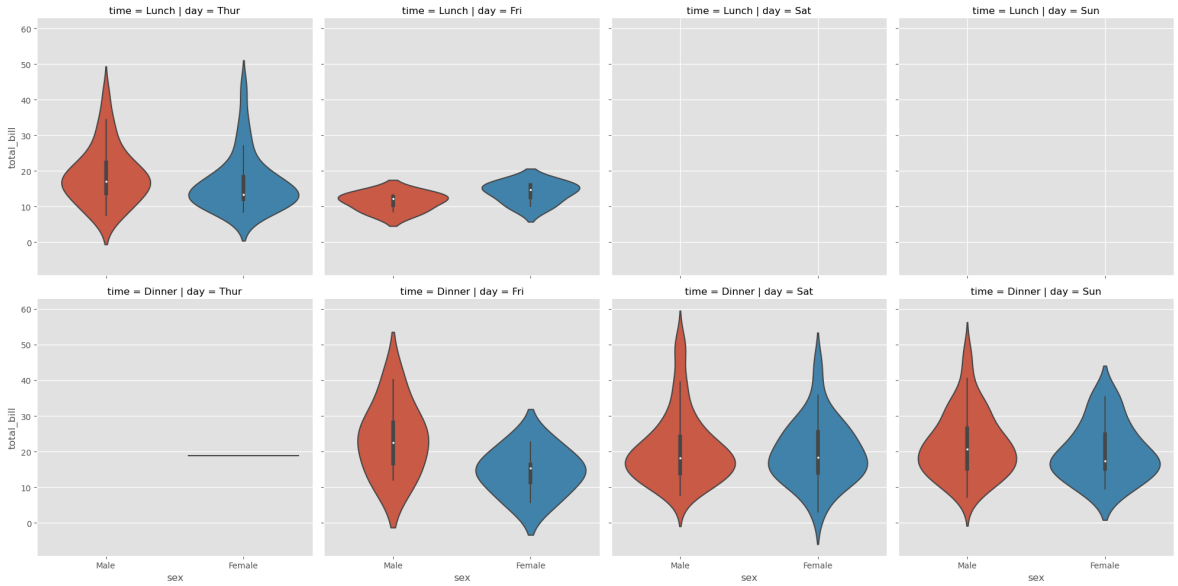


A second way to plot Facet plots -> FacetGrid

In [69]:
```
# figure level -->> relplot -->> distplot -->> catplot -->>lmplot

sns.catplot(data = tips, x = 'sex', y = 'total_bill',kind = 'violin',
            col = 'day', row = 'time')
```

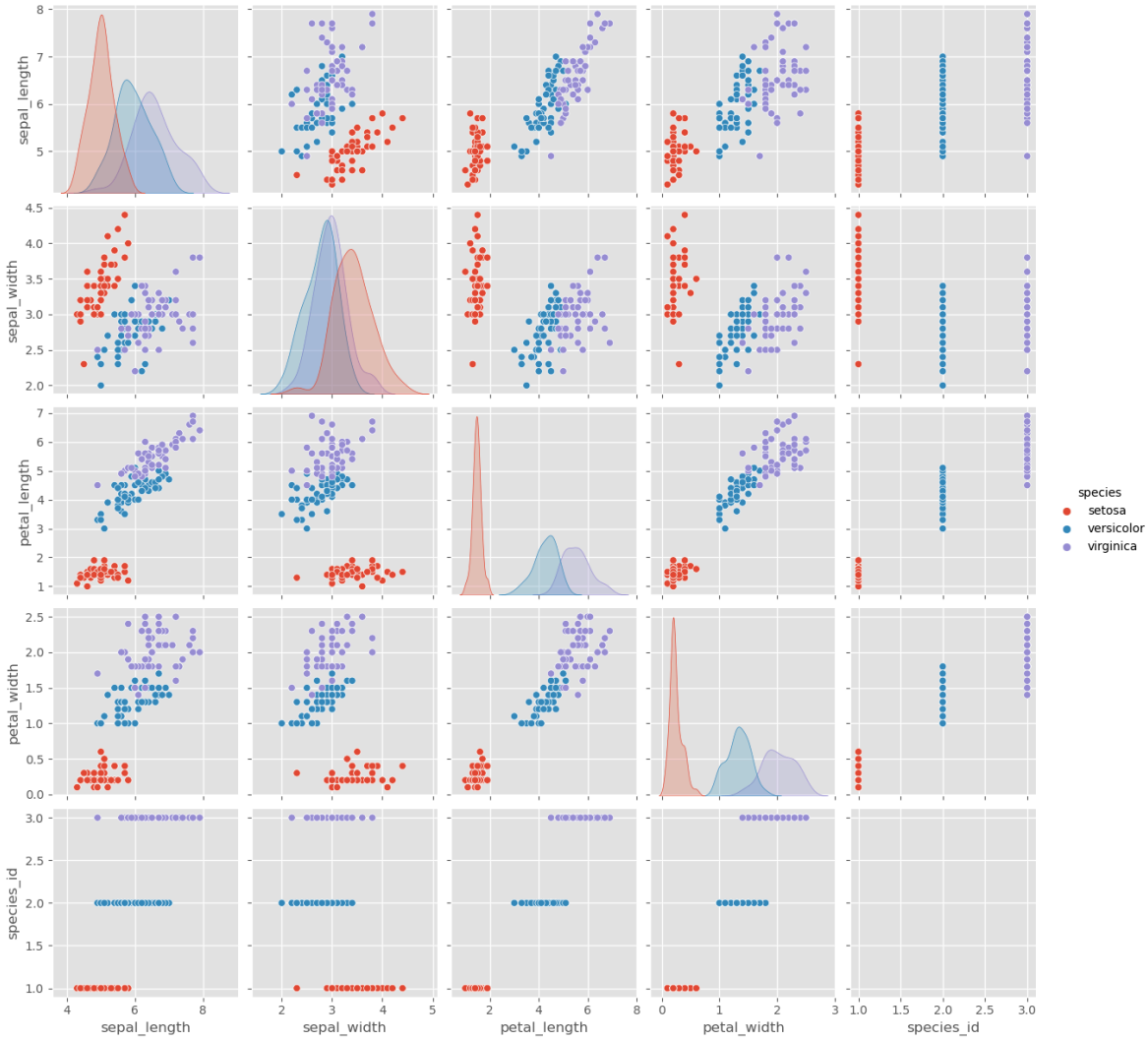
Out[69]:
```
<seaborn.axisgrid.FacetGrid at 0x26ea1d52cd0>
```



Plotting Pairwise Relationship (PairGrid Vs Pairplot)

In [70]:
```
sns.pairplot(iris, hue = 'species')
```

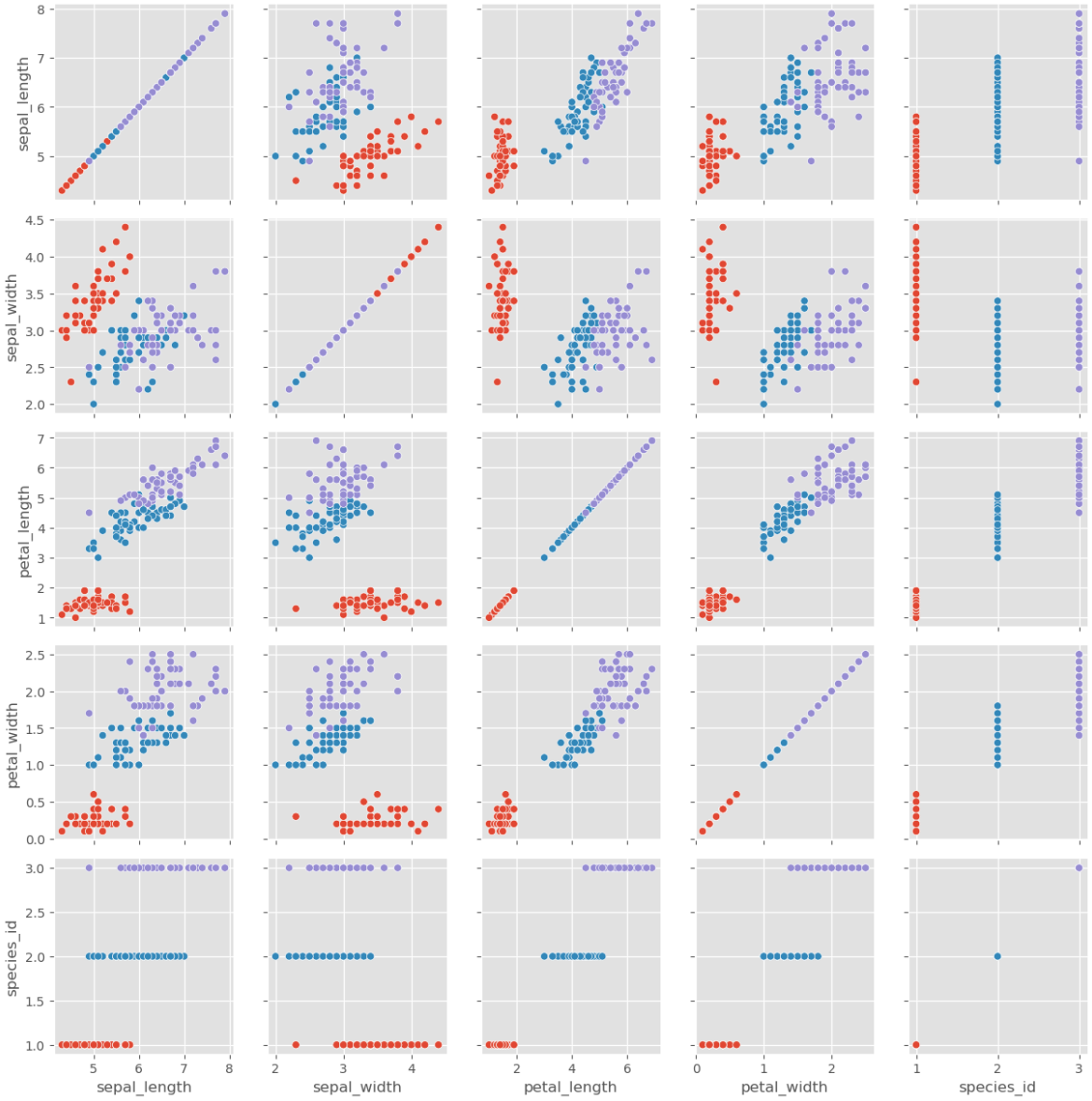
Out[70]: <seaborn.axisgrid.PairGrid at 0x26e9e9c6f90>



In [71]:
```
# pair grid

g = sns.PairGrid(data = iris, hue = 'species')
# g.map
g.map(sns.scatterplot)
```

Out[71]: <seaborn.axisgrid.PairGrid at 0x26ea33e1d50>

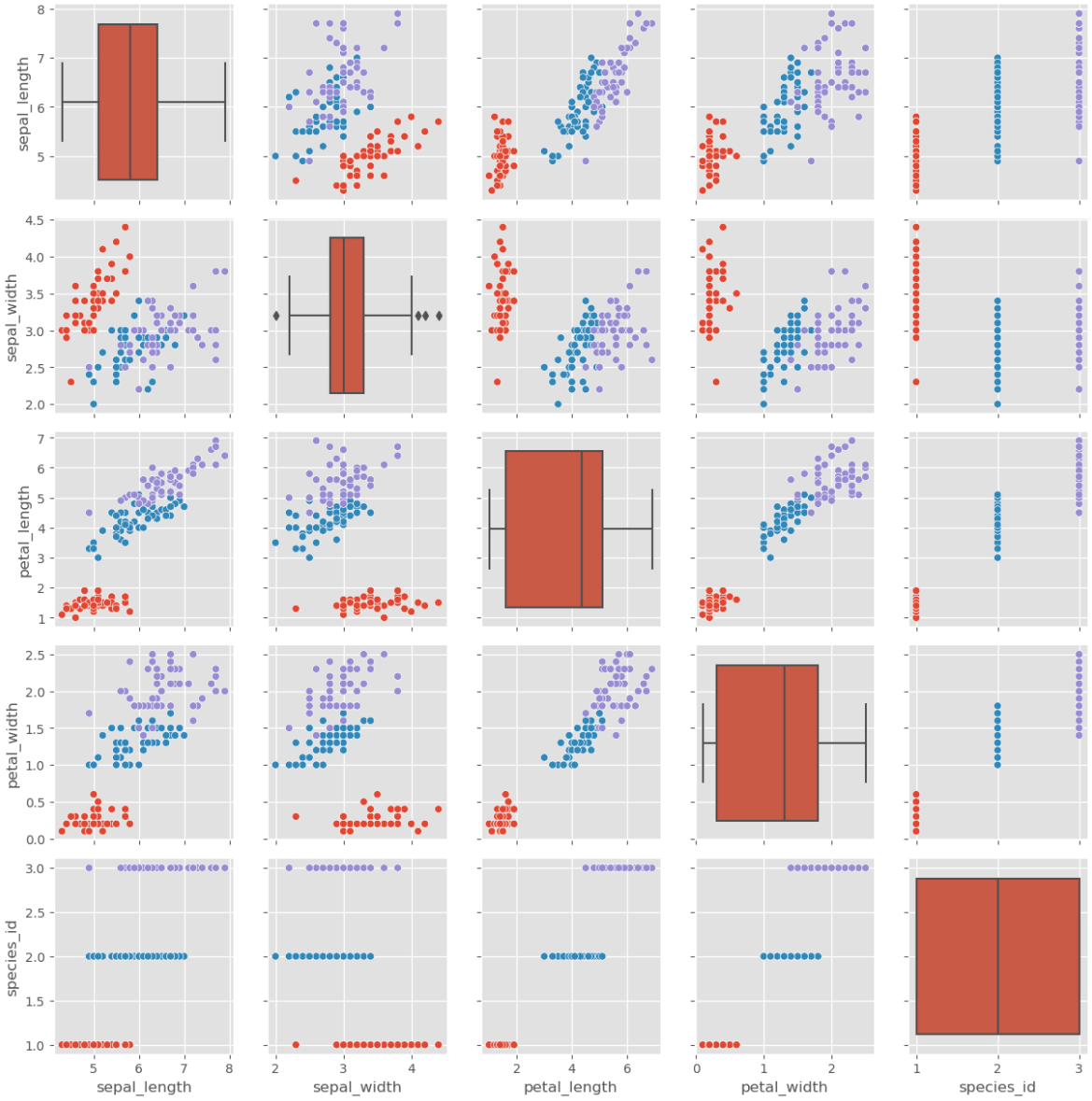


In [72]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [73]:
```
# map_diag -->> map_offdiag

g = sns.PairGrid(data = iris, hue = 'species')
g.map_diag(sns.boxplot)
# g.map_offdiag(sns.kdeplot)     ##  PairGrid.map_offdiag(func, **kwargs)
g.map_offdiag(sns.scatterplot)
```

Out[73]: <seaborn.axisgrid.PairGrid at 0x26ea69b6150>



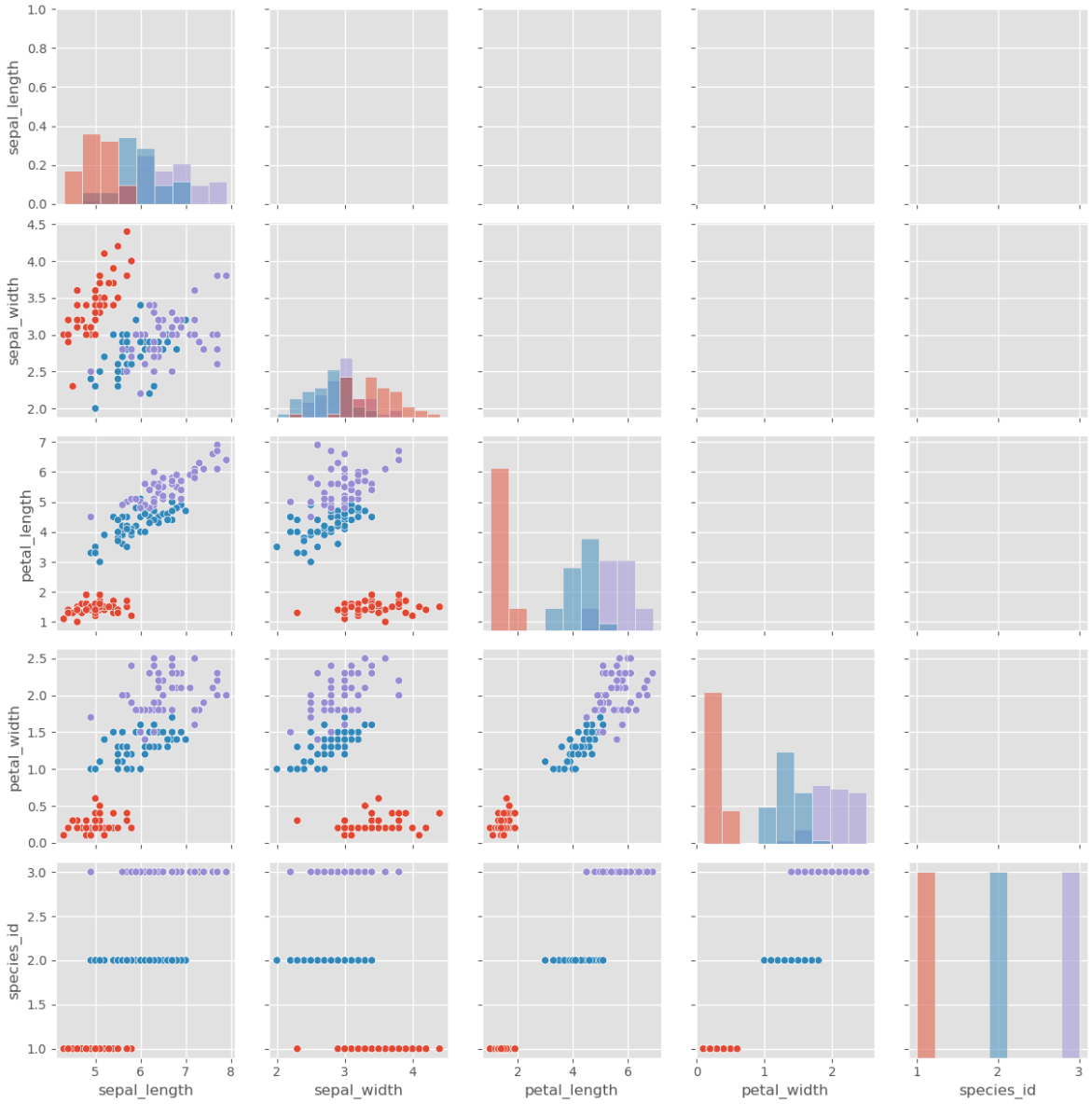
In [74]:
```
print(iris.isnull().sum())
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
species_id      0
dtype: int64
```

In [75]:
```
# map_diag -> map_upper -> map_lower
g = sns.PairGrid(data=iris,hue='species')
g.map_diag(sns.histplot)
print(iris.isnull().sum())
         ## PairGrid.map_upper(func, **kwargs)
g.map_lower(sns.scatterplot)     ## PairGrid.map_lower(func, **kwargs)
```

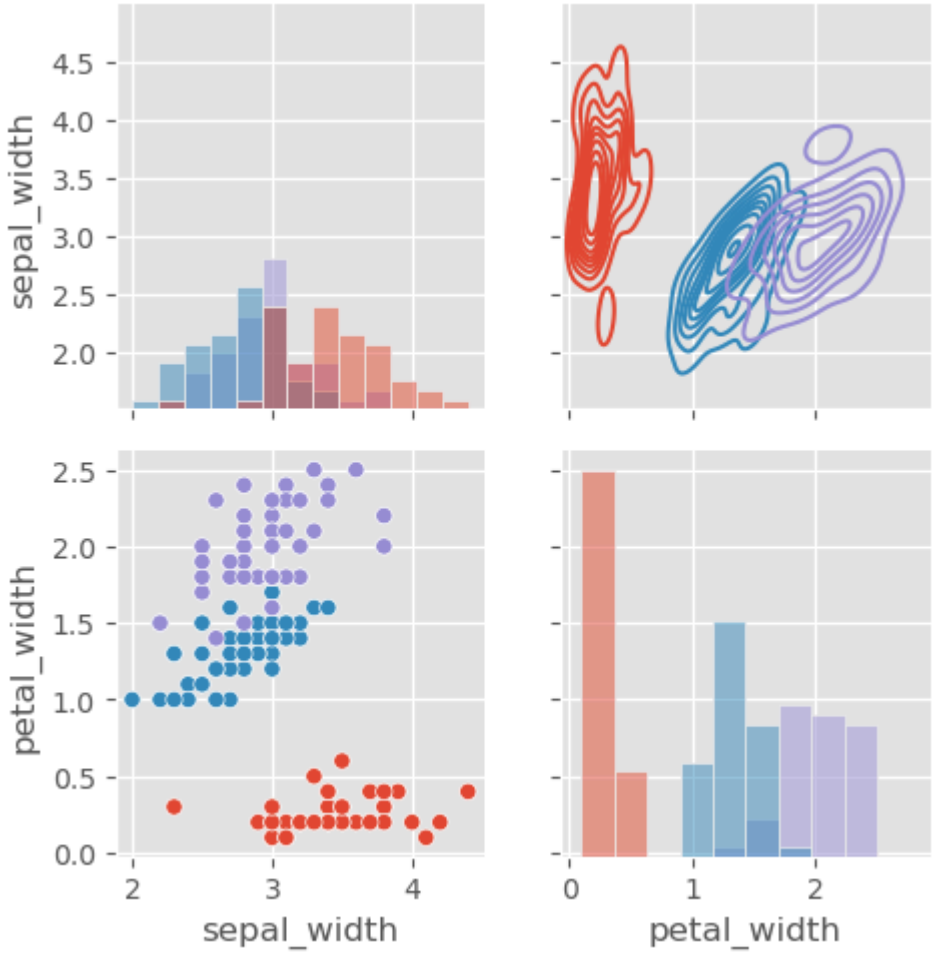
```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
species_id      0
dtype: int64
```

Out[75]: <seaborn.axisgrid.PairGrid at 0x26ea406b910>



In [76]:
```
# vars
g = sns.PairGrid(data=iris,hue='species',vars=['sepal_width','petal_width'])
g.map_diag(sns.histplot)
g.map_upper(sns.kdeplot)
g.map_lower(sns.scatterplot)
```

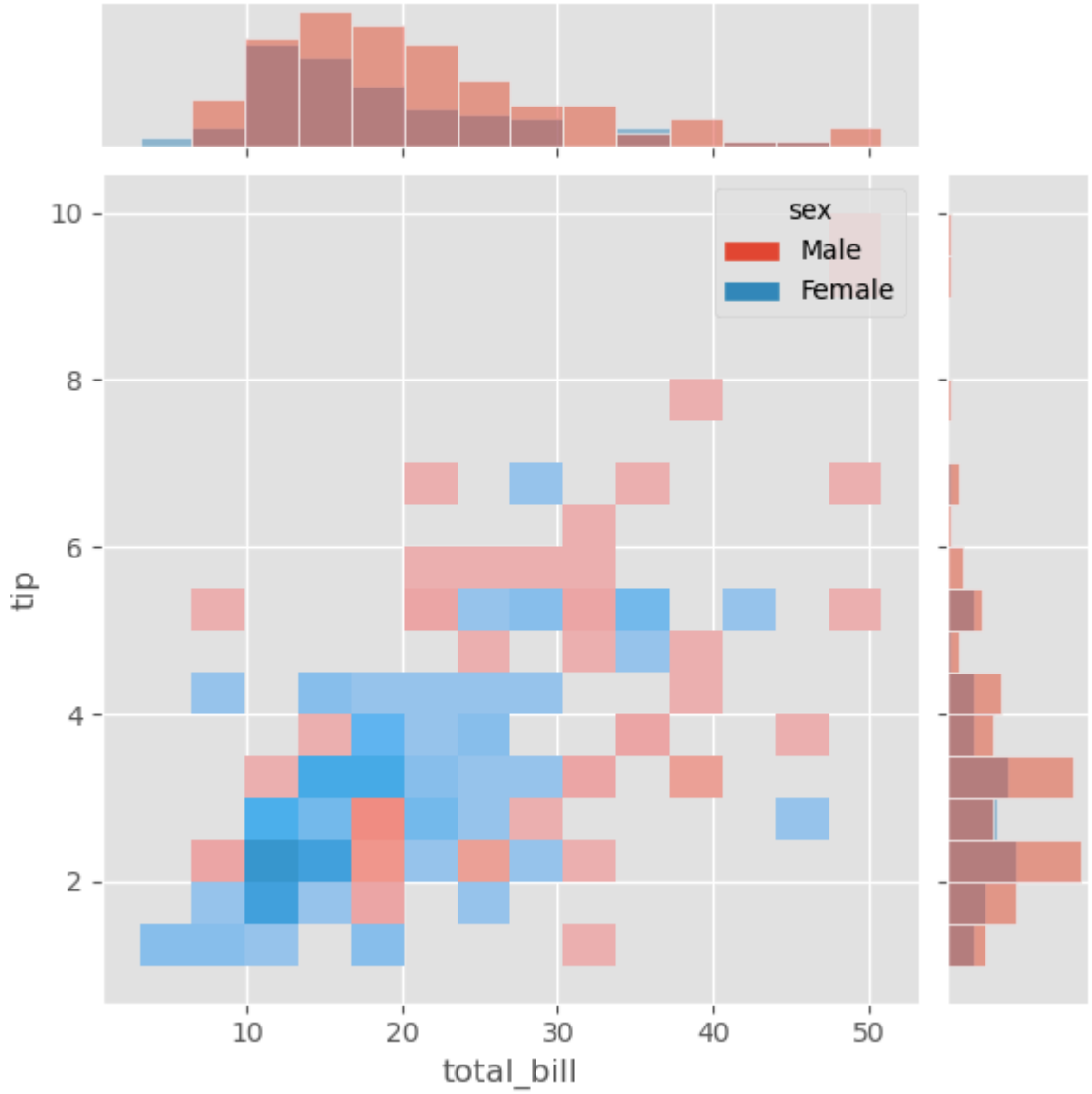
Out[76]: <seaborn.axisgrid.PairGrid at 0x26eaa6ccc90>



JointGrid Vs Jointplot

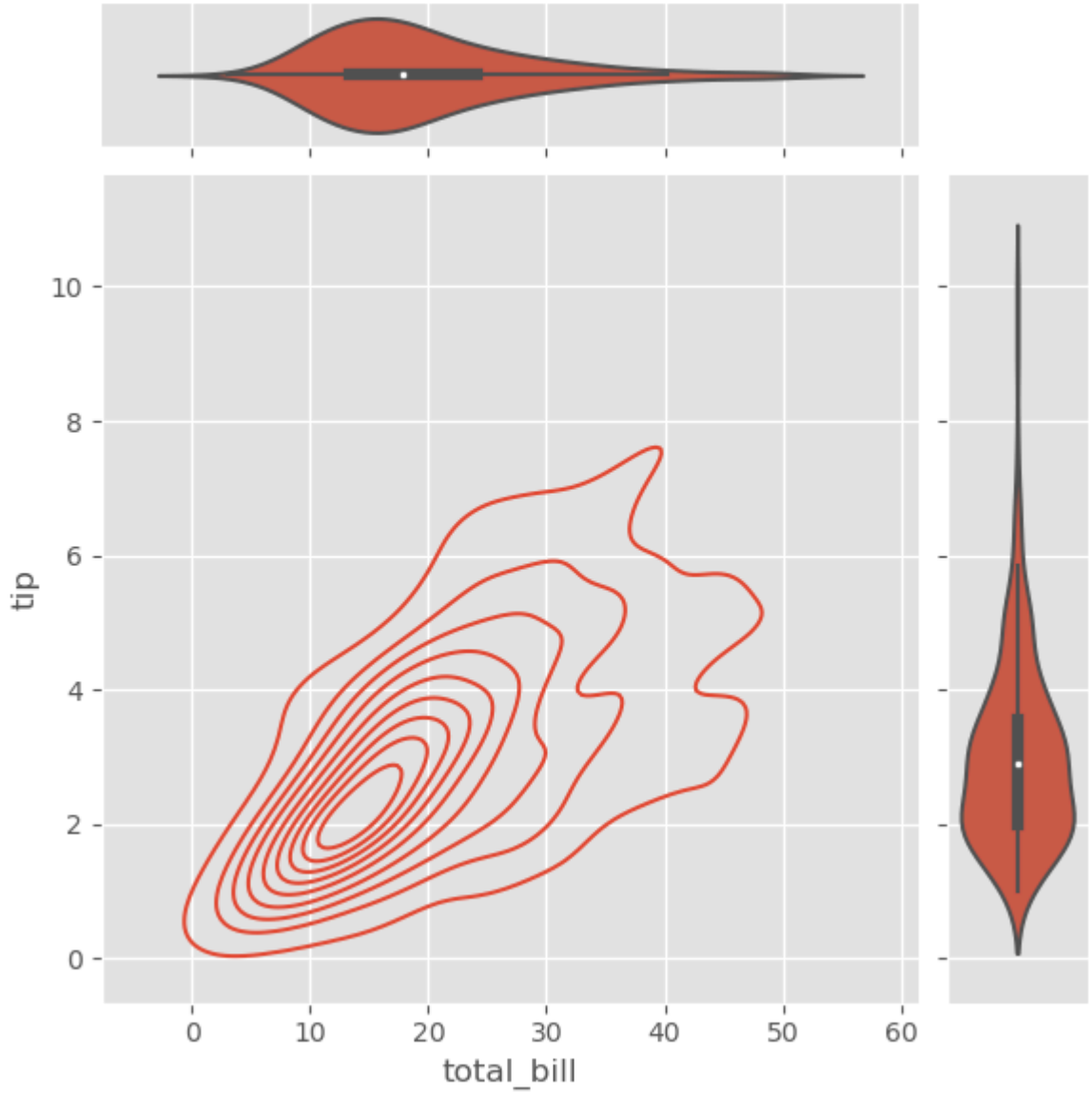
In [77]:
```
sns.jointplot(data=tips,x='total_bill',y='tip',kind='hist',hue='sex')
```

Out[77]:
```
<seaborn.axisgrid.JointGrid at 0x26eaa7d0b50>
```



In [78]:
```
g = sns.JointGrid(data=tips,x='total_bill',y='tip')
g.plot(sns.kdeplot,sns.violinplot)
```

Out[78]: <seaborn.axisgrid.JointGrid at 0x26eaa5abc90>



Utility Functions

In [79]:
```
# get dataset names

sns.get_dataset_names()
```

Out[79]:
```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic',
 'anagrams',
 'anagrams',
 'anscombe',
 'anscombe',
 'attention',
 'attention',
 'brain_networks',
 'brain_networks',
 'car_crashes',
 'car_crashes',
 'diamonds',
 'diamonds',
 'dots',
 'dots',
 'dowjones',
 'dowjones',
 'exercise',
 'exercise',
 'flights',
 'flights',
 'fmri',
 'fmri',
 'geyser',
 'geyser',
 'glue',
 'glue',
 'healthexp',
 'healthexp',
 'iris',
 'iris',
 'mpg',
 'mpg',
 'penguins',
 'penguins',
 'planets',
 'planets',
 'seaice',
 'seaice',
```

```
'taxis',
'taxis',
'tips',
'tips',
'titanic',
'titanic',
'anagrams',
'anscombe',
'attention',
'brain_networks',
'car_crashes',
'diamonds',
'dots',
'dowjones',
'exercise',
'flights',
'fmri',
'geyser',
'glue',
'healthexp',
'iris',
'mpg',
'penguins',
'planets',
'seaice',
'taxis',
'tips',
'titanic']
```

In [80]:
```
# Load dataset

planets = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/re
```

In [81]:
```
import seaborn as sns
import matplotlib.pyplot as plt


tips = sns.load_dataset('tips')
```

Themeing

- `set_theme`

  > Set aspects of the visual theme for all matplotlib and seaborn plots.

- `axes_style`

  > Get the parameters that control the general style of the plots.

- `set_style`

  > Set the parameters that control the general style of the plots.

- `plotting_context`

  > Get the parameters that control the scaling of plot elements.

- `set_context`

  > Set the parameters that control the scaling of plot elements.

- `set_color_codes`

  > Change how matplotlib color shorthands are interpreted.

- `reset_defaults`

  > Restore all RC params to default settings.

- `reset_orig`

  > Restore all RC params to original settings (respects custom rc).

`set_theme` function :

> This function is used to set the theme of your plots, it can take a variety of inputs such as 'darkgrid', 'whitegrid', 'dark', 'white' or 'ticks'.

Example:

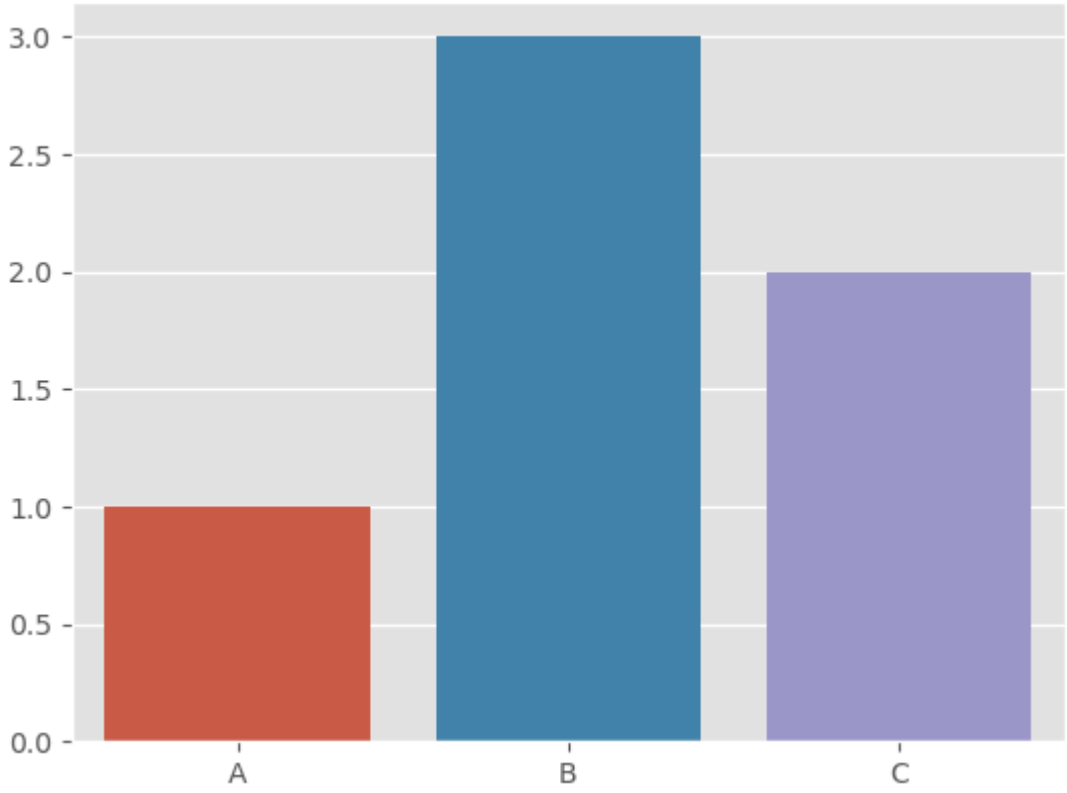
In [83]:
```
tips.head()
```

Out[83]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

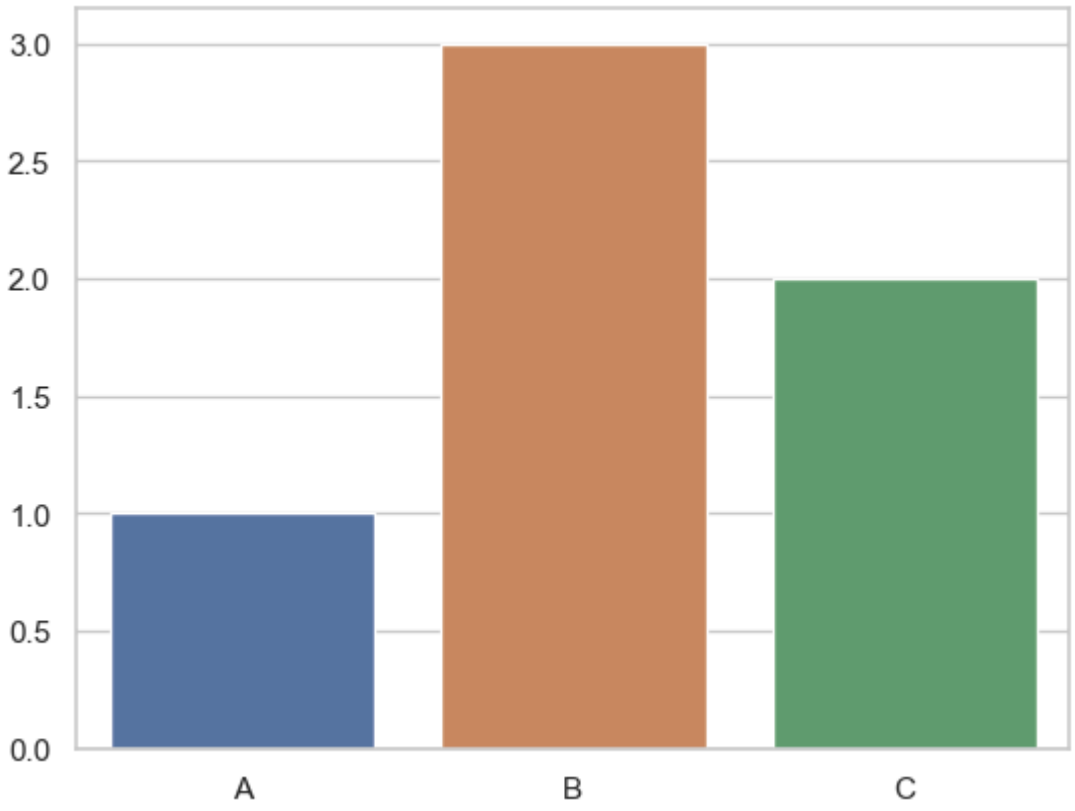
In [84]:
```
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Out[84]:
```
<Axes: >
```



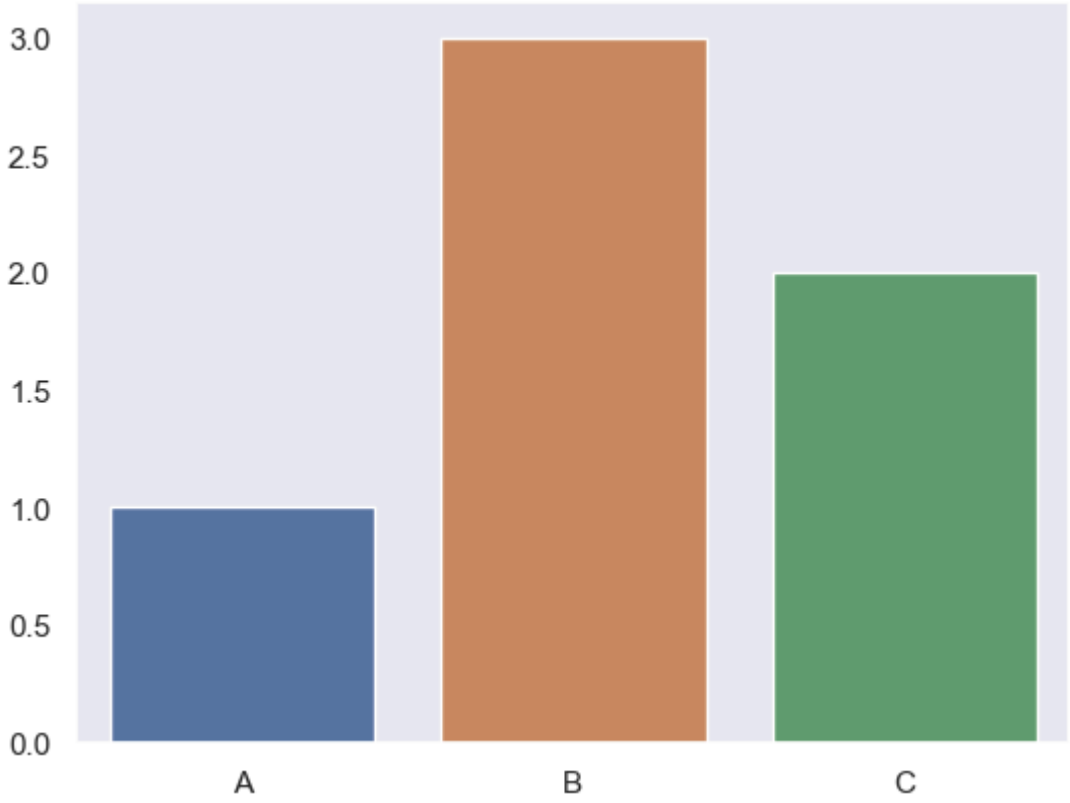
In [85]:
```
# Using whitegrid
sns.set_theme(style='whitegrid')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Out[85]: <Axes: >



In [86]:
```
# Using dark background
sns.set_theme(style='dark')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Out[86]: <Axes: >

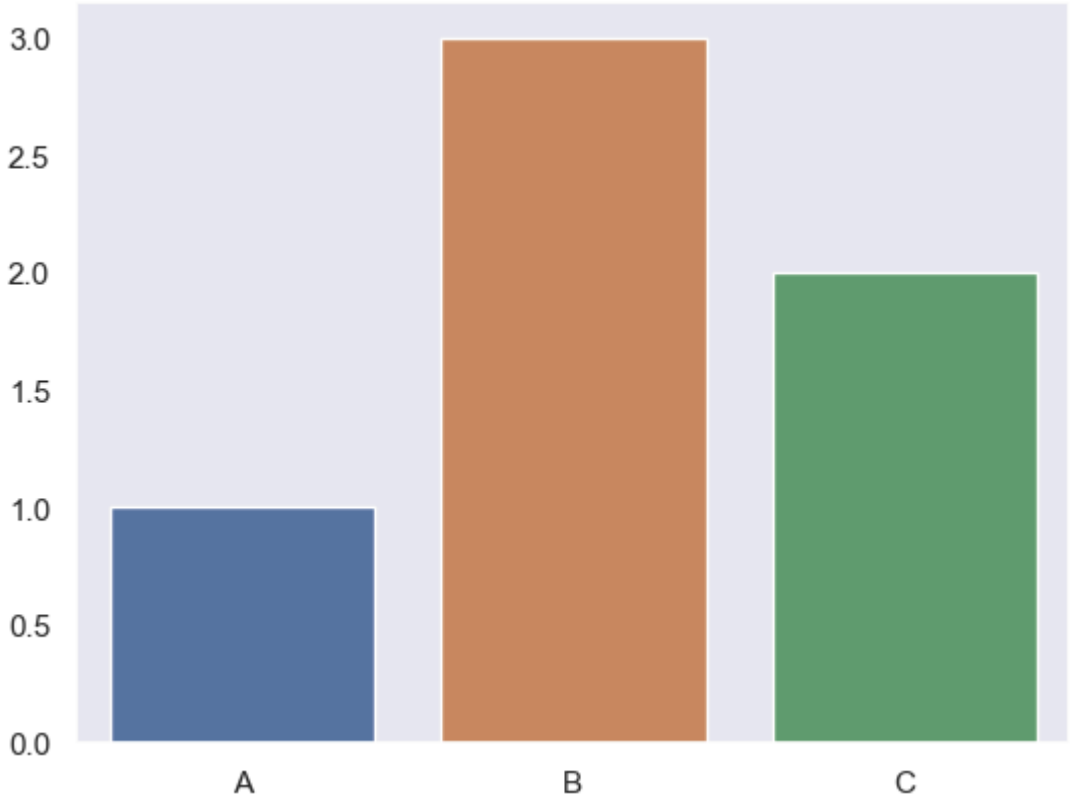


`axes_style` function :

This function is used to set the style of the axes of your plots. It can take a variety of inputs such as 'white', 'dark', 'ticks' or a dictionary with key-value pairs of valid style options.

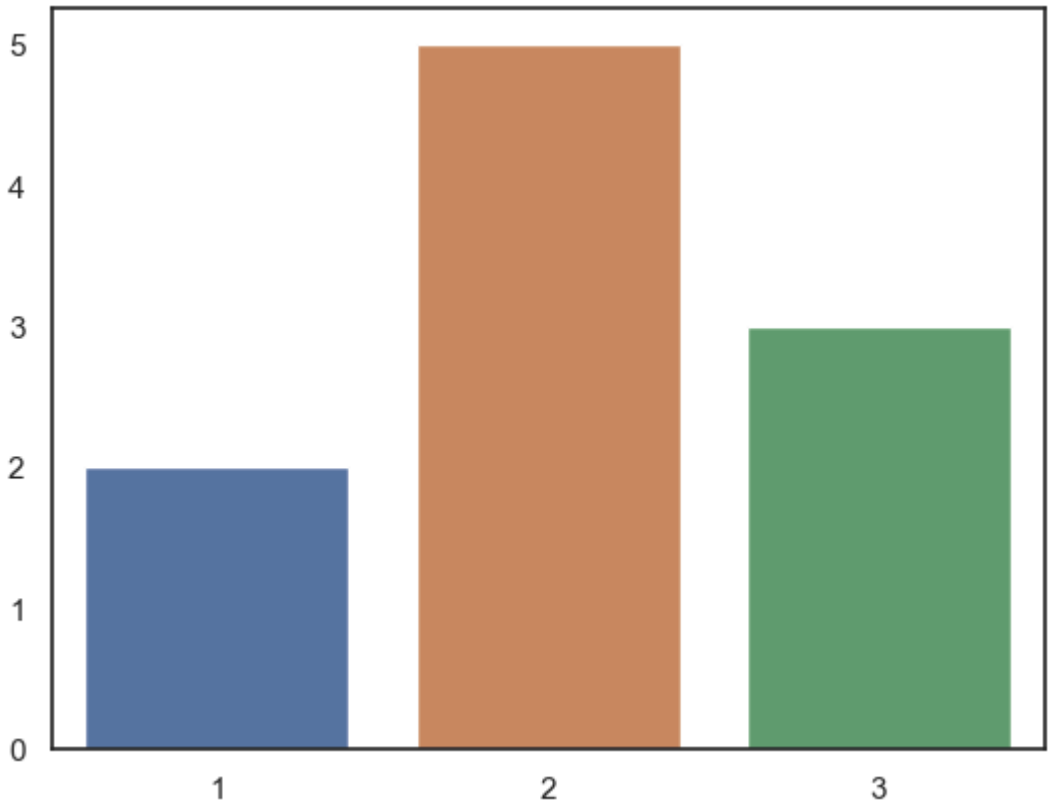
In [87]:
```
# Example:
sns.axes_style(style = 'white')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Out[87]: <Axes: >



In [88]:
```
# Use the function as a context manager to temporarily change the style of your
#                                                                          plots:

with sns.axes_style("white"):
    sns.barplot(x=[1, 2, 3], y=[2, 5, 3])
```



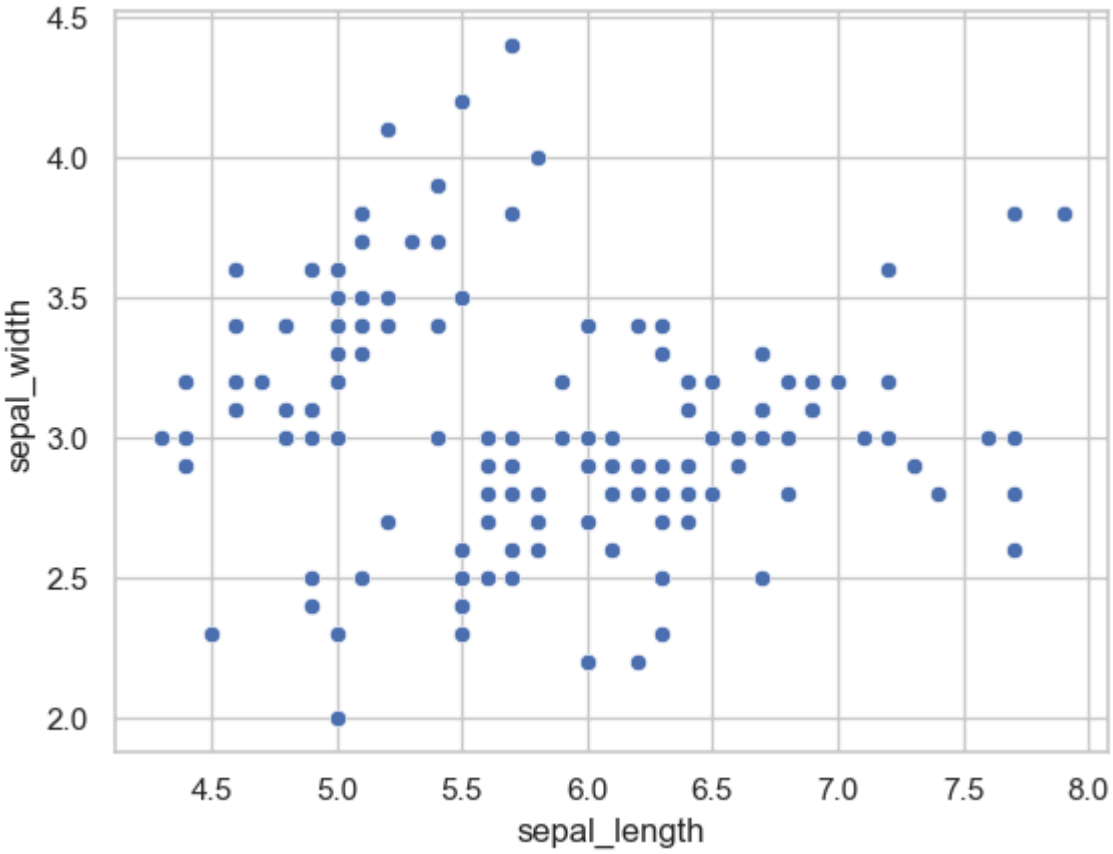
In [89]:
```
sns.get_data_home()
```

Out[89]: 'C:\\Users\\goura\\AppData\\Local\\seaborn\\seaborn\\Cache'

In [91]:
```
iris = sns.load_dataset('iris')
# Set the style of the axes to "darkgrid"
sns.set_style("whitegrid")

# Create a scatter plot of sepal length vs sepal width
sns.scatterplot(x="sepal_length", y="sepal_width", data=iris)

# Show the plot
plt.show()
```



Scaling Figure Styles - `sns.set_context()`

Matplotlib allows you to generate powerful plots, but styling those plots for different presentation purposes is difficult. Seaborn makes it easy to produce the same plots in a variety of different visual formats so you can customize the presentation of your data for the appropriate context, whether it be a research paper or a conference poster.

You can set the visual format, or context, using `sns.set_context()`

Within the usage of `sns.set_context()`, there are three levels of complexity:

- Pass in one parameter that adjusts the scale of the plot
- Pass in two parameters - one for the scale and the other for the font size
- Pass in three parameters - including the previous two, as well as the rc with the style parameter that you want to override

Scaling Plots

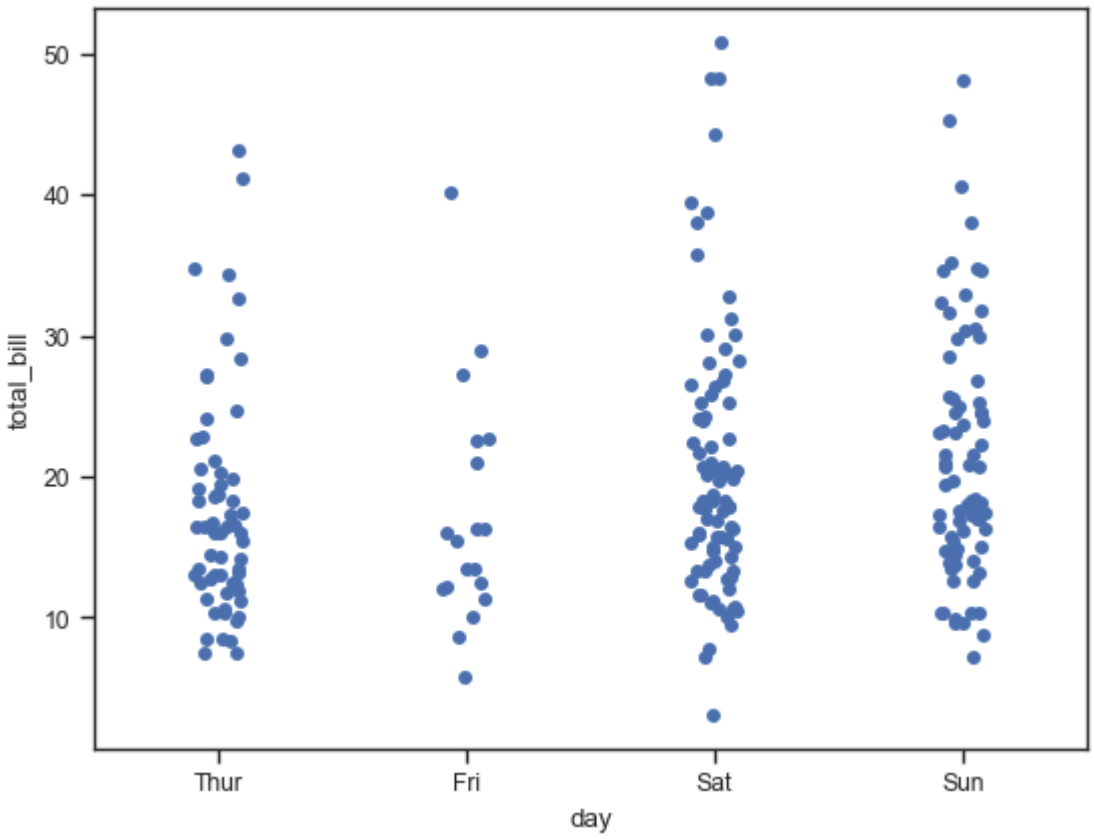
Seaborn has four presets which set the size of the plot and allow you to customize your figure depending on how it will be presented.

In order of relative size they are: `paper`, `notebook`, `talk`, and `poster`. The notebook style is the default.

In [92]:
```
sns.set_style("ticks")

# Smallest context: paper
sns.set_context("paper")
sns.stripplot(x="day", y="total_bill", data=tips)
```

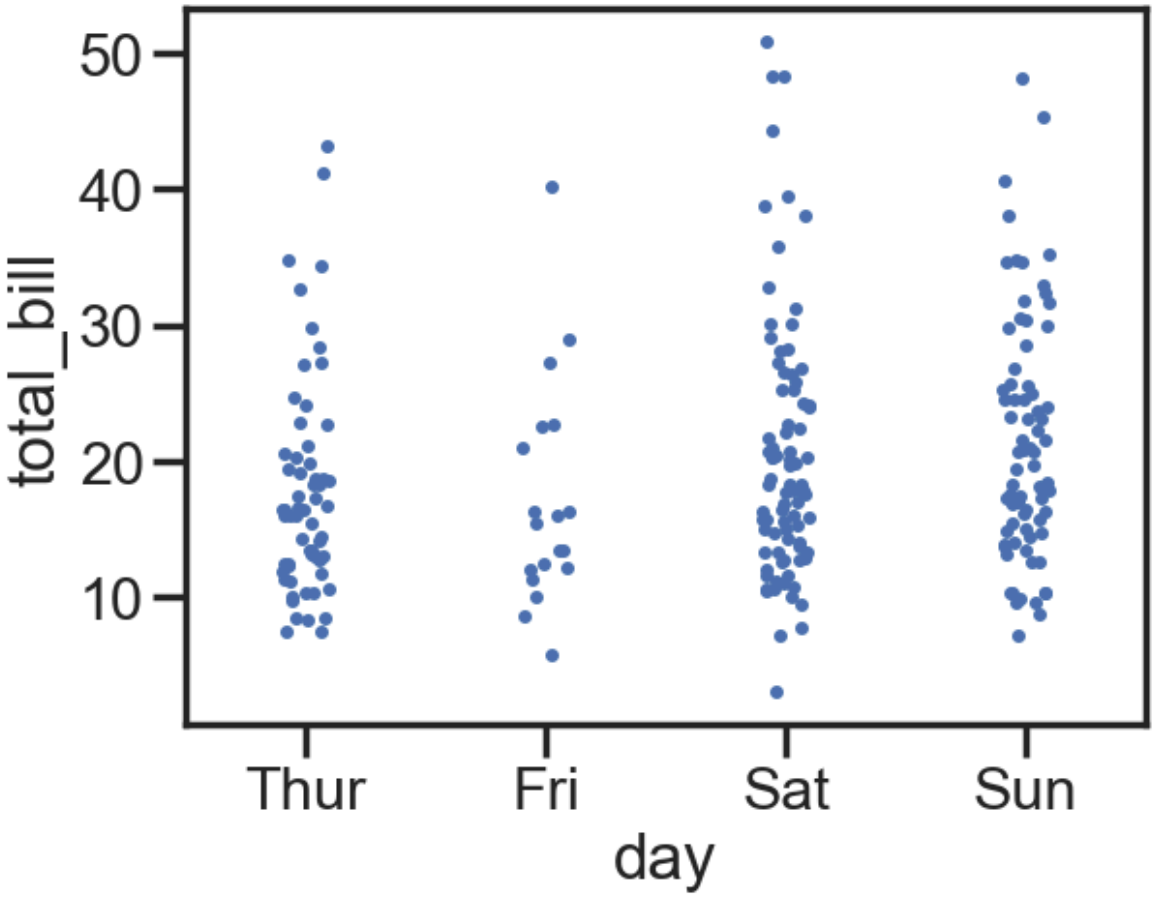
Out[92]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



In [93]:
```
sns.set_style("ticks")

# Largest Context: poster
sns.set_context("poster")
sns.stripplot(x="day", y="total_bill", data=tips)
```

Out[93]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



Scaling Fonts and Line Widths

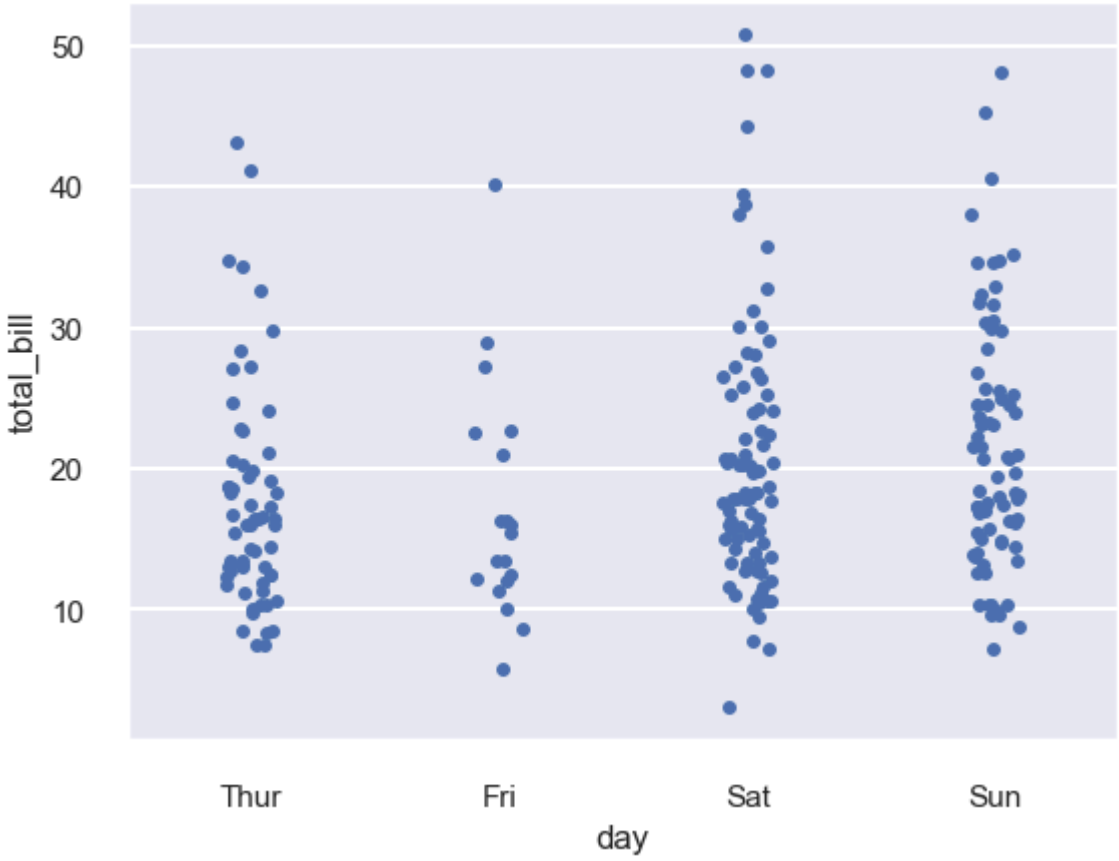
You are also able to change the size of the text using the font_scale parameter for `sns.set_context()`

You may want to also change the line width so it matches. We do this with the rc parameter, which we'll explain in detail below.

In [94]:
```
# Set font scale and reduce grid line width to match
sns.set_style("darkgrid")

sns.set_context("poster", font_scale = .5, rc={"grid.linewidth": 1.5})
sns.stripplot(x="day", y="total_bill", data=tips)
```

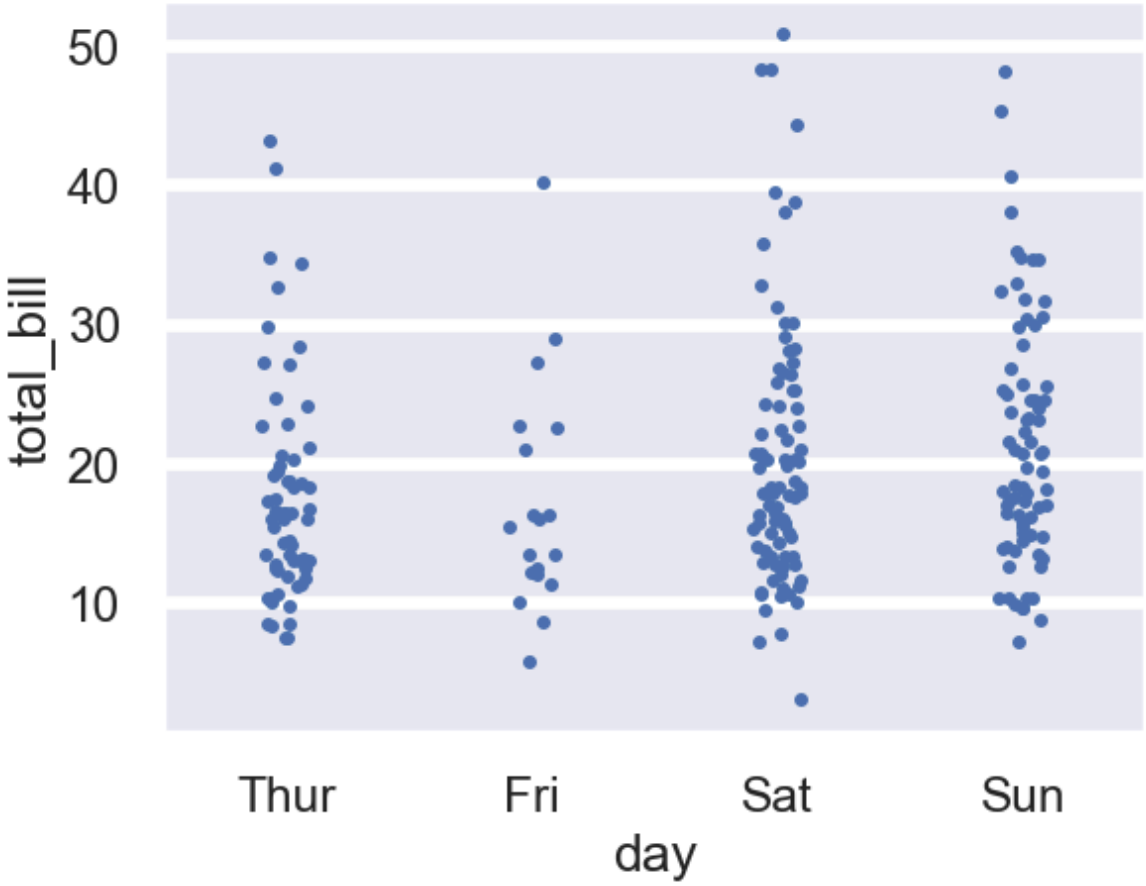
Out[94]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



> While you're able to change these parameters, you should keep in mind that it's not always useful to make certain changes. Notice in this example that we've changed the line width, but because of it's relative size to the plot, it distracts from the actual plotted data.

In [95]:
```
# Set font scale and increase grid line width to match
sns.set_context("poster", font_scale = .8, rc={"grid.linewidth": 5})
sns.stripplot(x="day", y="total_bill", data=tips)
```

Out[95]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



The RC Parameter

As we mentioned above, if you want to override any of these standards, you can use `sns.set_context` and pass in the parameter rc to target and reset the value of an individual parameter in a dictionary. `rc` stands for the phrase 'run command' - essentially, configurations which will execute when you run your code.

In [96]:
```
# Plotting context function

sns.plotting_context()
# These are the property you can tweak in rc parameter
```

Out[96]:
```
{'font.size': 19.200000000000003,
 'axes.labelsize': 19.200000000000003,
 'axes.titlesize': 19.200000000000003,
 'xtick.labelsize': 17.6,
 'ytick.labelsize': 17.6,
 'legend.fontsize': 17.6,
 'legend.title_fontsize': 19.200000000000003,
 'axes.linewidth': 2.5,
 'grid.linewidth': 5.0,
 'lines.linewidth': 3.0,
 'lines.markersize': 12.0,
 'patch.linewidth': 2.0,
 'xtick.major.width': 2.5,
 'ytick.major.width': 2.5,
 'xtick.minor.width': 2.0,
 'ytick.minor.width': 2.0,
 'xtick.major.size': 12.0,
 'ytick.major.size': 12.0,
 'xtick.minor.size': 8.0,
 'ytick.minor.size': 8.0}
```

`seaborn.set_color_codes(palette='deep')`

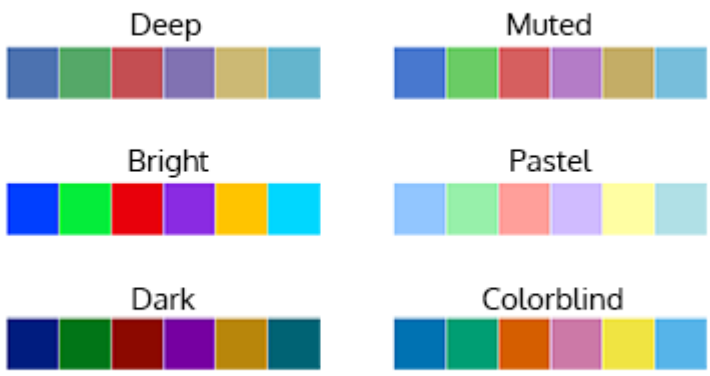
Change how matplotlib color shorthands are interpreted.

Calling this will change how shorthand codes like "b" or "g" are interpreted by matplotlib in subsequent plots.

Parameters:

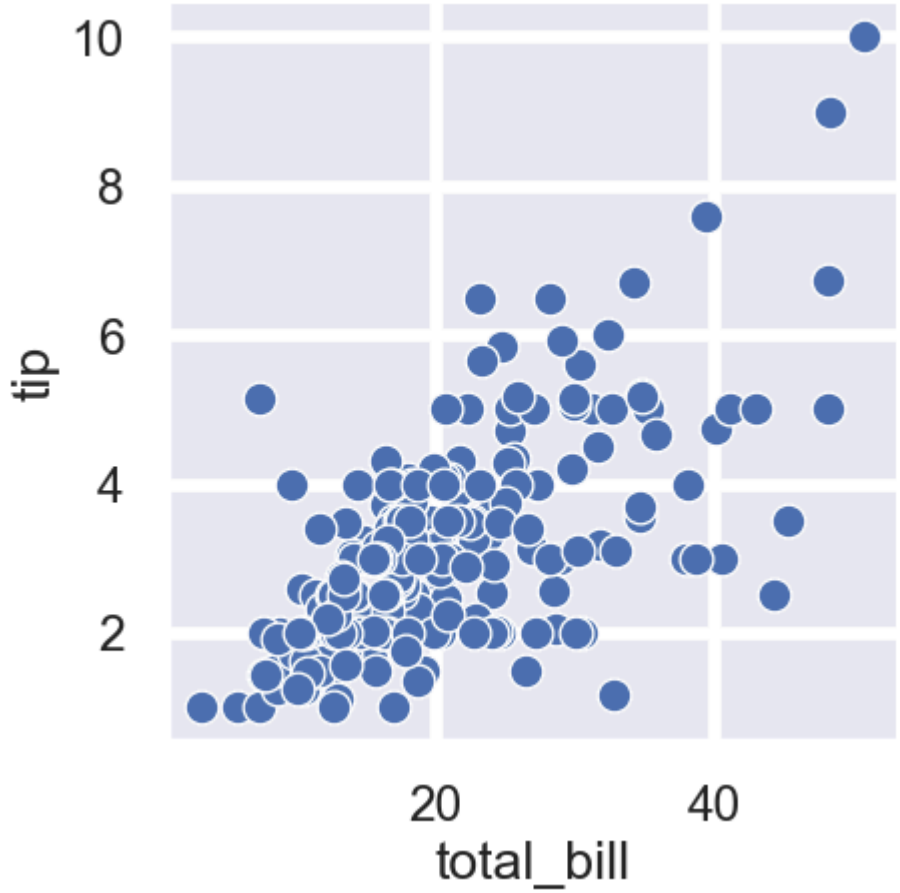
> `palette` : `{deep, muted, pastel, dark, bright, colorblind}`

Named seaborn palette to use as the source of colors.



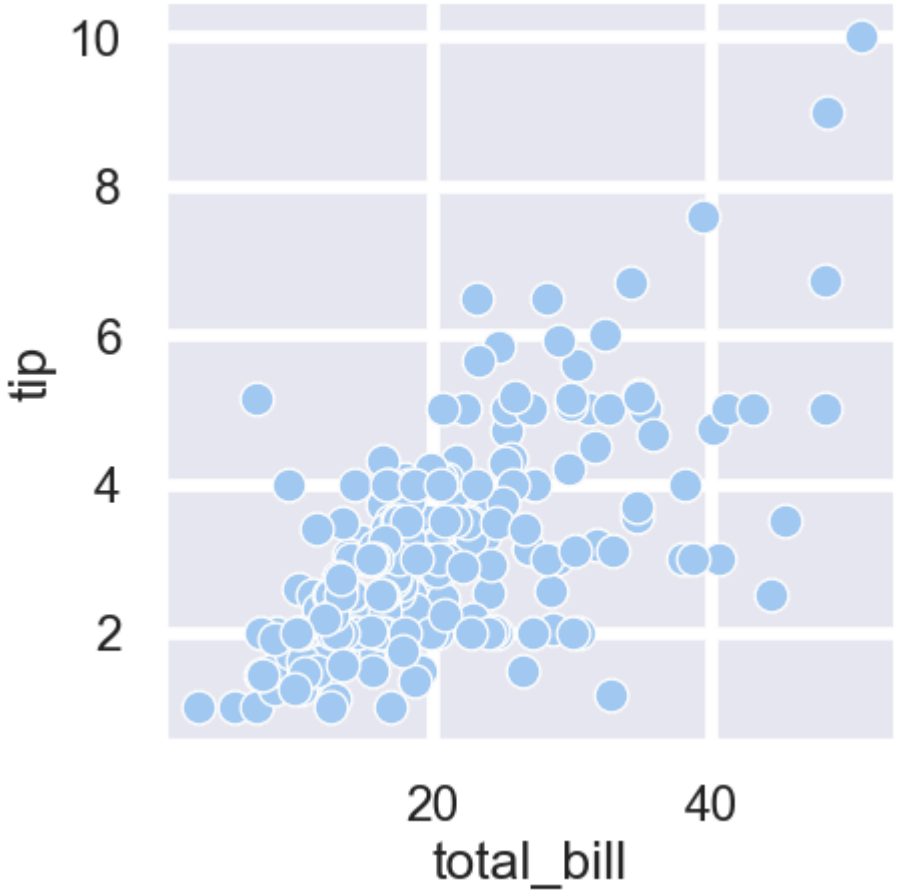
In [102…
```
# 'b' color code with deep palette
sns.set_color_codes(palette='deep')
sns.relplot(data=tips, x='total_bill', y='tip', color='b')
```

Out[102…
```
<seaborn.axisgrid.FacetGrid at 0x26eab794d50>
```



In [103…
```
# 'b' color code with pastel palette
sns.set_color_codes(palette='pastel')
sns.relplot(data=tips, x='total_bill', y='tip', color='b')
```

Out[103…
```
<seaborn.axisgrid.FacetGrid at 0x26eab810d50>
```



Color palettes

- set_palette

  > Set the matplotlib color cycle using a seaborn palette.

- color_palette

  > Return a list of colors or continuous colormap defining a palette.

- husl_palette

  > Return hues with constant lightness and saturation in the HUSL system.

- hls_palette

  > Return hues with constant lightness and saturation in the HLS system.

- cubehelix_palette

  > Make a sequential palette from the cubehelix system.

- dark_palette

  > Make a sequential palette that blends from dark to color.

- light_palette

> Make a sequential palette that blends from light to color.

- diverging_palette

  > Make a diverging palette between two HUSL colors.

- blend_palette

  > Make a palette that blends between a list of colors.

- xkcd_palette

  > Make a palette with color names from the xkcd color survey.

- crayon_palette

  > Make a palette with color names from Crayola crayons.

- mpl_palette

  > Return a palette or colormap from the matplotlib registry.

`color_palette`

https://seaborn.pydata.org/generated/seaborn.color_palette.html#seaborn.color_palette

In Seaborn, the `color_palette()` function allows you to easily specify the colors for your plots. You can use pre-defined palettes, such as "deep", "muted", "pastel", "bright", "dark", and "colorblind", or you can create your own custom palette.

When using a pre-defined palette, you can specify the number of colors you want to use by passing in the desired number as the argument.

For example, using the "deep" palette and specifying 6 colors will return an array of 6 RGB color codes that can be used in your plot.

In [104…

```
deep_colors = sns.color_palette("deep", 6)

# Just to show the color palette using palplot
sns.palplot(deep_colors)
plt.show()
```



- You can also create your own custom color palette by passing in a list of RGB color codes.

In [105…
```
colors = ["#00CD00", "#00FFAA", "#FCCC00", "#FF0000"]
colors = sns.color_palette(colors)
sns.palplot(colors)
plt.show()
```

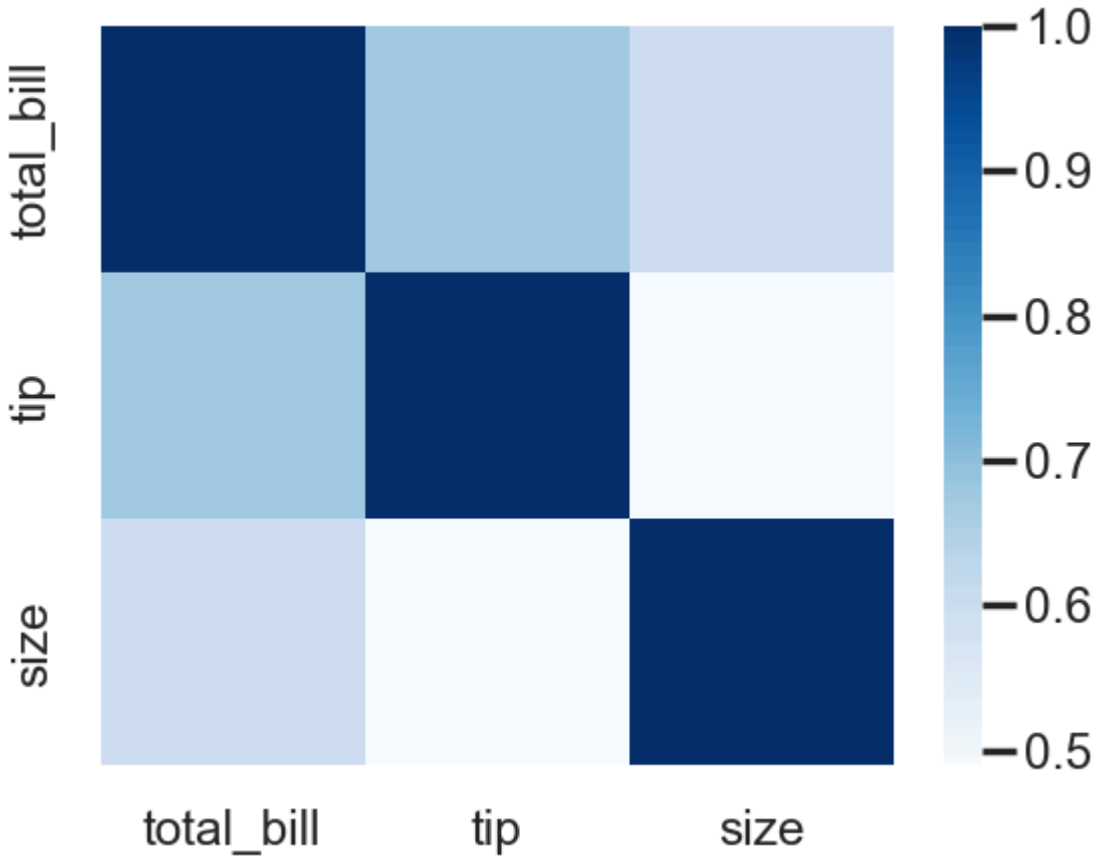


The `as_cmap` parameter in seaborn's `color_palette` function is a boolean flag that, when set to True, returns a colormap object instead of a list of RGB values. This can be useful when plotting data that needs to be colored based on a continuous variable, such as a heatmap or a 2D histogram. The colormap can then be passed to other plotting functions, such as heatmap or imshow, to color the plotted data. An example of using color_palette with as_cmap is:

In [106…
```
data = tips.corr(numeric_only=True)

# Create a colormap from a seaborn color palette
cmap = sns.color_palette("Blues", as_cmap=True)

# Create a heatmap using the colormap
sns.heatmap(data, cmap=cmap)
plt.show()
```



In [ ]:

In [108…
```
import pandas as pd

# Load the 'tips' dataset into a DataFrame
tips = sns.load_dataset('tips')

# Calculate correlation for numeric columns only
data = tips.corr(numeric_only=True)

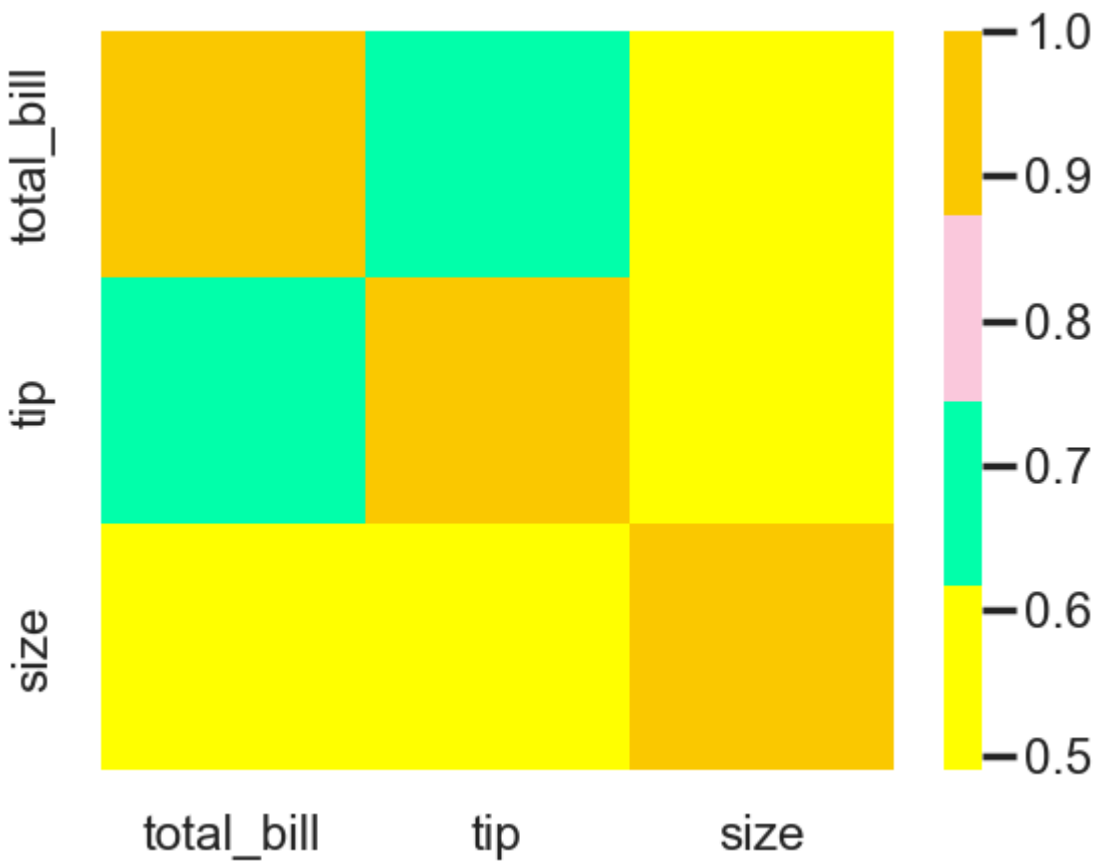
# Display the correlation matrix
print(data)
```

```
            total_bill       tip      size
total_bill    1.000000  0.675734  0.598315
tip           0.675734  1.000000  0.489299
size          0.598315  0.489299  1.000000
```

In [109…
```
tips = sns.load_dataset('tips')
data = tips.corr(numeric_only=True)

# Create a colormap using your own custom color code
colors = ["#FFFF00", "#00FFAA", "#FCCCDD", "#FCCC00"]
cmap = sns.color_palette(colors, as_cmap=True)

# Create a heatmap using the colormap
sns.heatmap(data, cmap=cmap)
plt.show()
```



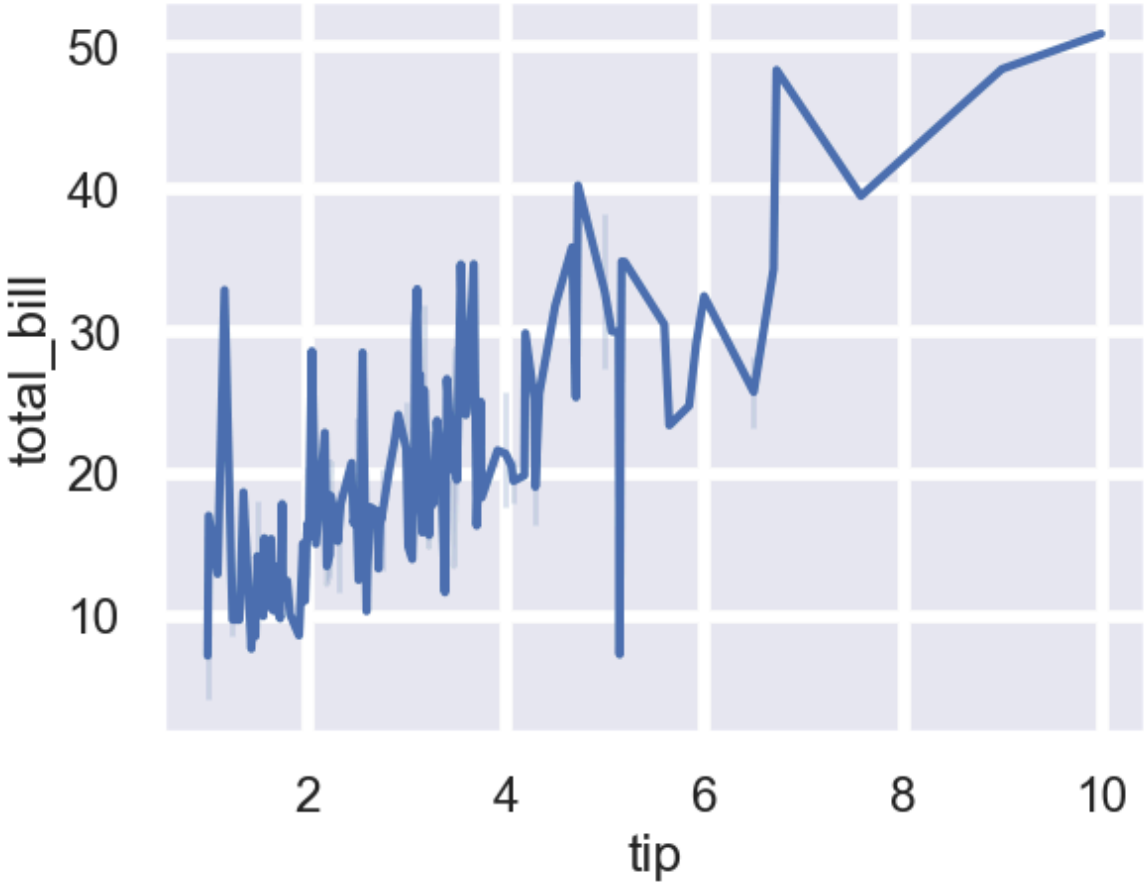
set_palette

The `set_palette()` function in seaborn allows you to specify a color palette for your plots. This can be done by passing in one of the pre-defined seaborn palettes (such as

"deep", "muted", "bright", etc.) or by passing in your own custom list of colors or color_palette.

Here is an example of using set_palette() to specify the "deep" palette:

In [110…
```
# This will set palette as deep
sns.set_palette('deep')
sns.lineplot(data=tips, x='tip', y='total_bill')
plt.show()
```



In [111…
```
# Loading different data
import plotly.express as px

gap = px.data.gapminder()
gap.head()
```

Out[111…

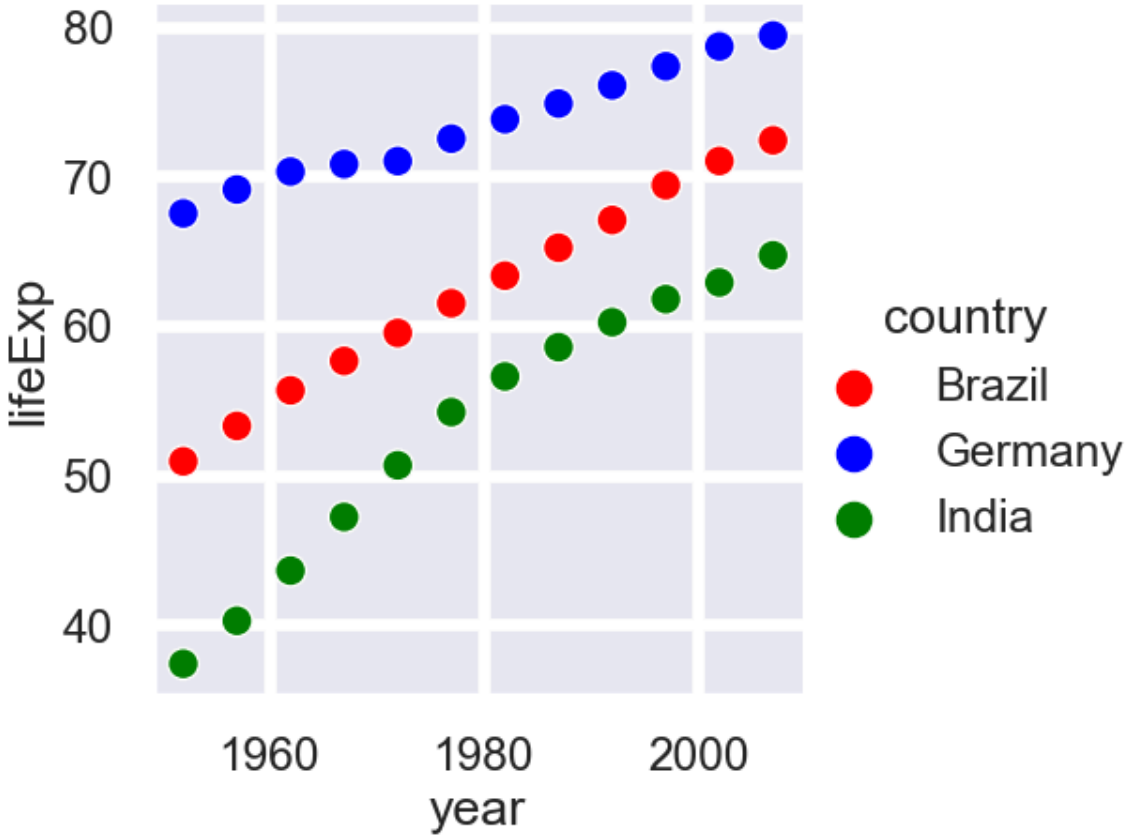
	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	4
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	4
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	4
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	4
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	4

- You can also pass in a custom list of colors. For example, the following code would set the palette to the colors red, blue, and green:

In [112…
```
temp_df = gap[gap['country'].isin(['India','Brazil','Germany'])]

# Seting palette to three colors
sns.set_palette(["red", "blue", "green"])
sns.relplot(data=temp_df, kind='scatter', x='year', y='lifeExp', hue='country')
```

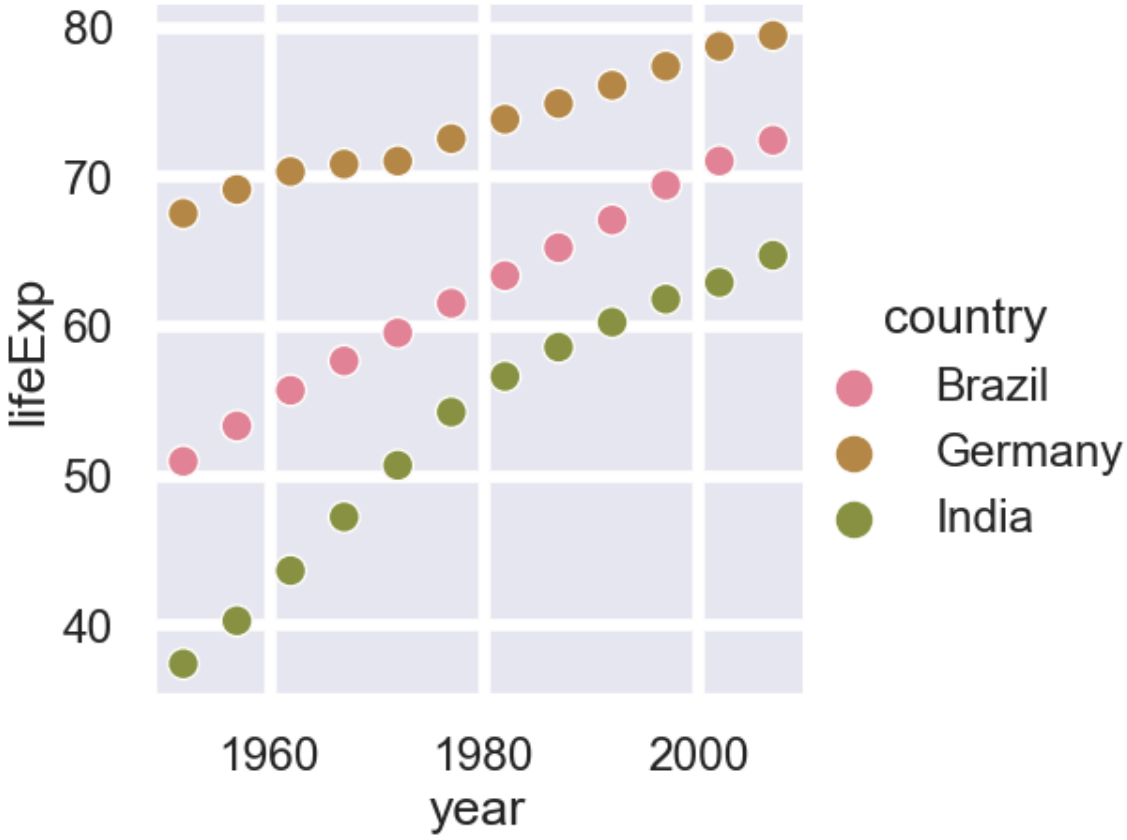
Out[112… <seaborn.axisgrid.FacetGrid at 0x26eacfa2050>



You can also pass in a number of different arguments to set_palette. For example, the following code sets the color palette to a specific hue, with 8 colors, and a desaturated lightness:

In [113…
```
sns.set_palette("husl", 8, .7)
sns.relplot(data=temp_df, kind='scatter', x='year', y='lifeExp', hue='country')
```

Out[113… <seaborn.axisgrid.FacetGrid at 0x26eacff2bd0>



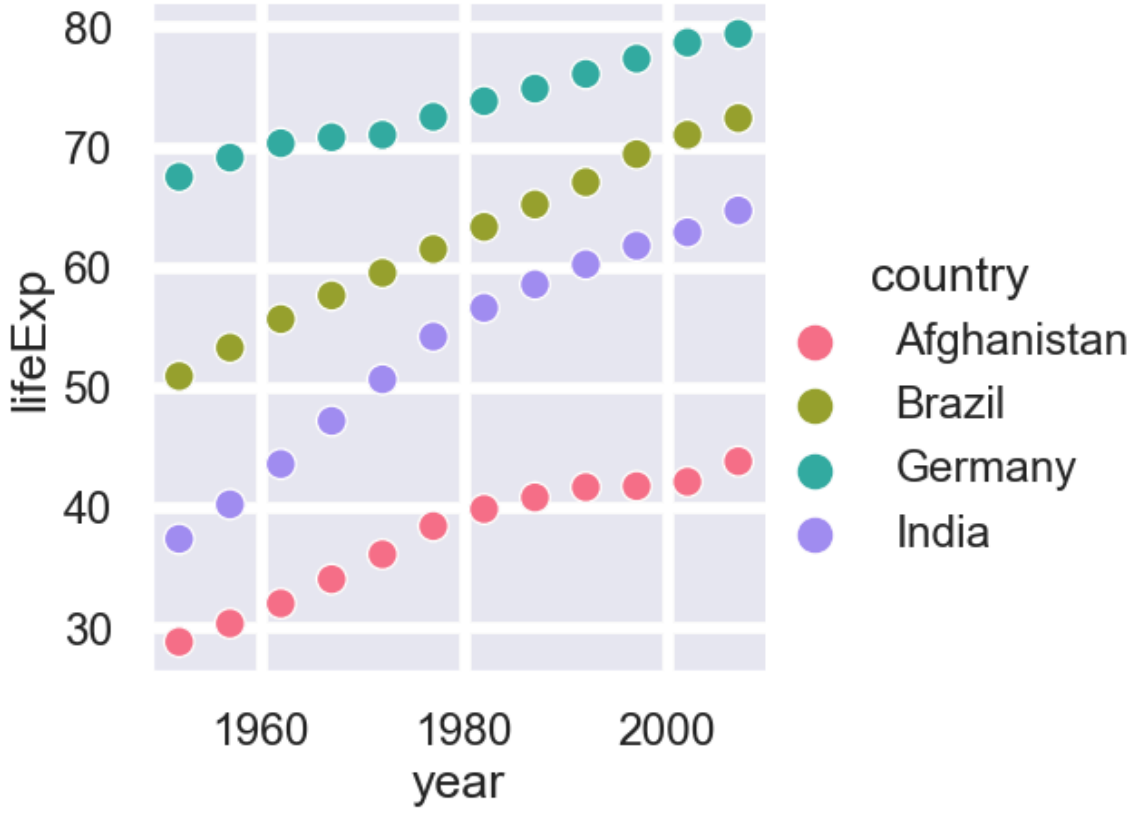
Now say we have set pallete colors and passed three colors, like we did above and want to plot of 4 or more country line plots?

What color will be assinged to 4th country? Let's see

In [115…
```
temp_df = gap[gap['country'].isin(['India','Brazil','Germany','Afghanistan'])]

# Seting palette to three colors
sns.set_palette(["red", "blue", "green"])
sns.relplot(data=temp_df, kind='scatter', x='year', y='lifeExp', hue='country')
```

Out[115…
```
<seaborn.axisgrid.FacetGrid at 0x26eacc10210>
```



See it took, color palette we set as - sns.set_palette("husl",8, .7), with eight colors. Even if we are specifying set palette as `['red', 'blue', 'green']`

In [116…

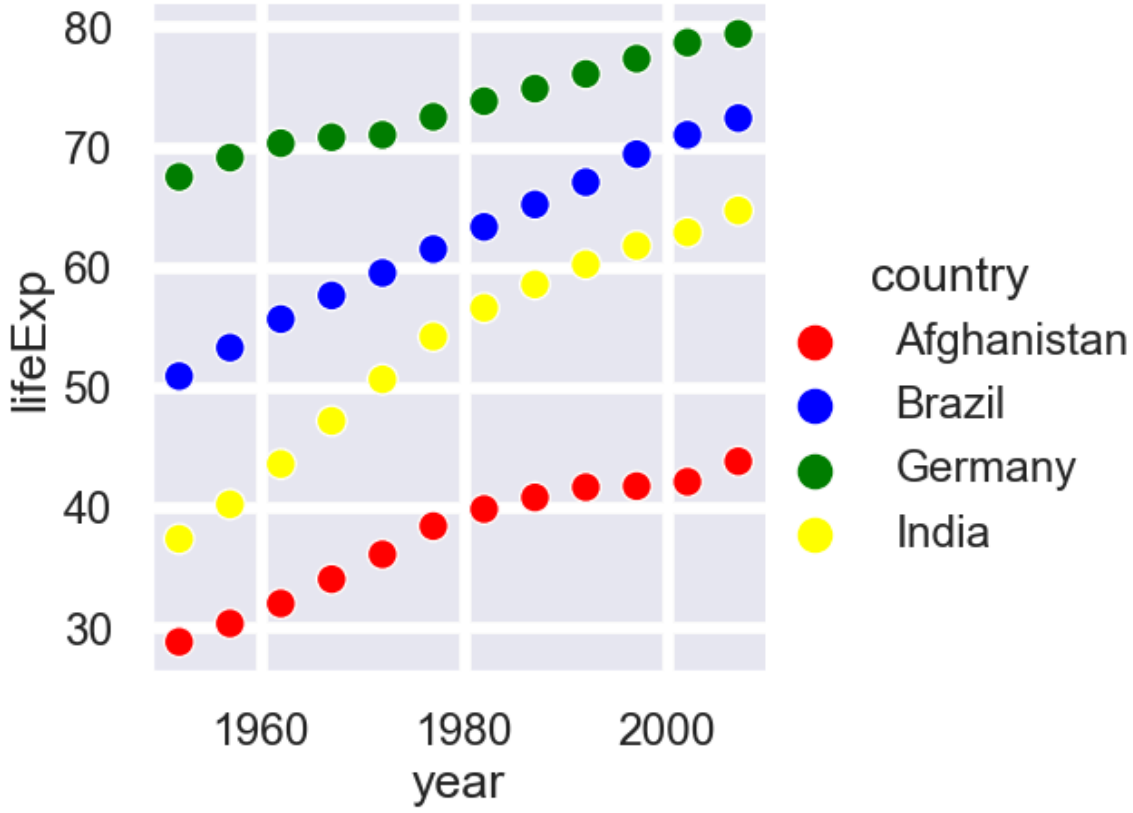
```
temp_df = gap[gap['country'].isin(['India','Brazil','Germany','Afghanistan'])]

# Seting palette to four colors
sns.set_palette(["red", "blue", "green", 'yellow'])
sns.relplot(data=temp_df, kind='scatter', x='year', y='lifeExp', hue='country')

# This will give right expected result as it has enough colors in the palette
#                                                                    to show.
```

Out[116…

```
<seaborn.axisgrid.FacetGrid at 0x26eacbecd50>
```



seaborn.husl_palette

`seaborn.husl_palette(n_colors=6, h=0.01, s=0.9, l=0.65, as_cmap=False)`

> Return hues with constant lightness and saturation in the HUSL system.
>
> The hues are evenly sampled along a circular path. The resulting palette will be appropriate for categorical or cyclical data.
>
> The `h`, `l`, and `s` values should be between 0 and 1.

Parameters:

> `n_colors` : `int` Number of colors in the palette.
>
> `h` : `float` (0-1) The value of the first hue.
>
> `l` : `float` (0-1) The lightness value.
>
> `s` : `float` (0-1) The saturation intensity.
>
> `as_cmap` : `bool` If True, return a matplotlib colormap object.

`sns.hls_palette()` > This function is similar to husl_palette(), but it uses a nonlinear color space that is more perceptually uniform, and saturation in the HLS system.

We can also use 'husl' or 'hsl' parameter in set_palette function for the same. Like we did in above example.

In [119…
```
iris = sns.load_dataset('iris')
```

cubehelix_palette

The seaborn.cubehelix_palette function is used to generate a colormap based on the cubehelix color scheme, which is a sequential color map with a linear increase in brightness and a smooth progression through the hues of the spectrum. This function takes several optional parameters such as `start`, `rot`, `gamma`, `light`, `dark`, `reverse` and `as_cmap` to control the properties of the color palette.

For example, the following code generates a cubehelix color palette with 8 colors, starting from a blue hue, and with increasing brightness and a rotation of 0.5:

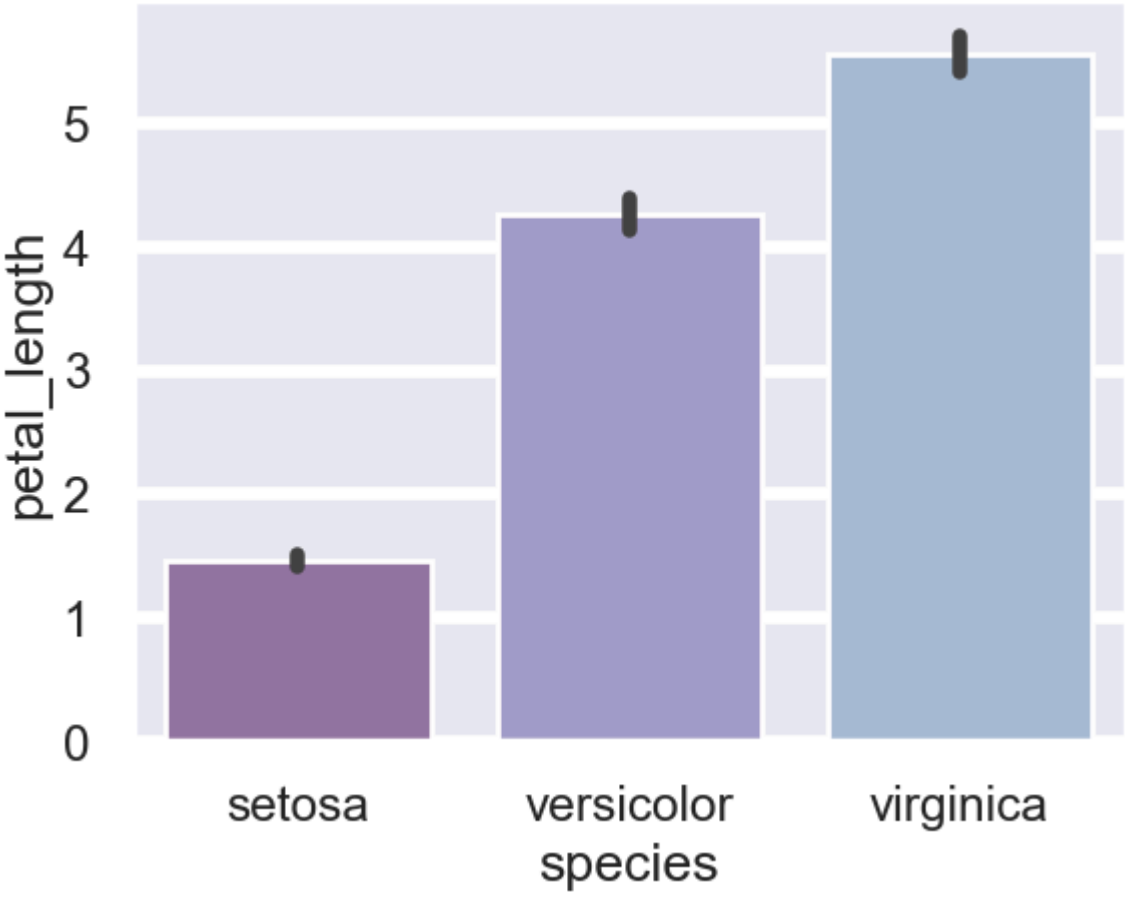
In [120…
```
colors = sns.cubehelix_palette(8, start=.5, rot=-.75, gamma=.3, light=.9,
                               dark=.1, reverse=True)
sns.palplot(colors)
```



This palette can be used to color various plotting elements such as bars, lines, and points in a graph.

In [121…
```
sns.barplot(x='species', y='petal_length', data=iris, palette=colors)
```

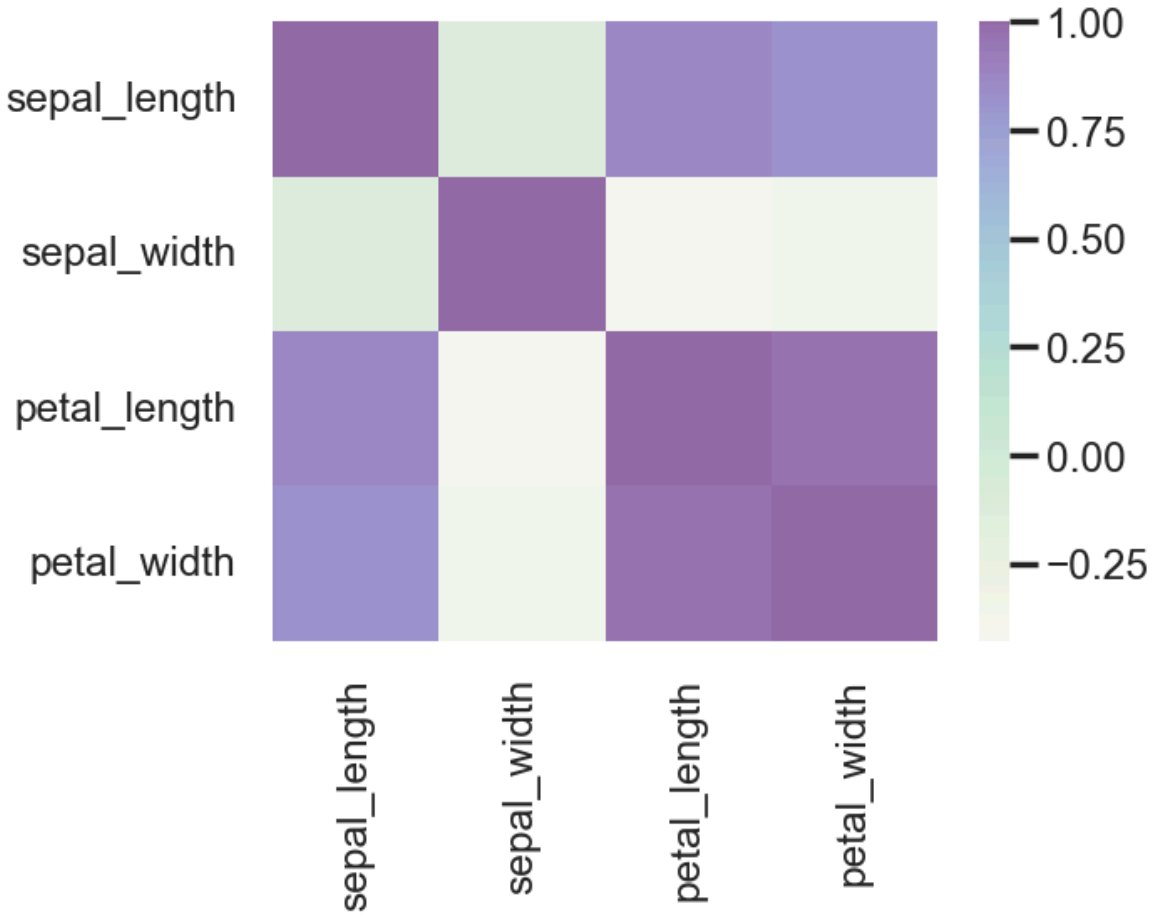
Out[121…
```
<Axes: xlabel='species', ylabel='petal_length'>
```



Alternatively, it can also be passed as a colormap to a heatmap or a 2D histogram.

In [122…
```
sns.heatmap(iris.corr(numeric_only=True
), cmap=sns.cubehelix_palette(8, start=.5, rot=-.75,
                                    gamma=.3, light=.9, dark=.1, as_cmap=True))
```

Out[122… <Axes: >



All those function mentioned in coloring have similar implementation.https://

Follow below links to read about specif palette - seaborn.pydata.org/api.html#color-palettes

TASK

In [123…
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("ggplot")
```

In [ ]:
```

```

In [124…
```
df = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/refs/he
```

In [125…
```
print(df.shape)
df.head()
```

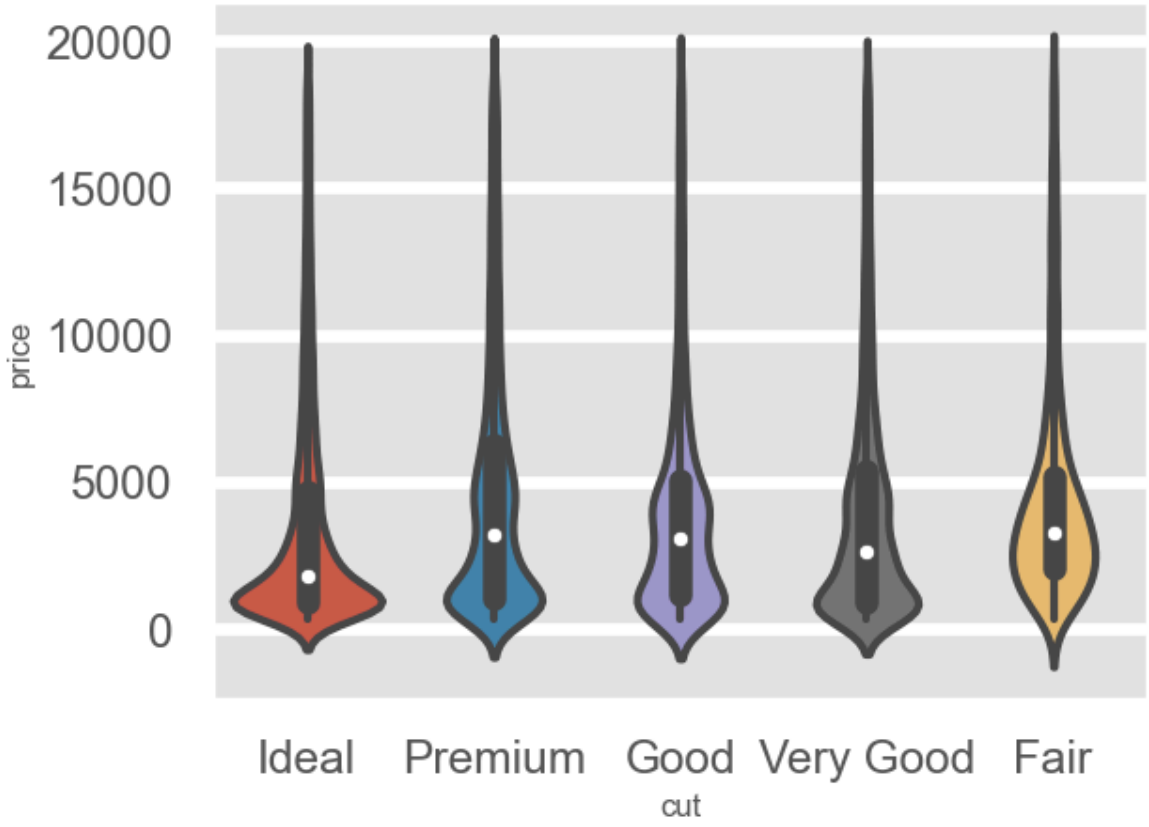
```
(53940, 10)
```

Out[125...

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

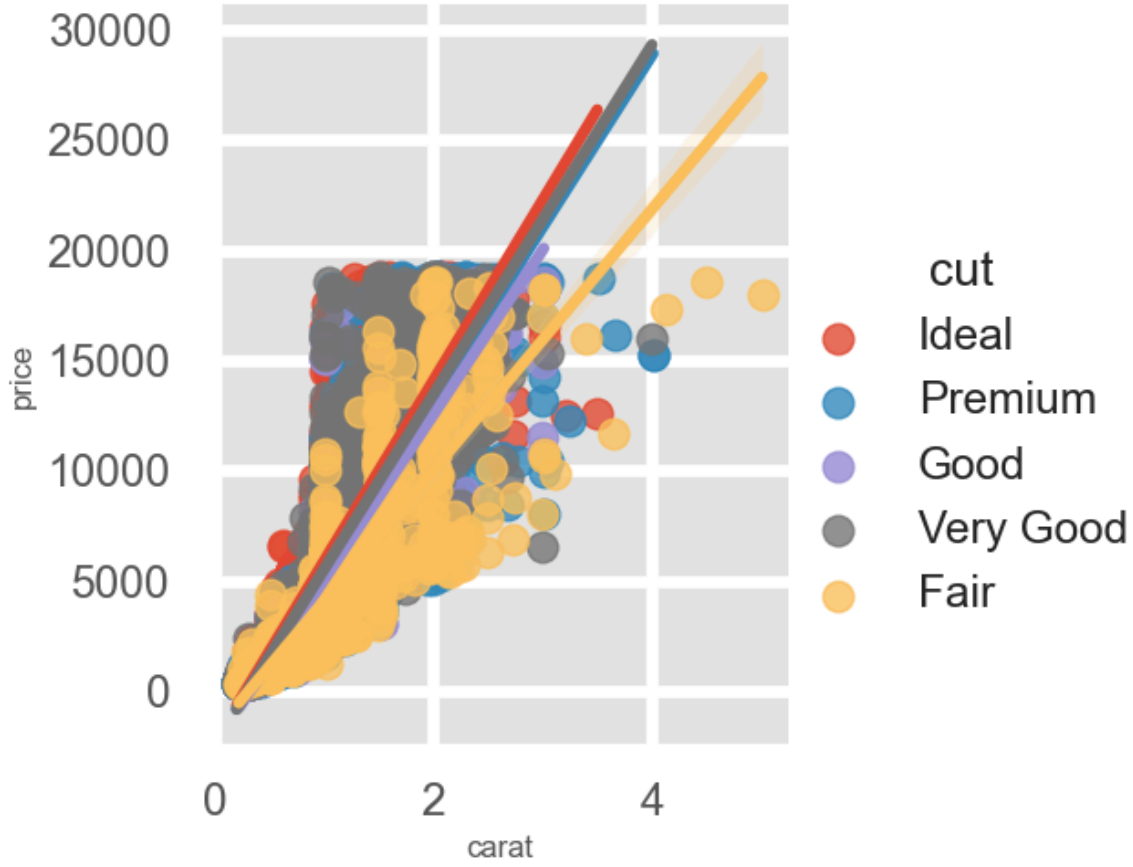
In [126...
```
sns.violinplot(data=df,x='cut',y='price')
```

Out[126...
```
<Axes: xlabel='cut', ylabel='price'>
```



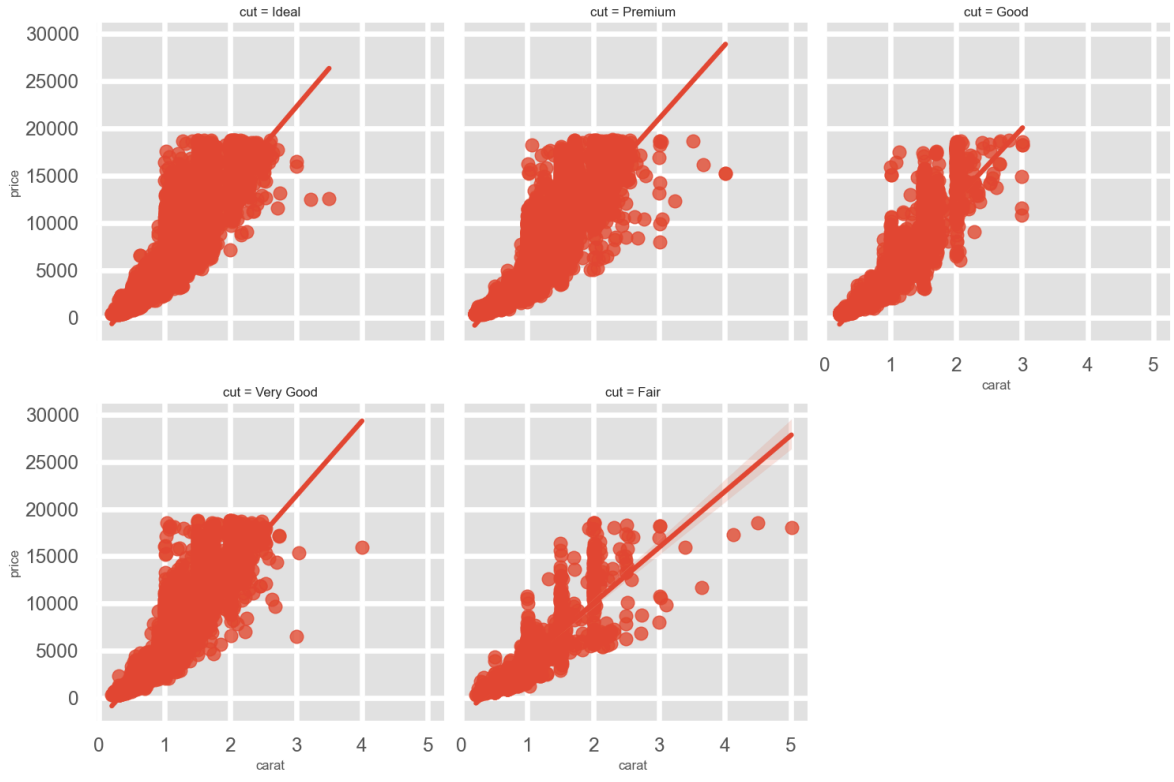
In [127...
```
sns.lmplot(data=df,x='carat',y='price',hue='cut')
```

Out[127...
```
<seaborn.axisgrid.FacetGrid at 0x26eaecc4810>
```



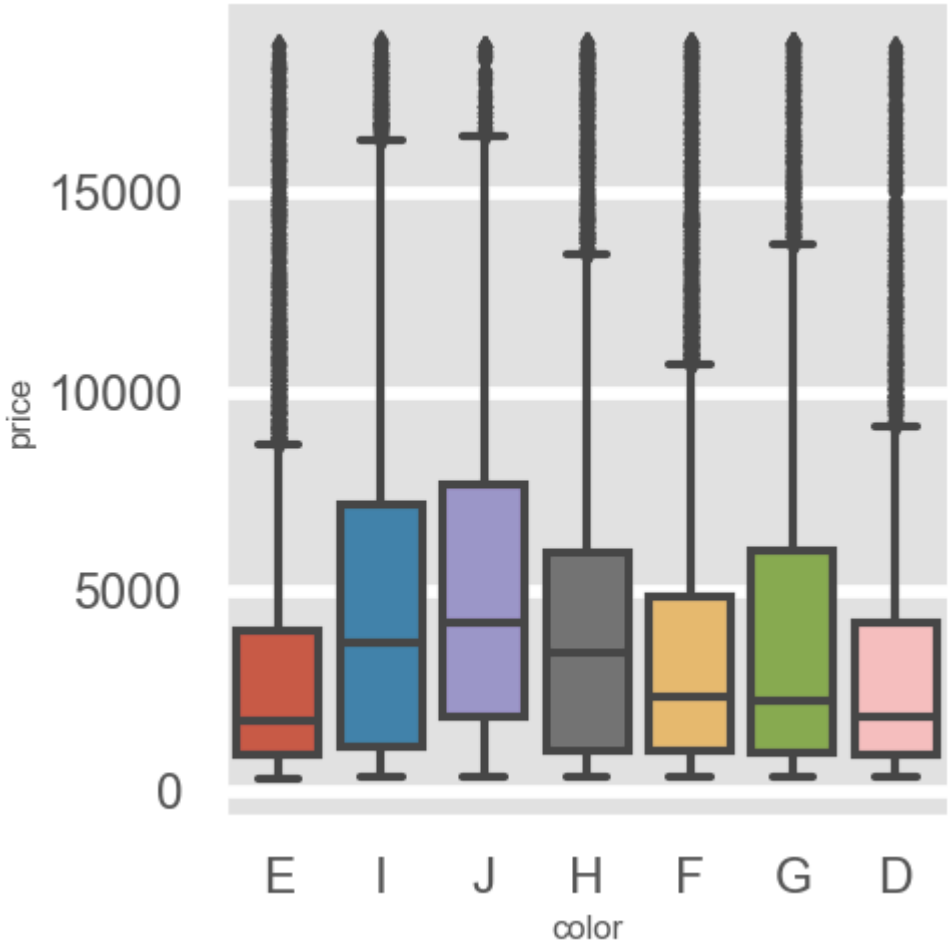
In [128…
```
sns.lmplot(data=df,x='carat',y='price',col='cut',col_wrap=3)
```

Out[128…
```
<seaborn.axisgrid.FacetGrid at 0x26eacc4a810>
```



In [129…
```
sns.catplot(data=df,x='color',y='price',kind='box')
```

Out[129…
```
<seaborn.axisgrid.FacetGrid at 0x26eaf6556d0>
```



In [132…
```
# code here
df = pd.read_csv("taxis_cleaned.csv")
```

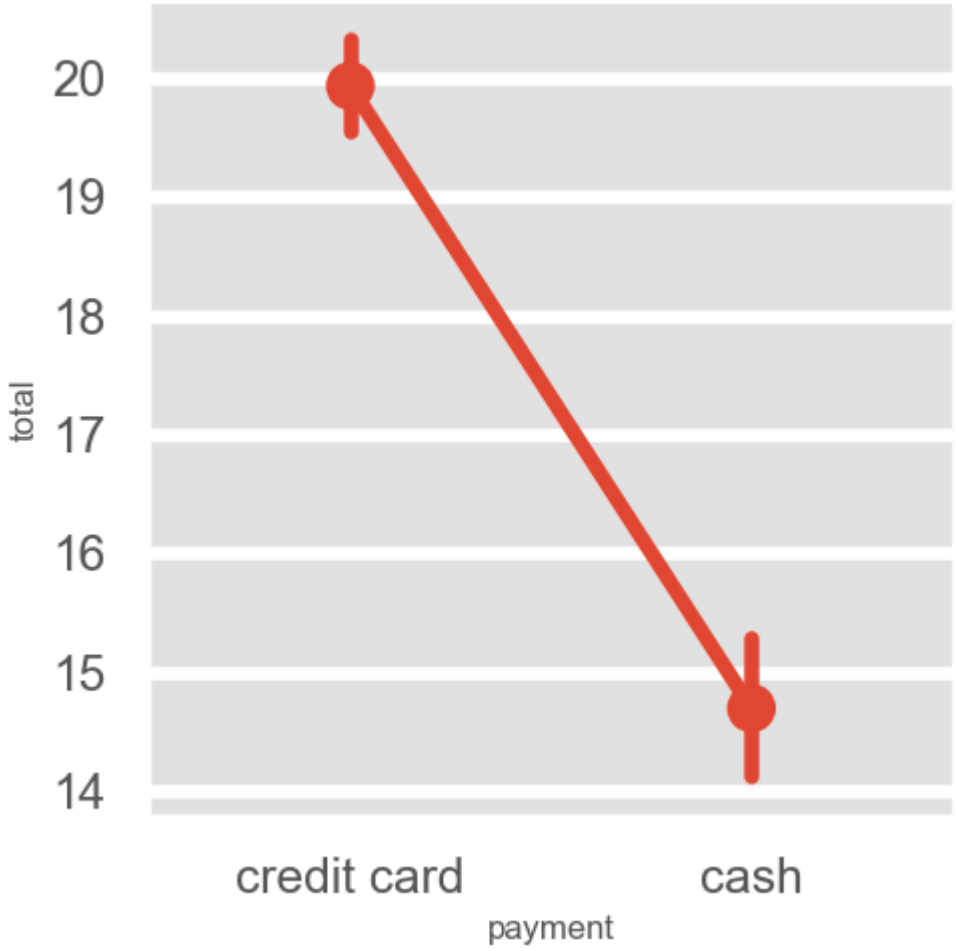
In [133…
```
df.head()
```

Out[133…

	pickup	dropoff	passengers	distance	fare	tip	tolls	total	color	payment	pi
0	2019-03-23 20:21:09	2019-03-23 20:27:24	1	1.60	7.0	2.15	0.0	12.95	yellow	credit card	
1	2019-03-04 16:11:55	2019-03-04 16:19:00	1	0.79	5.0	0.00	0.0	9.30	yellow	cash	
2	2019-03-27 17:53:01	2019-03-27 18:00:25	1	1.37	7.5	2.36	0.0	14.16	yellow	credit card	
3	2019-03-10 01:23:59	2019-03-10 01:49:51	1	7.70	27.0	6.15	0.0	36.95	yellow	credit card	
4	2019-03-30 13:27:42	2019-03-30 13:37:14	3	2.16	9.0	1.10	0.0	13.40	yellow	credit card	

In [134…
```
sns.catplot(data=df,x='payment',y='total',kind='point')
```

Out[134… <seaborn.axisgrid.FacetGrid at 0x26eaf7f9310>

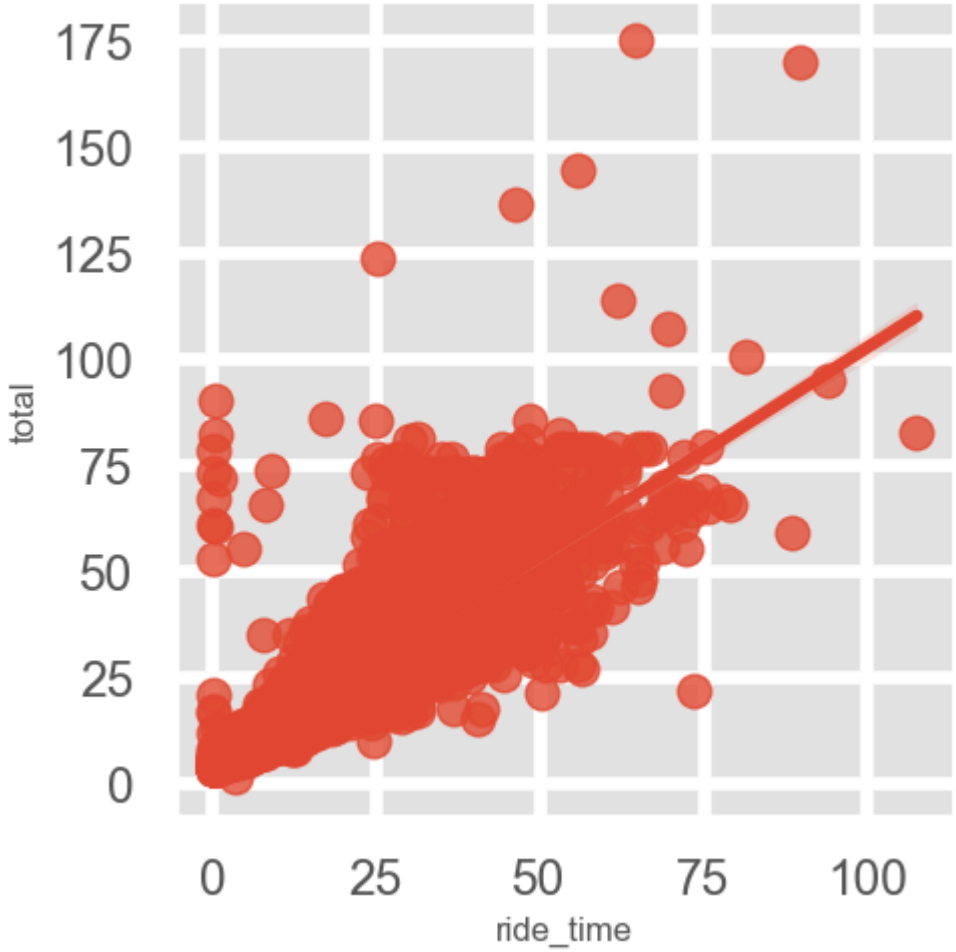


In [135…
```
df['ride_time'] = pd.to_datetime(df['dropoff']) - pd.to_datetime(df['pickup'])
```

In [136…
```
df['ride_time'] = round(df['ride_time'].dt.total_seconds()/60,2)
```

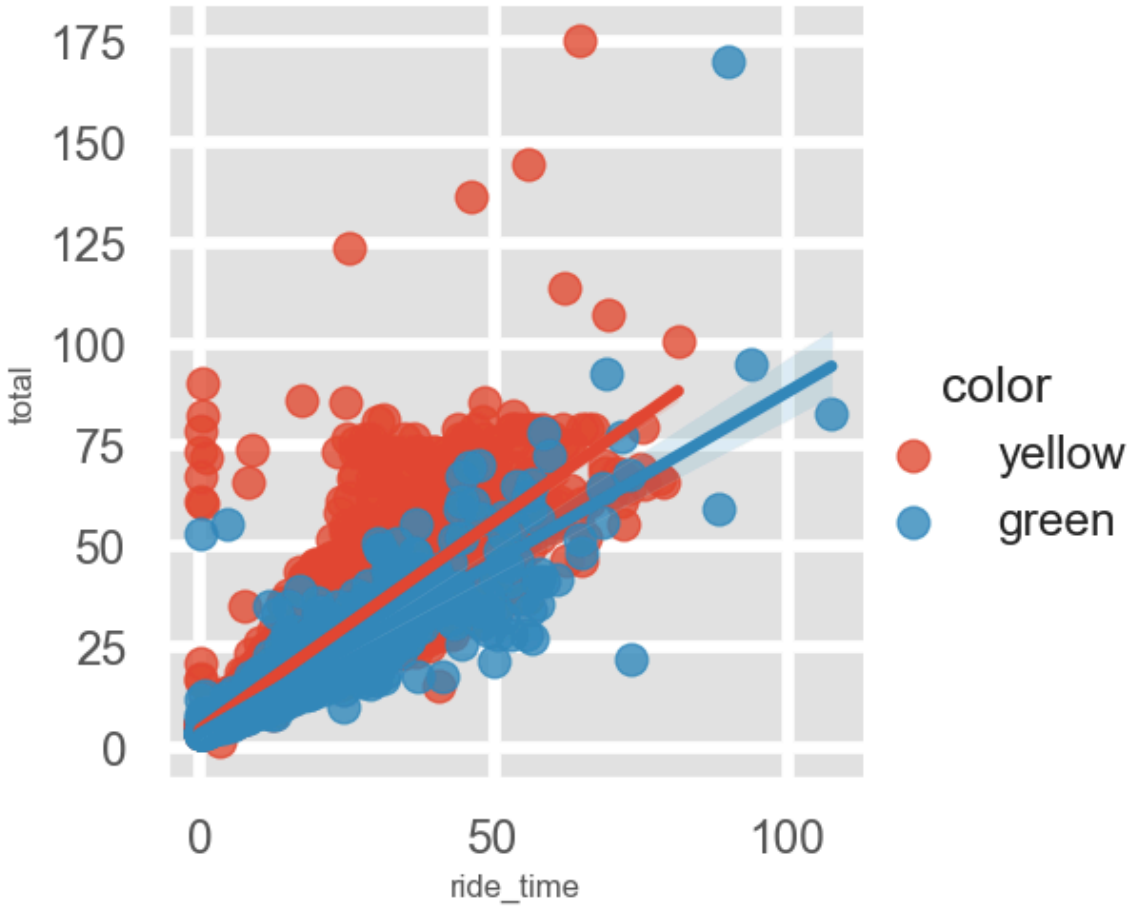
In [137…
```
sns.lmplot(data=df,x='ride_time',y='total')
```

Out[137… <seaborn.axisgrid.FacetGrid at 0x26eb3239750>



In [138…
```
sns.lmplot(data=df,x='ride_time',y='total',hue='color')
```

Out[138…
```
<seaborn.axisgrid.FacetGrid at 0x26eb329f5d0>
```



In [139…
```
sns.lmplot(data=df,x='ride_time',y='total',hue='payment')
```

Out[139…
```
<seaborn.axisgrid.FacetGrid at 0x26eb3319750>
```



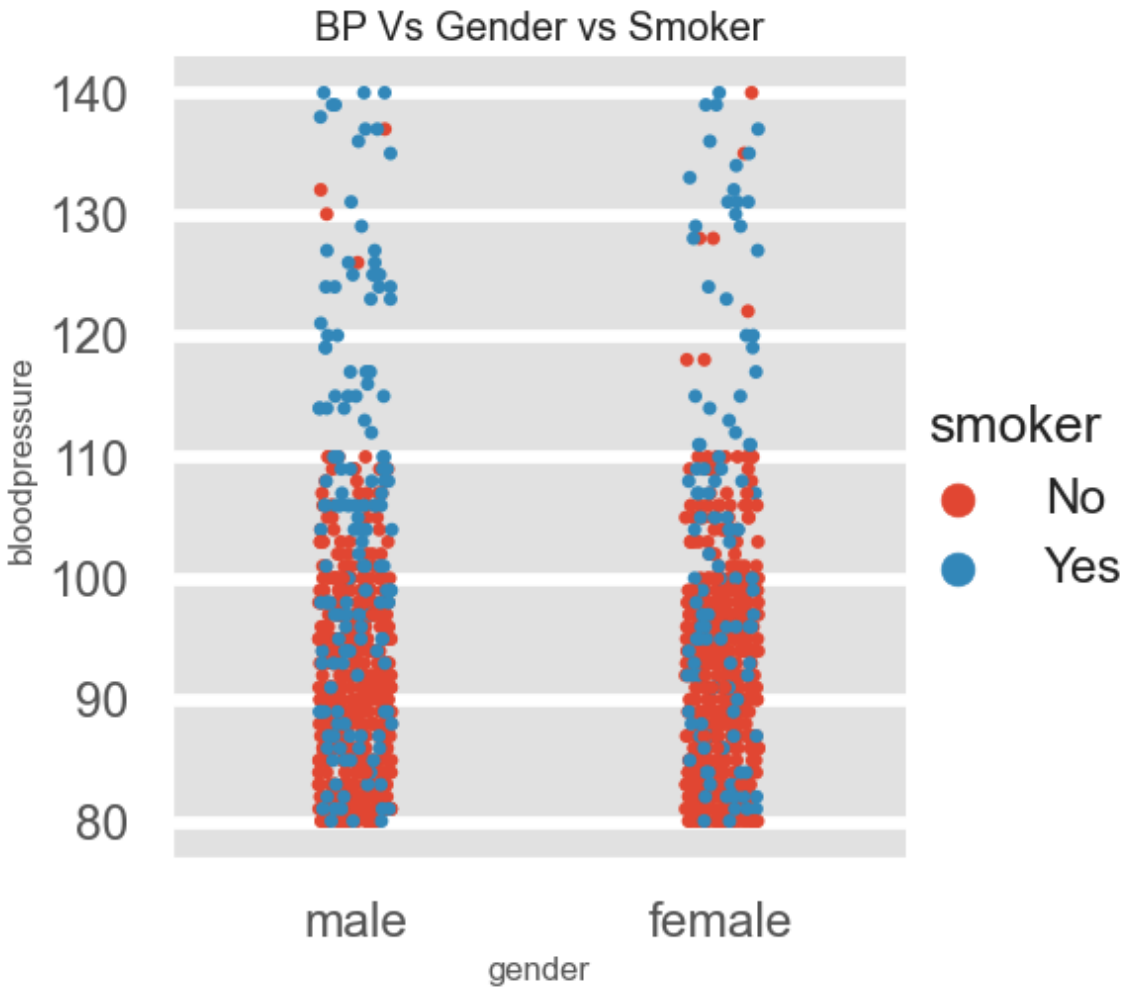
In [140…
```
# code here
df = pd.read_csv('https://docs.google.com/spreadsheets/d/e/2PACX-1vQVpcVtdYdZU4z
df.head()
```

Out[140…

	index	PatientID	age	gender	bmi	bloodpressure	diabetic	children	smoker	
0	0	1	39.0	male	23.2	91	Yes	0	No	sou
1	1	2	24.0	male	30.1	87	No	0	No	sou
2	2	3	NaN	male	33.3	82	Yes	0	No	sou
3	3	4	NaN	male	33.7	80	No	0	No	nor
4	4	5	NaN	male	34.1	100	No	0	No	nor

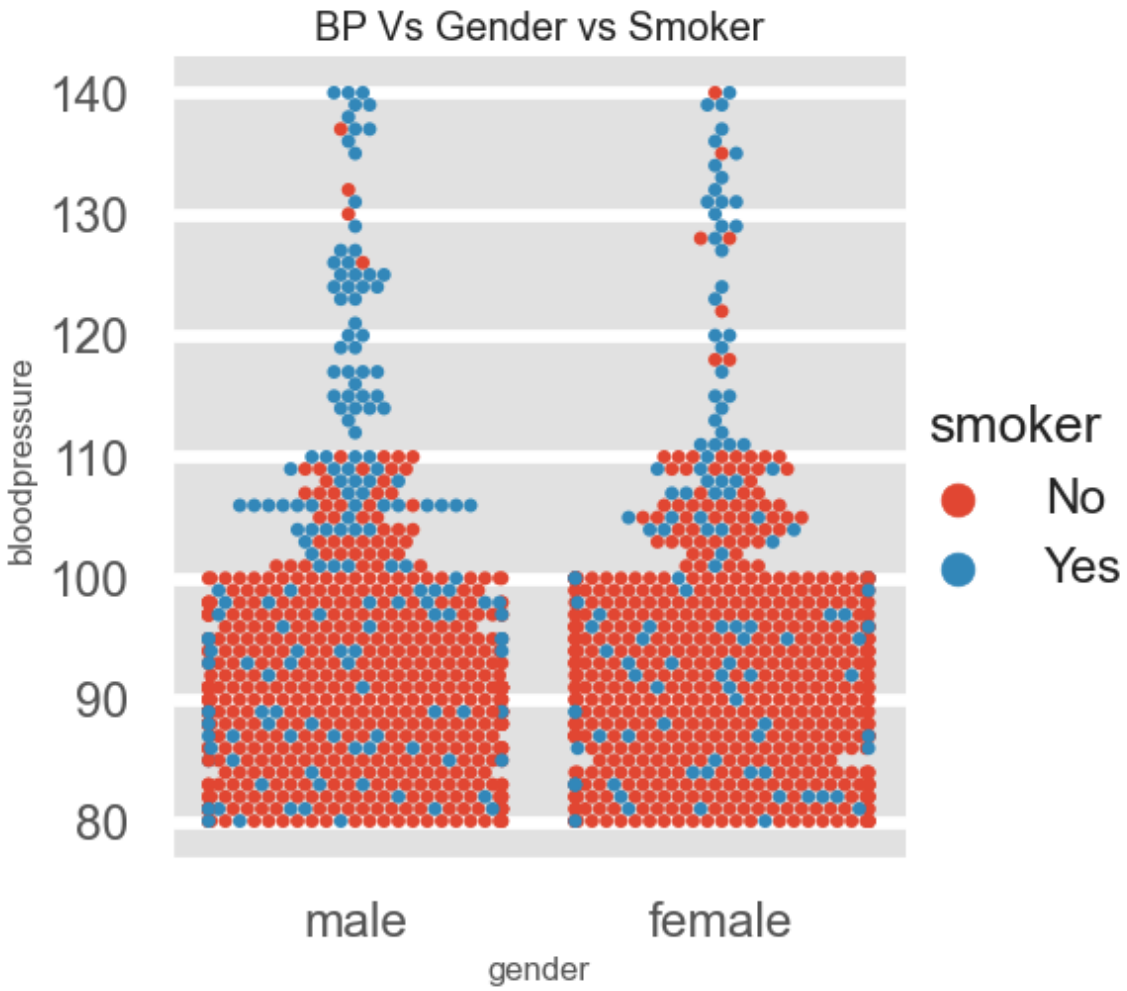
In [141…
```
sns.catplot(data=df,kind='strip',x='gender',y='bloodpressure',hue='smoker')
plt.title('BP Vs Gender vs Smoker')
```

Out[141… `Text(0.5, 1.0, 'BP Vs Gender vs Smoker')`



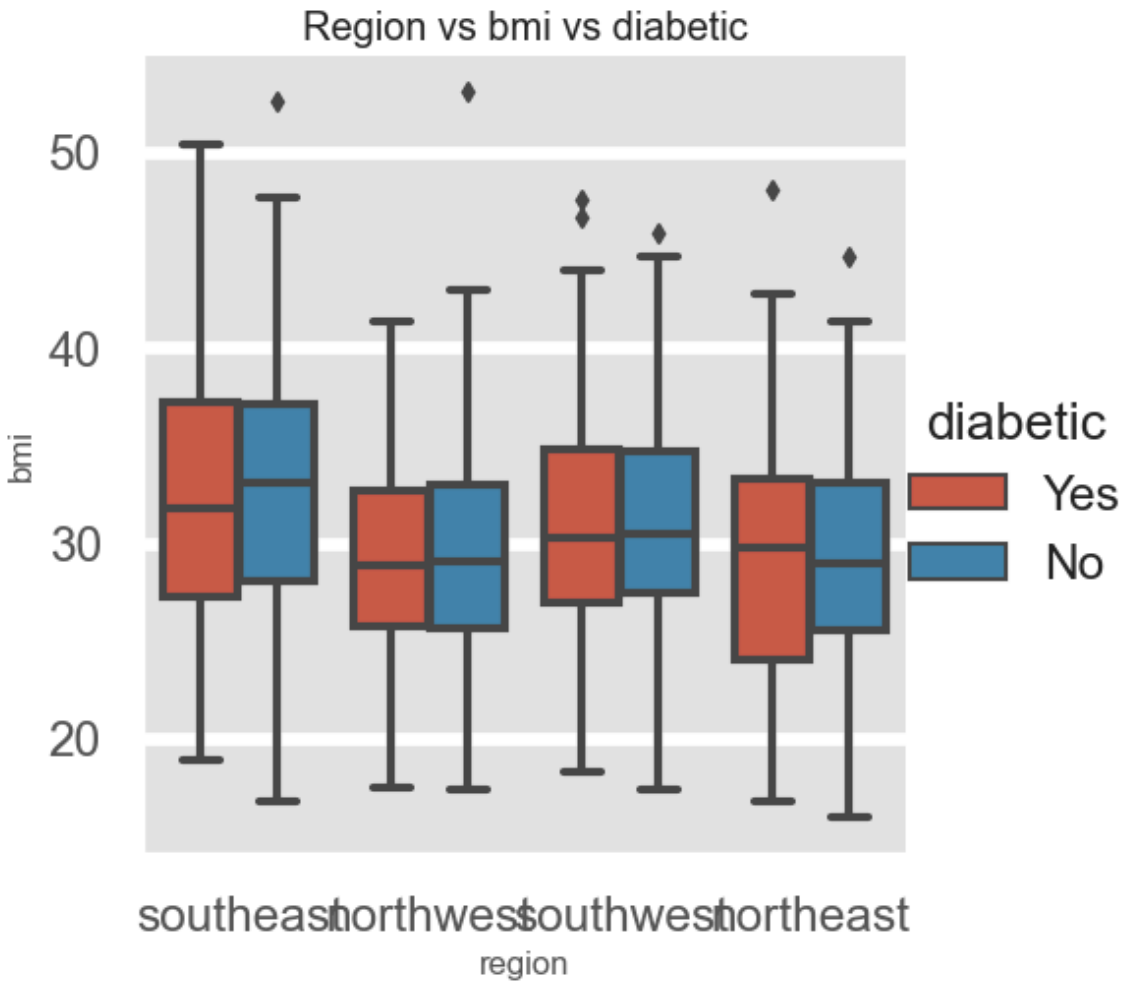
In [142…
```
sns.catplot(data=df,kind='swarm',x='gender',y='bloodpressure',hue='smoker')
plt.title('BP Vs Gender vs Smoker')
```

Out[142… `Text(0.5, 1.0, 'BP Vs Gender vs Smoker')`



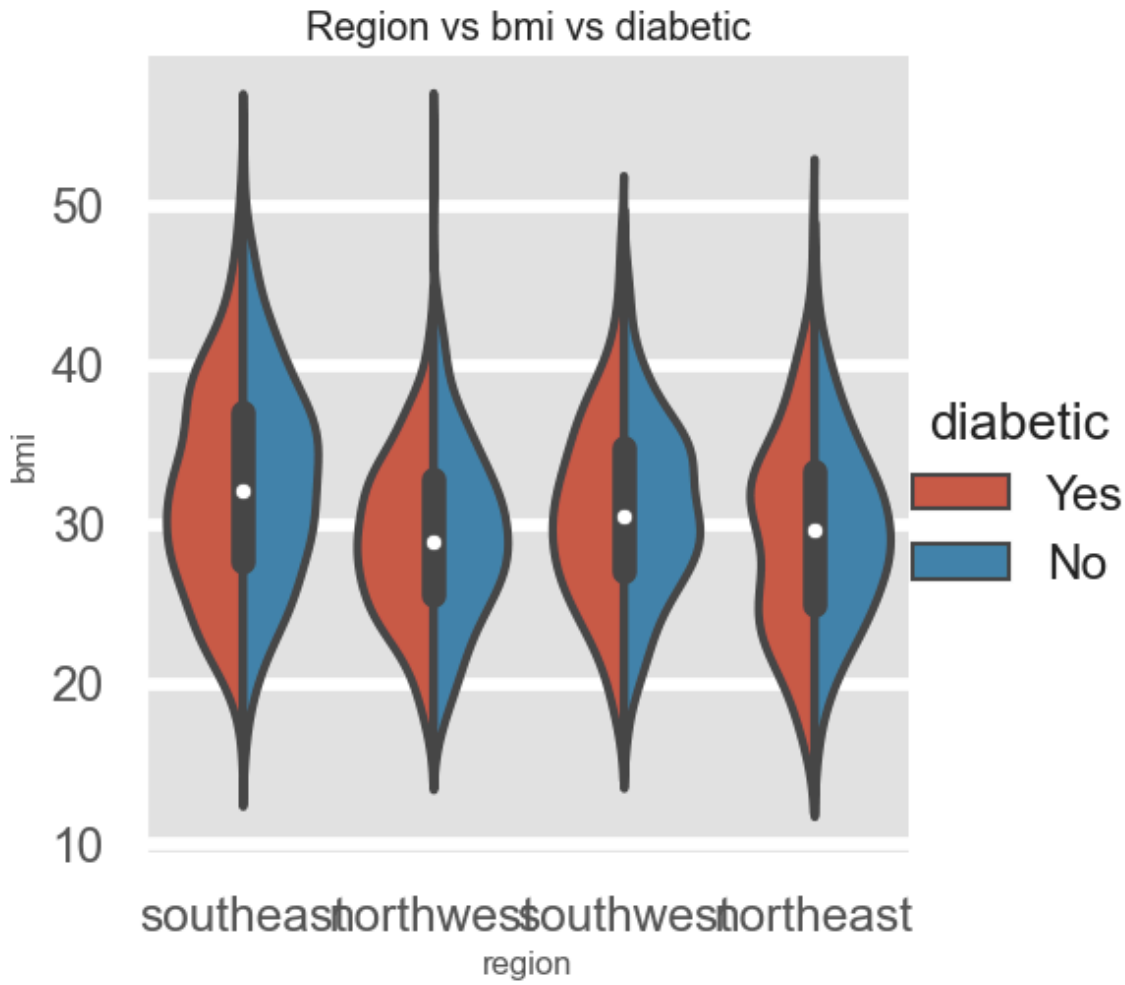
In [143…
```
# code here
sns.catplot(data = df, x = 'region', y = 'bmi', hue = 'diabetic', kind = 'box')
plt.title('Region vs bmi vs diabetic')
```

Out[143…
```
Text(0.5, 1.0, 'Region vs bmi vs diabetic')
```



In [144…
```
sns.catplot(data = df, x = 'region', y = 'bmi', hue = 'diabetic',
            kind = 'violin', split = True)
plt.title('Region vs bmi vs diabetic')
```

Out[144…
```
Text(0.5, 1.0, 'Region vs bmi vs diabetic')
```

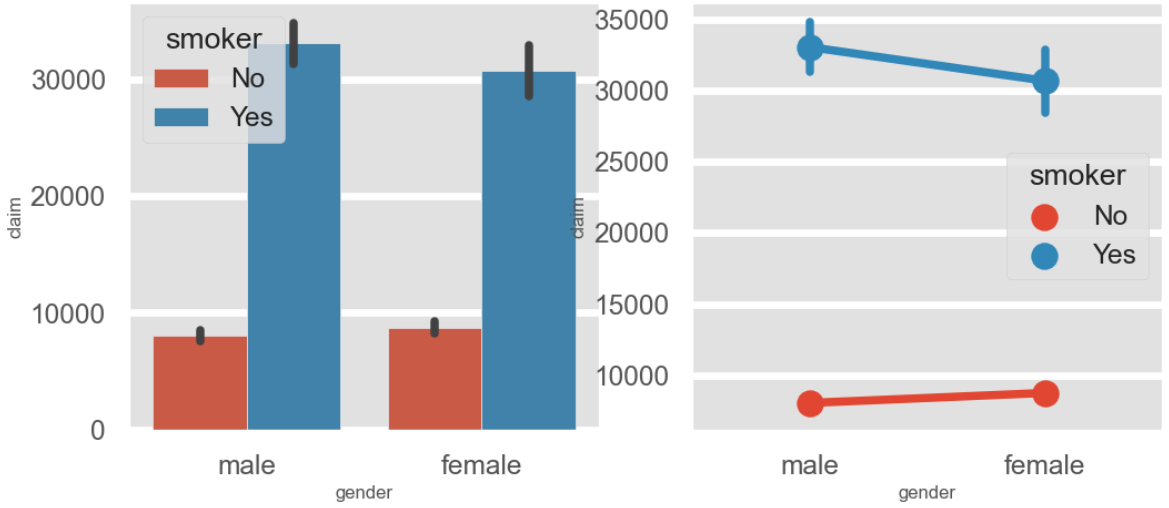


In [145…
```
# code here
fig, ax = plt.subplots(1,2,figsize=(12,5))

sns.barplot(x='gender',y='claim',hue='smoker',data=df,ax=ax[0])

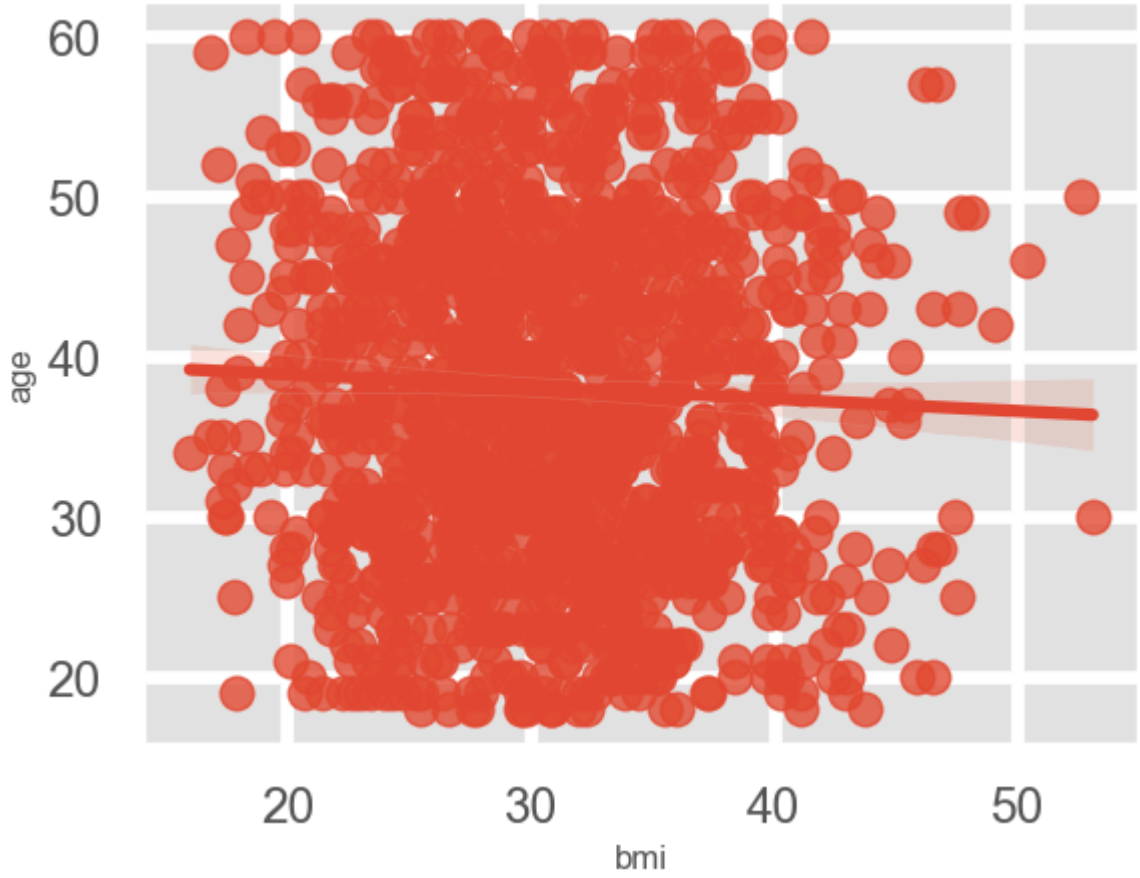
sns.pointplot(x='gender',y='claim',hue='smoker',data=df,ax=ax[1])

plt.show()
```



In [146…
```
# code here
sns.regplot(x='bmi', y='age', data=df)
```

Out[146… `<Axes: xlabel='bmi', ylabel='age'>`



In [147…
```
# code here
sns.pairplot(data=df.drop(columns=['index',     'PatientID']),hue='gender')
```

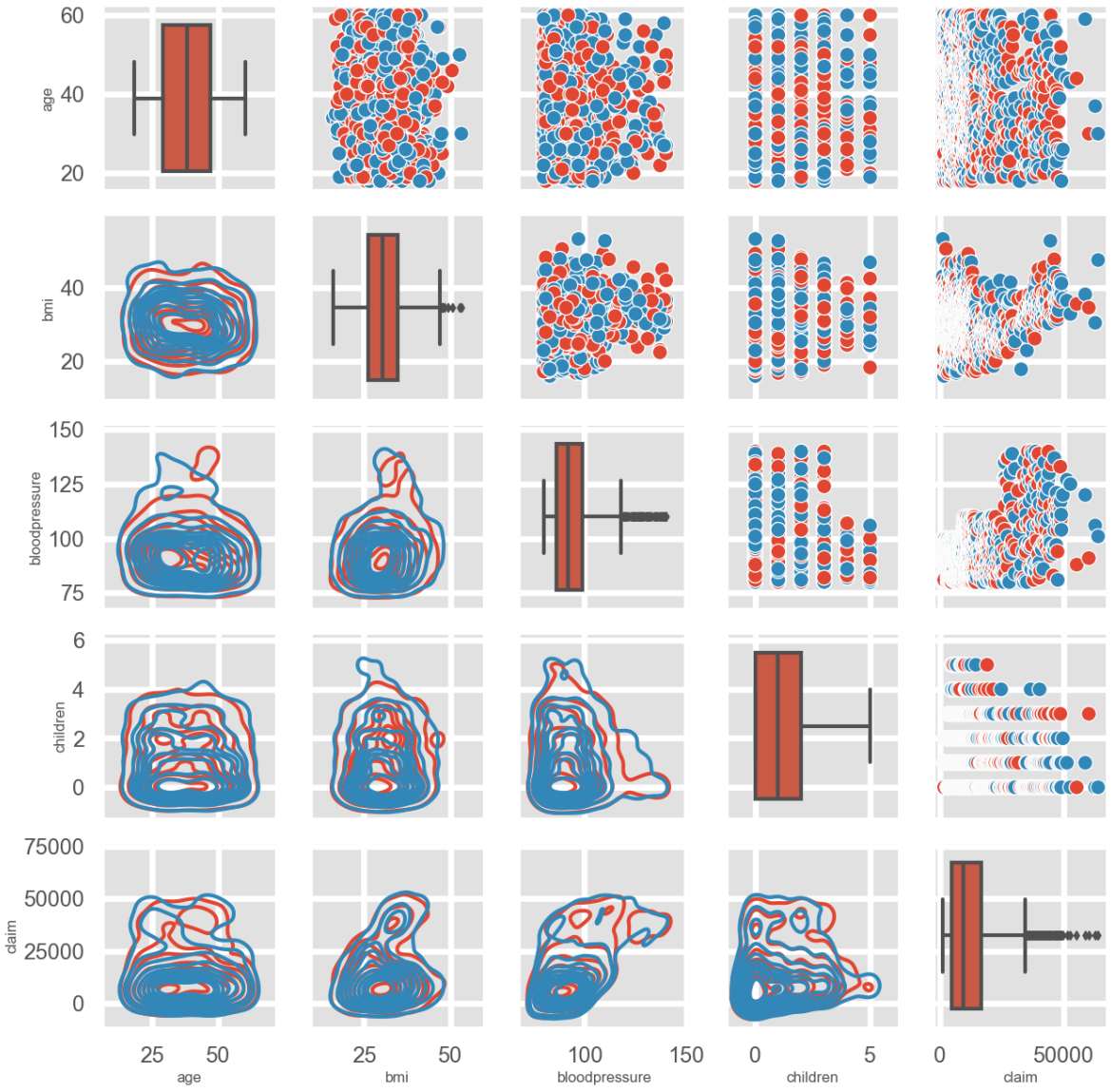
Out[147…
```
<seaborn.axisgrid.PairGrid at 0x26eb33cf3d0>
```



In [148…
```
# code here
g = sns.PairGrid(df.drop(columns=['index','PatientID']),hue='diabetic')

g.map_diag(sns.boxplot)
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot)
```

Out[148… <seaborn.axisgrid.PairGrid at 0x26eb5832350>



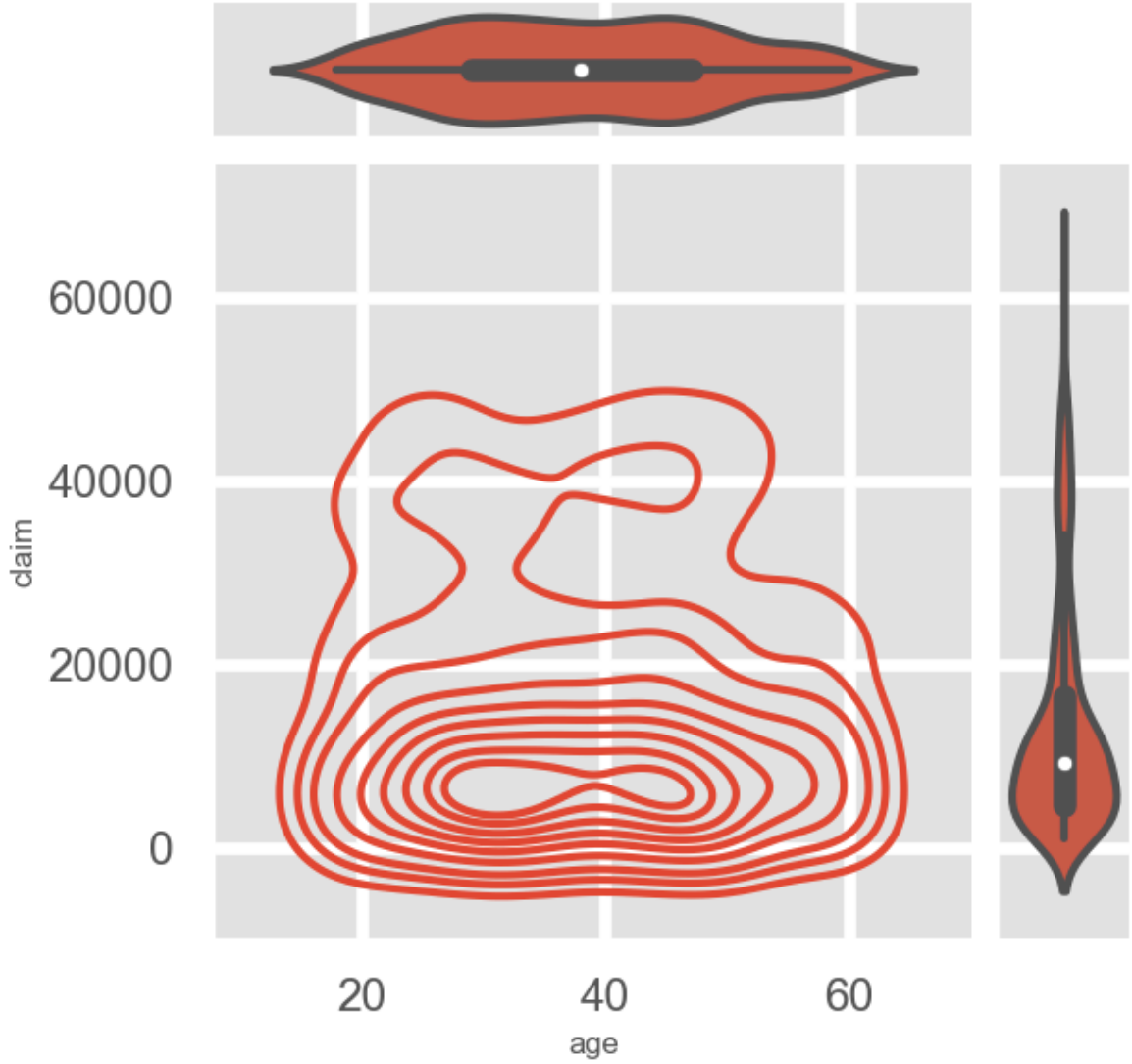
In [149…
```
# code here
sns.jointplot(x='bloodpressure', y='bmi', data=df, kind='scatter',hue='smoker')
```

Out[149…
```
<seaborn.axisgrid.JointGrid at 0x26eb78d64d0>
```



In [150…
```
# code here
g = sns.JointGrid(x='age',y='claim',data=df)
g.plot(sns.kdeplot,sns.violinplot)
```

Out[150…
```
<seaborn.axisgrid.JointGrid at 0x26eaf336f90>
```



In [ ]: